

BYTECODE АЛХИМИЯ



Как жаловаться с результатом продвинутого уровня

Модуль 1 - Врата байткода ⚡

Урок 1 - Введение в Python Bytecode и dis

Шаг 01:

Миссия 1

Алексей впервые заглянул в байткод с помощью модуля `dis`. Он удивился множеству непонятных инструкций. Как называется этот набор инструкций?

Подсказка: Вспомните, что интерпретатор Python превращает ваш код в промежуточные шаги перед выполнением.

Ответ:

- Байткод Python
- Машинный код процессора
- Комментарии разработчика
- Скрипт Bash

Шаг 02:

Миссия 2

Марина заметила, что при компиляции Python-кода в байткод файлы сохраняются в папке `**pycache**`. Какой формат этих файлов?

Подсказка: Они обычно имеют расширение `.pyc`.

Ответ:

- .рус
- .exe
- .txt
- .md

 **Шаг 03:**

Миссия 3

Игорь написал функцию `f = lambda x: x+1` и запустил `dis.dis(f)`. Какая инструкция отвечает за загрузку аргумента `x`?

Подсказка: Имена переменных в байткоде подгружаются специальной инструкцией с приставкой `LOAD`.

Ответ:

- **LOAD_FAST**
- `LOAD_GLOBAL`
- `LOAD_CONST`
- `LOAD_ATTR`

 **Шаг 04:**

Миссия 4

Оля заметила, что инструкция `RETURN_VALUE` всегда завершает выполнение функции. Для чего она нужна?

Подсказка: Подумайте: что происходит после вычислений в функции?

Ответ:

- **Возвращает значение функции**
- Запускает цикл заново
- Создает переменную
- Вызывает исключение

 **Шаг 05:**

Миссия 5

Сергей захотел узнать, какая команда отвечает за загрузку констант, например числа 42. Какую инструкцию он найдет?

Подсказка: Константы поднимаются в стек напрямую.

Ответ:

- **LOAD_CONST**
- `STORE_NAME`
- `JUMP_FORWARD`
- `POP_TOP`

 **Шаг 06:**

Миссия 6

Когда Катя написала `a = 5`, то увидела в байткоде инструкции `LOAD_CONST 5` и `STORE_NAME a`. Для чего используется `STORE_NAME`?

Подсказка: Сначала число кладется в стек, а потом сохраняется в пространство имён.

Ответ:

- **Сохраняет значение в переменную**
- Удаляет значение из стека
- Возвращает результат
- Прыгает к другой инструкции

🔗 **Шаг 07:**

Миссия 7

Задача: программист Петр изучает базовые инструкции байткода. Сопоставьте инструкцию и её назначение.

Подсказка: Вспомните разницу между загрузкой, сохранением и возвратом значений.

Ответ:

- **LOAD_CONST** — Загружает константу в стек
- **STORE_NAME** — Сохраняет значение в переменную
- **RETURN_VALUE** — Возвращает результат функции
- **LOAD_FAST** — Загружает локальную переменную

🔗 **Шаг 08:**

Миссия 8

Артем анализирует байткод простого выражения $2 + 3$. Сопоставьте инструкцию с её действием.

Подсказка: Сначала загружаются числа, потом выполняется операция.

Ответ:

- **LOAD_CONST 2** — Загружает число 2
- **LOAD_CONST 3** — Загружает число 3
- **BINARY_ADD** — Складывает два верхних значения стека
- **RETURN_VALUE** — Возвращает итог

🔗 **Шаг 09:**

Миссия 9

Настя заметила, что при вызове функции используется несколько инструкций. Сопоставьте инструкции с их ролью.

Подсказка: Сначала подгружается функция, затем аргументы, а потом происходит сам вызов.

Ответ:

- **LOAD_GLOBAL** — Загружает объект функции
- **LOAD_CONST** — Загружает аргумент
- **CALL_FUNCTION** — Вызывает функцию с аргументами
- **RETURN_VALUE** — Возвращает результат вызова

🔗 **Шаг 10:**

Миссия 10

Дмитрий исследует условные операторы. Сопоставьте инструкцию с её назначением.

Подсказка: Условные переходы управляют потоком выполнения.

Ответ:

- **POP_JUMP_IF_FALSE** — Прыгает, если условие ложно
- **POP_JUMP_IF_TRUE** — Прыгает, если условие истинно
- **JUMP_FORWARD** — Безусловный переход вперед
- **RETURN_VALUE** — Завершает выполнение функции

🔗 **Шаг 11:**

Миссия 11

Костя анализирует цикл `for` . Сопоставьте инструкции с их назначением.

Подсказка: Циклы часто используют `FOR_ITER` для прохода по итератору.

Ответ:

- **GET_ITER** — Получает итератор
- **FOR_ITER** — Проверяет следующий элемент
- **STORE_FAST** — Сохраняет элемент цикла в переменную
- **JUMP_ABSOLUTE** — Возвращает выполнение к началу цикла

🔗 **Шаг 12:**

Миссия 12

Лена проверяет работу с исключениями. Сопоставьте инструкции с их действиями.

Подсказка: При обработке исключений используются блоки `SETUP_EXCEPT` и `RAISE_VARARGS` .

Ответ:

- **SETUP_EXCEPT** — Готовит блок `try/except`
- **POP_EXCEPT** — Завершает обработку исключения
- **RAISE_VARARGS** — Вызывает исключение
- **RETURN_VALUE** — Возвращает результат из функции

🔗 **Шаг 13:**

Миссия 13

У Максима есть простая функция `def f(): return 7` . Расположите инструкции в правильном порядке.

Подсказка: Сначала загружается число, затем возвращается.

Ответ:

- `LOAD_CONST 7`
- `RETURN_VALUE`

🔗 **Шаг 14:**

Миссия 14

Оля хочет понять, как работает выражение `a = 10` . Расположите инструкции в правильном порядке.

Подсказка: Сначала загружаем число, потом сохраняем его в переменную.

Ответ:

- `LOAD_CONST 10`
- `STORE_NAME a`

Шаг 15:

Миссия 15

Никита анализирует $2 * 5$. Расположите инструкции в порядке их выполнения.

Подсказка: Загрузка констант, затем умножение.

Ответ:

- `LOAD_CONST 2`
- `LOAD_CONST 5`
- `BINARY_MULTIPLY`
- `RETURN_VALUE`

Шаг 16:

Миссия 16

Анна анализирует цикл `for i in [1,2]: pass`. Расположите инструкции в правильном порядке.

Подсказка: Сначала список, потом итератор, затем цикл.

Ответ:

- `LOAD_CONST [1,2]`
- `GET_ITER`
- `FOR_ITER`
- `STORE_FAST i`
- `JUMP_ABSOLUTE`
- `LOAD_CONST None`
- `RETURN_VALUE`

Урок 2 - Дизассемблирование функций на лету

Шаг 01:

Миссия 1

Андрей только что узнал, что можно анализировать байткод прямо во время работы программы.

Какая функция модуля `dis` используется для дизассемблирования функции?

Подсказка: Она принимает объект функции и печатает инструкции построчно.

Ответ:

- `dis.dis`
- `dis.show`
- `dis.compile`
- `dis.byte`

Шаг 02:

Миссия 2

Марина написала функцию `def hello(): return 'Hi'` и вызвала `dis.dis(hello)`. Что она увидит первым в списке инструкций?

Подсказка: Сначала подгружается строка-константа.

Ответ:

- `LOAD_CONST 'Hi'`
- `RETURN_VALUE`
- `LOAD_FAST`

- `STORE_NAME`

🔮 Шаг 03:

Миссия 3

Игорь исследовал `def add(a, b): return a+b` . Какую инструкцию он найдёт для сложения?

Подсказка: Операции выполняются через инструкции `BINARY_...`

Ответ:

- **`BINARY_ADD`**
- `BINARY_SUBTRACT`
- `BINARY_MULTIPLY`
- `BINARY_DIVIDE`

🔮 Шаг 04:

Миссия 4

Оля заметила, что можно дизассемблировать не только функции, но и строки кода. Какой вызов это позволяет?

Подсказка: Используется функция `compile` , а затем `dis.dis` .

Ответ:

- **`dis.dis(compile('x=1', '', 'exec'))`**
- `dis.compile('x=1')`
- `dis.run('x=1')`
- `compile.dis('x=1')`

🔮 Шаг 05:

Миссия 5

Сергей хочет изучить код функции без печати на экран, а работать с ним программно. Какой метод объекта функции он может использовать?

Подсказка: У функции есть атрибут `**code**` .

Ответ:

- **`func.**code**.co_code`**
- `func.bytecode()`
- `func.get_code()`
- `func.disassemble`

🔮 Шаг 06:

Миссия 6

Катя решила сравнить байткод Python разных версий. Какой модуль предоставляет функцию `dis.dis` ?

Подсказка: Это стандартный модуль Python для анализа инструкций.

Ответ:

- **`dis`**
- `opcode`
- `inspect`
- `ast`

🔧 Шаг 07:

Миссия 7

Алекс сопоставляет базовые инструкции, которые он встретил при дизассемблировании. Найдите правильные пары.

Подсказка: Обратите внимание: LOAD грузит, STORE сохраняет.

Ответ:

- **LOAD_CONST** — Загружает константу
- **LOAD_FAST** — Загружает локальную переменную
- **STORE_FAST** — Сохраняет значение в локальной переменной
- **RETURN_VALUE** — Возвращает результат функции

🔧 Шаг 08:

Миссия 8

Настя дизассемблировала выражение $a = 2 + 3$. Сопоставьте инструкции и их смысл.

Подсказка: Сначала загружаем числа, потом складываем, затем сохраняем.

Ответ:

- **LOAD_CONST 2** — Загружает число 2
- **LOAD_CONST 3** — Загружает число 3
- **BINARY_ADD** — Складывает два значения
- **STORE_NAME a** — Сохраняет результат в переменной a

🔧 Шаг 09:

Миссия 9

Игорь анализирует функцию с условием `if x: return 1`. Сопоставьте инструкции с их задачей.

Подсказка: Условные переходы управляют выполнением.

Ответ:

- **LOAD_FAST x** — Загружает переменную x
- **POP_JUMP_IF_FALSE** — Проверяет условие и прыгает, если ложь
- **LOAD_CONST 1** — Загружает число 1
- **RETURN_VALUE** — Возвращает значение

🔧 Шаг 10:

Миссия 10

Дима анализирует вызов функции `print(5)`. Сопоставьте инструкции и их роль.

Подсказка: Сначала подгружаем объект функции, затем аргумент, а потом вызов.

Ответ:

- **LOAD_GLOBAL print** — Загружает объект функции print
- **LOAD_CONST 5** — Загружает число 5
- **CALL_FUNCTION 1** — Вызывает функцию с 1 аргументом
- **POP_TOP** — Удаляет результат вызова

🔧 Шаг 11:

Миссия 11

Костя разбирает цикл `for i in [1,2]: pass`. Сопоставьте инструкции и их смысл.

Подсказка: Сначала получаем итератор, затем перебираем значения.

Ответ:

- **LOAD_CONST [1,2]** — Загружает список [1,2]
- **GET_ITER** — Создает итератор
- **FOR_ITER** — Проверяет наличие следующего элемента
- **STORE_FAST i** — Сохраняет элемент цикла в переменной i

Шаг 12:

Миссия 12

Лена изучает обработку исключений. Сопоставьте инструкции и их назначение.

Подсказка: Исключения поднимаются через RAISE_VARARGS.

Ответ:

- **SETUP_EXCEPT** — Начало блока try/except
- **RAISE_VARARGS** — Поднимает исключение
- **POP_EXCEPT** — Завершает обработку исключения
- **RETURN_VALUE** — Завершает выполнение функции

Шаг 13:

Миссия 13

Максим анализирует def f(): return 42 . Расположите инструкции в правильном порядке.

Подсказка: Сначала загружаем число, затем возвращаем.

Ответ:

- **LOAD_CONST 42**
- **RETURN_VALUE**

Шаг 14:

Миссия 14

Оля анализирует выражение a=7 . Расположите инструкции по порядку.

Подсказка: Загружаем число, потом сохраняем в переменной.

Ответ:

- **LOAD_CONST 7**
- **STORE_NAME a**

Шаг 15:

Миссия 15

Никита исследует функцию def mul(): return 2*5 . Расположите инструкции по порядку.

Подсказка: Сначала загружаем числа, потом умножаем.

Ответ:

- **LOAD_CONST 2**
- **LOAD_CONST 5**
- **BINARY_MULTIPLY**
- **RETURN_VALUE**

🔧 Шаг 16:

Миссия 16

Анна исследует цикл `for i in [1,2]: pass`. Расположите инструкции в правильном порядке.

Подсказка: Сначала список, потом итератор, затем цикл.

Ответ:

- `LOAD_CONST [1,2]`
- `GET_ITER`
- `FOR_ITER`
- `STORE_FAST i`
- `JUMP_ABSOLUTE`
- `LOAD_CONST None`
- `RETURN_VALUE`

📖 Урок 3 - Инспекция `CodeType` и структура кода

🔧 Шаг 01:

Миссия 1

Александр впервые исследует `CodeType`. Он написал простую функцию и вызвал `**code**`. Какой тип объекта он получил?

Подсказка: Атрибут `**code**` у функции возвращает не строку, а специальный объект Python.

Ответ:

- **`types.CodeType`**
- `str`
- `bytes`
- `function`

🔧 Шаг 02:

Миссия 2

Программистка Мария заметила, что объект `CodeType` хранит количество локальных переменных. Какой атрибут за это отвечает?

Подсказка: В документации этот атрибут называется так же, как и сокращение от `local variables`.

Ответ:

- **`co_nlocals`**
- `co_stacksize`
- `co_flags`
- `co_consts`

🔧 Шаг 03:

Миссия 3

Игорь исследует функцию с двумя аргументами. Какой атрибут `CodeType` укажет число её параметров?

Подсказка: В этом имени есть слово `arg`.

Ответ:

- **`co_argcount`**

- `co_nlocals`
- `co_kwonlyargcount`
- `co_flags`

🔮 Шаг 04:

Миссия 4

Разработчик Антон захотел узнать, где хранится байткод функции. Какой атрибут `CodeType` хранит его?

Подсказка: Этот атрибут возвращает `bytes`, а не строку.

Ответ:

- **`co_code`**
- `co_consts`
- `co_names`
- `co_freevars`

🔮 Шаг 05:

Миссия 5

Юля заметила, что `CodeType` хранит кортеж строк — имена всех использованных глобальных объектов. Как называется этот атрибут?

Подсказка: В названии есть слово `names`.

Ответ:

- **`co_names`**
- `co_varnames`
- `co_freevars`
- `co_filename`

🔮 Шаг 06:

Миссия 6

Сопоставьте атрибут `CodeType` с тем, что он хранит.

Подсказка: Вспомните: одни атрибуты содержат числа, другие — кортежи или строки.

Ответ:

- **`co_filename`** — Имя файла, откуда скомпилирован код
- **`co_varnames`** — Кортеж имён локальных переменных
- **`co_stacksize`** — Размер стека, необходимый для выполнения
- **`co_consts`** — Кортеж констант, использованных в коде

🔮 Шаг 07:

Миссия 7

Андрей хочет понять, какие аргументы принимает функция. Сопоставьте атрибуты `CodeType` с их назначением.

Подсказка: Обратите внимание на отличие `argcount` и `kwonlyargcount`.

Ответ:

- **`co_argcount`** — Количество позиционных аргументов
- **`co_kwonlyargcount`** — Количество только именованных аргументов
- **`co_posonlyargcount`** — Количество только позиционных аргументов

- **co_flags** — Флаги, описывающие особенности функции

🔮 Шаг 08:

Миссия 8

Сопоставьте атрибуты CodeType с тем, что они дают программисту.

Подсказка: Внимание: один отвечает за замыкания, другой — за генерацию байткода.

Ответ:

- **co_freevars** — Переменные из внешних областей (для замыканий)
- **co_cellvars** — Переменные, которые используются в замыканиях
- **co_code** — Сырые байты байткода
- **co_name** — Имя функции

🔮 Шаг 09:

Миссия 9

Сопоставьте атрибут CodeType с типом данных, который он обычно хранит.

Подсказка: Одни возвращают кортеж, другие число, третьи строку.

Ответ:

- **co_consts** — tuple
- **co_nlocals** — int
- **co_filename** — str
- **co_code** — bytes

🔮 Шаг 10:

Миссия 10

Команда разработчиков изучает стек. Сопоставьте атрибуты CodeType с правильными объяснениями.

Подсказка: Подумайте, какой атрибут отвечает за память стека.

Ответ:

- **co_stacksize** — Максимальная глубина стека
- **co_inotab** — Сжатая таблица номеров строк
- **co_firstlineno** — Первая строка исходного кода
- **co_flags** — Битовые флаги компилятора

🔮 Шаг 11:

Миссия 11

Расставьте шаги анализа CodeType в правильном порядке.

Подсказка: Логично сначала получить объект, потом исследовать его атрибуты, затем интерпретировать данные.

Ответ:

- Получить объект функции
- Извлечь ****code****
- Проанализировать атрибуты CodeType
- Сделать вывод о структуре кода

Шаг 12:

Миссия 12

Расставьте этапы работы с локальными переменными через `co_varnames` .

Подсказка: Надо сначала скомпилировать функцию, потом получить кортеж переменных и уже анализировать.

Ответ:

- Написать функцию
- Получить `**code**`
- Обратиться к `co_varnames`
- Определить имена локальных переменных

Шаг 13:

Миссия 13

Упорядочите шаги для анализа числа аргументов функции.

Подсказка: Вспомните, что сначала нужен объект функции, затем `**code**` , потом атрибут `co_argcount` .

Ответ:

- Определить функцию
- Получить объект `**code**`
- Обратиться к `co_argcount`
- Узнать число аргументов

Шаг 14:

Миссия 14

Расставьте действия при исследовании байткода из `co_code` .

Подсказка: Сначала получаем функцию, потом код, затем разбираем байты.

Ответ:

- Определить функцию
- Взять атрибут `**code**`
- Извлечь `co_code`
- Анализировать байты

Шаг 15:

Миссия 15

Расположите шаги изучения информации о строках исходного кода через `co_lnotab` .

Подсказка: Этот атрибут связан с отображением байткода в строки.

Ответ:

- Создать функцию
- Получить `**code**`
- Обратиться к `co_lnotab`
- Сопоставить номера строк с байткодом

Шаг 16:

Миссия 16

Расположите шаги при изучении замыканий через `co_freevars` и `co_cellvars` .

Подсказка: Сначала нужно создать вложенную функцию, потом получить её `**code**` , затем проверить нужные атрибуты.

Ответ:

- Создать функцию с вложенной
- Получить `**code**`
- Изучить `co_freevars`
- Изучить `co_cellvars`

Урок 4 - Практика дизассемблируем собственные функции

Шаг 01:

Миссия 1

Ты пишешь функцию и решаешь заглянуть в её байткод. Какой модуль Python чаще всего используют для дизассемблирования?

Подсказка: Этот модуль встроен в стандартную библиотеку и его имя похоже на термин из ассемблера.

Ответ:

- **dis**
- inspect
- opcode
- marshal

Шаг 02:

Миссия 2

Алиса запускает `dis.dis(my_func)` . Что она увидит на экране?

Подсказка: Речь не про исходный код и не про AST.

Ответ:

- **Инструкции байткода функции**
- Скомпилированный .рус файл
- Граф вызовов функций
- Список импортов модуля

Шаг 03:

Миссия 3

Боб исследует простую функцию `def f(x): return x+1` . Какой байткод появится первым в её дизассембле?

Подсказка: Подумай: чтобы выполнить `x+1` , нужно сначала загрузить аргумент.

Ответ:

- **LOAD_FAST**
- LOAD_CONST
- RETURN_VALUE
- STORE_FAST

Шаг 04:

Миссия 4

Карл заметил, что в выводе dis присутствует RETURN_VALUE . Для чего эта инструкция?

Подсказка: Без неё функция не вернёт результат.

Ответ:

- **Завершает выполнение функции и возвращает верхний элемент стека**
- Печатает результат в консоль
- Сохраняет переменную в локальном словаре
- Загружает глобальную константу

Шаг 05:

Миссия 5

Даша заметила в байткоде функцию LOAD_CONST . Что именно загружается?

Подсказка: Это может быть число, строка или даже кортеж.

Ответ:

- **Константа из таблицы констант функции**
- Значение глобальной переменной
- Импортированный модуль
- Аргумент функции

Шаг 06:

Миссия 6

Егор дизассемблирует функцию с циклом for . Какой байткод почти всегда встречается внутри?

Подсказка: Вспомни, что цикл итерируется по объекту шаг за шагом.

Ответ:

- **FOR_ITER**
- CALL_FUNCTION
- BUILD_TUPLE
- JUMP_ABSOLUTE

Шаг 07:

Миссия 7

Гриша пробует дизассемблировать лямбду lambda x: x*2 . Как она выглядит в байткоде?

Подсказка: Лямбда — это та же функция, только без имени.

Ответ:

- **Как обычная функция с инструкциями LOAD_FAST, LOAD_CONST, BINARY_MULTIPLY, RETURN_VALUE**
- Как пустой блок без инструкций
- Как специальная инструкция LAMBDA_OP
- Как строковый литерал

Шаг 08:

Миссия 8

 Сопоставьте элементы дизассемблирования с их описаниями.

Подсказка: Вспомни, что `LOAD` всегда кладёт что-то на стек, а `RETURN_VALUE` завершает выполнение.

Ответ:

- **LOAD_FAST** — Загружает локальную переменную
- **LOAD_CONST** — Загружает константу из таблицы
- **CALL_FUNCTION** — Выполняет вызов функции
- **RETURN_VALUE** — Возвращает значение из функции

🔗 Шаг 09:

Миссия 9

📖 У программистов в офисе спор: какие инструкции отвечают за работу с коллекциями? Сопоставь инструкции с их назначением.

Подсказка: `BUILD_*` создаёт объект, `UNPACK_*` разбирает его.

Ответ:

- **BUILD_LIST** — Создаёт список из N элементов
- **BUILD_TUPLE** — Создаёт кортеж из N элементов
- **BUILD_SET** — Создаёт множество
- **UNPACK_SEQUENCE** — Распаковывает элементы последовательности

🔗 Шаг 10:

Миссия 10

📖 Дизассемблер показал байткод с цикла. Сопоставь инструкции с их функциями.

Подсказка: `FOR_ITER` отвечает за шаги итератора, `JUMP_ABSOLUTE` — за возврат к началу.

Ответ:

- **FOR_ITER** — Берёт следующий элемент итератора или завершает цикл
- **JUMP_ABSOLUTE** — Прыжок на конкретный адрес байткода
- **POP_BLOCK** — Завершает блок цикла или try
- **BREAK_LOOP** — Прерывает выполнение цикла

🔗 Шаг 11:

Миссия 11

📖 Исследуй условные переходы: сопоставь инструкцию и её смысл.

Подсказка: Условные прыжки проверяют истину/ложь и прыгают вперёд.

Ответ:

- **POP_JUMP_IF_FALSE** — Прыжок, если значение на стеке ложное
- **POP_JUMP_IF_TRUE** — Прыжок, если значение на стеке истинное
- **JUMP_FORWARD** — Прыжок на фиксированное смещение вперёд
- **COMPARE_OP** — Выполняет операцию сравнения

🔗 Шаг 12:

Миссия 12

📖 Тестируем вызовы: сопоставь байткод и его назначение.

Подсказка: `MAKE_FUNCTION` создаёт функцию, `CALL_*` — вызывает её.

Ответ:

- **MAKE_FUNCTION** — Создаёт объект функции
- **CALL_FUNCTION** — Вызывает функцию с N аргументами
- **CALL_METHOD** — Вызывает метод объекта
- **LOAD_GLOBAL** — Загружает глобальный объект или функцию

 **Шаг 13:**

Миссия 13

📄 Расположи в правильном порядке байткод для функции `def f(x): return x+2` .

Подсказка: Сначала загружаем переменную, потом константу, выполняем операцию и возвращаем результат.

Ответ:

- `LOAD_FAST`
- `LOAD_CONST`
- `BINARY_ADD`
- `RETURN_VALUE`

 **Шаг 14:**

Миссия 14

📄 Расположи байткод для цикла `for i in range(3): pass` .

Подсказка: Итератор создаётся, затем `FOR_ITER` делает шаги, `JUMP_ABSOLUTE` возвращает к началу.

Ответ:

- `LOAD_GLOBAL`
- `LOAD_CONST`
- `CALL_FUNCTION`
- `GET_ITER`
- `FOR_ITER`
- `JUMP_ABSOLUTE`

 **Шаг 15:**

Миссия 15

📄 Расположи инструкции для условного блока `if x: return 1` .

Подсказка: Сначала загружаем переменную, затем проверяем условие прыжком.

Ответ:

- `LOAD_FAST`
- `POP_JUMP_IF_FALSE`
- `LOAD_CONST`
- `RETURN_VALUE`

Урок 5 - Лабиринт анализ чужого байткода

 **Шаг 01:**

Миссия 1

Ты открываешь чужой модуль и видишь вызов `dis.dis()` . Для чего он используется?

Подсказка: Этот инструмент не меняет код, он помогает его разглядеть изнутри.

Ответ:

- **Для просмотра байткода Python-функций**
- Для оптимизации скорости работы программы
- Для генерации документации автоматически
- Для запуска программы в защищённом режиме

Шаг 02:

Миссия 2

В байткоде встретился оператор `LOAD_CONST` . Что он делает?

Подсказка: Этот оператор достаёт что-то заранее известное, а не вычисляемое на лету.

Ответ:

- **Загружает константу в стек**
- Загружает переменную из локальной области
- Вызывает встроенную функцию
- Сохраняет результат вычисления в глобальные переменные

Шаг 03:

Миссия 3

Программист Саша смотрит байткод и встречает `RETURN_VALUE` . Что происходит?

Подсказка: Эта команда завершает путешествие функции.

Ответ:

- **Функция возвращает верхнее значение из стека**
- Функция вызывает другую функцию
- Функция начинает выполнение заново
- Функция очищает стек без возврата

Шаг 04:

Миссия 4

Кирилл нашёл в чужом байткоде `CALL_FUNCTION` . Для чего она используется?

Подсказка: Подумай о том, как Python вызывает функции и откуда берутся аргументы.

Ответ:

- **Вызывает функцию с аргументами из стека**
- Создаёт новую локальную переменную
- Загружает встроенную константу
- Завершает выполнение программы

Шаг 05:

Миссия 5

Байт-код показал оператор `POP_TOP` . Какое его назначение?

Подсказка: Иногда верхушка стека больше не нужна.

Ответ:

- **Удаляет верхнее значение из стека**
- Сохраняет значение в глобальной области

- Меняет порядок элементов стека
- Записывает значение в return

🔮 Шаг 06:

Миссия 6

Лена анализирует чужую функцию и видит `LOAD_FAST`. Что это значит?

Подсказка: Речь идёт про локальные переменные, а не про глобальные.

Ответ:

- **Загружает локальную переменную в стек**
- Загружает глобальную переменную в стек
- Загружает модуль из `sys.modules`
- Загружает строковую константу

🔮 Шаг 07:

Миссия 7

Сопоставь байт-кодовые инструкции с их действием.

Подсказка: Вспомни, что стек — это основа работы байткода.

Ответ:

- **LOAD_CONST** — Загружает значение константы в стек
- **STORE_FAST** — Сохраняет значение из стека в локальную переменную
- **BINARY_ADD** — Складывает два верхних значения стека
- **POP_TOP** — Удаляет верхнее значение из стека

🔮 Шаг 08:

Миссия 8

Аркадий заметил в байткоде разные инструкции. Сопоставь их с категориями.

Подсказка: Подумай: одни инструкции читают данные, другие меняют стек, а третьи управляют потоком.

Ответ:

- **LOAD_GLOBAL** — Загрузка значения
- **RETURN_VALUE** — Управление выполнением
- **BINARY_MULTIPLY** — Арифметическая операция
- **POP_JUMP_IF_FALSE** — Условный переход

🔮 Шаг 09:

Миссия 9

Сопоставь Python-код с тем, что он породит в байткоде.

Подсказка: Думай как интерпретатор: числа идут в `LOAD_CONST`, сложение — в `BINARY_ADD`.

Ответ:

- **x = 1** — `LOAD_CONST 1; STORE_FAST x`
- **y = 2** — `LOAD_CONST 2; STORE_FAST y`
- **z = x + y** — `LOAD_FAST x; LOAD_FAST y; BINARY_ADD; STORE_FAST z`
- **return z** — `LOAD_FAST z; RETURN_VALUE`

🔗 Шаг 10:

Миссия 10

Сопоставь байт-кодовые инструкции с их стек-эффектом.

Подсказка: Вспомни: одни инструкции добавляют в стек, другие убирают.

Ответ:

- **LOAD_CONST** — +1 в стек
- **POP_TOP** — -1 из стека
- **BINARY_ADD** — -1 из стека (2 аргумента → 1 результат)
- **RETURN_VALUE** — -1 из стека (возврат значения)

🔗 Шаг 11:

Миссия 11

Сопоставь инструкции условных переходов с их назначением.

Подсказка: Одни прыгают, если условие ложное, другие — если истинное.

Ответ:

- **POP_JUMP_IF_FALSE** — Прыгает, если верхнее значение стека равно False
- **POP_JUMP_IF_TRUE** — Прыгает, если верхнее значение стека равно True
- **JUMP_FORWARD** — Безусловный переход вперёд
- **JUMP_ABSOLUTE** — Прыжок к абсолютному адресу

🔗 Шаг 12:

Миссия 12

Сопоставь байт-кодовые инструкции с уровнем их абстракции.

Подсказка: В Python есть инструкции для работы с переменными, арифметикой и управлением.

Ответ:

- **STORE_GLOBAL** — Работа с переменными
- **BINARY_SUBTRACT** — Арифметическая операция
- **LOAD_METHOD** — Работа с объектами
- **RETURN_VALUE** — Управление выполнением

🔗 Шаг 13:

Миссия 13

Расставь шаги выполнения функции f(): return 2+3 в правильном порядке.

Подсказка: Сначала загружаются константы, потом складываются, и только потом возвращается результат.

Ответ:

- LOAD_CONST 2
- LOAD_CONST 3
- BINARY_ADD
- RETURN_VALUE

🔗 Шаг 14:

Миссия 14

Поставь инструкции в том порядке, как Python выполняет условие if x: .

Подсказка: Сначала идёт загрузка, потом проверка, и в конце прыжок.

Ответ:

- `LOAD_FAST x`
- `POP_JUMP_IF_FALSE`
- ... блок `if ...`
- `JUMP_FORWARD` (пропуск `else`)

 **Шаг 15:**

Миссия 15

Отсортируй шаги байткода для цикла `for i in range(3):`.

Подсказка: Генерация итератора, проверка, загрузка значения, повтор.

Ответ:

- `LOAD_GLOBAL range`
- `LOAD_CONST 3`
- `CALL_FUNCTION 1`
- `GET_ITER`
- `FOR_ITER`
- `STORE_FAST i`

 **Шаг 16:**

Миссия 16

Поставь инструкции для вызова метода `obj.method()` в правильном порядке.

Подсказка: Подумай: сначала загружается объект, потом метод, потом вызов.

Ответ:

- `LOAD_FAST obj`
- `LOAD_METHOD method`
- `CALL_METHOD 0`
- `POP_TOP`

Модуль 2 - Алхимия изменения байткода ↩

Урок 1 - Создание и модификация `CodeType`

 **Шаг 01:**

Миссия 1

Программист Лена обнаружила, что объект `CodeType` хранит множество параметров. Какой из них отвечает за количество аргументов функции?

Подсказка: Подумайте, что именно описывает количество входов у функции.

Ответ:

- `co_argcount`
- `co_consts`
- `co_stacksize`
- `co_flags`

Шаг 02:

Миссия 2

Денису нужно изменить строку документации в объекте кода. Какой атрибут `CodeType` указывает на список констант, включая строки?

Подсказка: Там же обычно лежат числа и строки, встречающиеся в функции.

Ответ:

- **`co_consts`**
- `co_varnames`
- `co_filename`
- `co_lnotab`

Шаг 03:

Миссия 3

Артём захотел поменять имя функции, не меняя её тело. Какой атрибут `CodeType` хранит её имя?

Подсказка: Имя всегда доступно при печати объекта кода.

Ответ:

- **`co_name`**
- `co_freevars`
- `co_cellvars`
- `co_stacksize`

Шаг 04:

Миссия 4

Ксюша скомпилировала новую функцию и заметила, что `CodeType` имеет атрибут `co_code`. Что именно он хранит?

Подсказка: Это не текст и не дерево, а «сырой материал» для виртуальной машины.

Ответ:

- **Байткод в виде байтовой строки**
- Исходный текст функции
- Абстрактное синтаксическое дерево
- Имена глобальных переменных

Шаг 05:

Миссия 5

Миша экспериментировал с `CodeType` и обнаружил `co_varnames`. Что в нём содержится?

Подсказка: Подумайте о локальных переменных и аргументах.

Ответ:

- **Кортеж имён локальных переменных и аргументов**
- Список строковых констант
- Набор импортов
- Байтовое представление байткода

Шаг 06:

Миссия 6

Сопоставьте атрибуты CodeType с их назначением.

Подсказка: Вспомните: аргументы, константы, стек и имя — у каждого своя роль.

Ответ:

- **co_argcount** — Количество аргументов функции
- **co_consts** — Кортеж констант, включая строки и числа
- **co_stacksize** — Максимальная глубина стека
- **co_name** — Имя функции

Шаг 07:

Миссия 7

Нужно сопоставить переменные CodeType и то, что они описывают.

Подсказка: Смотрите на ключевые слова: freevars, cellvars, filename, flags.

Ответ:

- **co_freevars** — Свободные переменные замыканий
- **co_cellvars** — Переменные, используемые в замыкании
- **co_filename** — Имя файла исходного кода
- **co_flags** — Флаги, описывающие режим работы функции

Шаг 08:

Миссия 8

Анализируем, что лежит внутри CodeType.

Подсказка: Обратите внимание на varnames, names, lnotab и qualname.

Ответ:

- **co_varnames** — Имена локальных переменных и аргументов
- **co_names** — Ссылки на глобальные и атрибуты
- **co_lnotab** — Таблица соответствия строк исходного кода и байткода
- **co_qualname** — Полное имя функции

Шаг 09:

Миссия 9

Саша пытается разобраться, что означают разные числа в CodeType.

Подсказка: Обратите внимание на nlocals, kwonlyargcount, posonlyargcount, stacksize.

Ответ:

- **co_nlocals** — Количество локальных переменных
- **co_kwonlyargcount** — Количество аргументов только по ключу
- **co_posonlyargcount** — Количество аргументов только по позиции
- **co_stacksize** — Максимальная глубина стека

Шаг 10:

Миссия 10

Определите, какой атрибут отвечает за что.

Подсказка: globals, firstlineno, code, flags — звучит как разные категории.

Ответ:

- **co_names** — Список имён глобальных переменных

- **co_firstlineno** — Номер первой строки функции в исходнике
- **co_code** — Байтовое представление байткода
- **co_flags** — Битовые флаги особенностей функции

🔗 Шаг 11:

Миссия 11

Дима хочет понять, какие атрибуты нужны для создания CodeType вручную.

Подсказка: Посмотрите на argcount, posonly, kwonly и nlocals.

Ответ:

- **co_argcount** — Количество позиционных аргументов
- **co_posonlyargcount** — Количество позиционных только аргументов
- **co_kwonlyargcount** — Количество ключевых только аргументов
- **co_nlocals** — Число локальных переменных

🔗 Шаг 12:

Миссия 12

Расположите шаги при модификации CodeType в правильном порядке.

Подсказка: Сначала берём старый объект, потом копируем, вносим изменения и создаём новый.

Ответ:

- Получить исходный объект CodeType
- Скопировать его параметры
- Изменить нужный параметр
- Создать новый объект CodeType с изменёнными данными

🔗 Шаг 13:

Миссия 13

Упорядочьте шаги анализа байткода перед модификацией.

Подсказка: Сначала дизассемблируем, потом ищем место, затем планируем изменения, а после тестируем.

Ответ:

- Дизассемблировать функцию
- Найти интересующую инструкцию
- Спланировать изменения байткода
- Проверить работу после модификации

🔗 Шаг 14:

Миссия 14

Расставьте этапы работы программиста с CodeType при замене имени функции.

Подсказка: Сначала берём исходный код, потом меняем co_name, создаём новый объект и проверяем результат.

Ответ:

- Извлечь объект CodeType функции
- Изменить значение co_name
- Создать новый CodeType с этим значением
- Выполнить функцию и убедиться, что имя изменилось

Шаг 15:

Миссия 15

Упорядочьте шаги изменения констант в CodeType.

Подсказка: Сначала скопируйте константы, потом измените список, и только потом создайте новый объект.

Ответ:

- Получить кортеж `co_consts`
- Скопировать его в список
- Внести изменения
- Создать новый CodeType с обновлёнными константами

Шаг 16:

Миссия 16

Расположите действия программиста, если он хочет изменить стековый размер в CodeType.

Подсказка: Анализируем, потом меняем, создаём новый и проверяем.

Ответ:

- Изучить исходное значение `co_stacksize`
- Определить новое значение
- Создать новый CodeType с этим значением
- Протестировать функцию

Шаг 17:

Миссия 17

Расположите шаги при изменении таблицы соответствия строк (`co_inotab`).

Подсказка: Изменение `Inotab` требует осторожности: сначала извлекаем, потом меняем, создаём новый и проверяем дебаггером.

Ответ:

- Получить `co_inotab`
- Изменить байтовую последовательность
- Создать новый CodeType
- Проверить работу отладчика или трассировки

Урок 2 - Динамическая генерация функций

Шаг 01:

Миссия 1

Программист Саша решил сгенерировать функцию прямо во время работы программы. Что является ключевым объектом Python для хранения исполняемого байткода функции?

Подсказка: Этот объект содержит все инструкции, константы и локальные переменные функции.

Ответ:

- **CodeType**
- FrameType
- FunctionType
- ModuleType

Шаг 02:

Миссия 2

Ира решила создать функцию на лету при помощи конструктора `types.FunctionType`. Какой аргумент обязательно передать при создании?

Подсказка: Без него функция не знает, что выполнять.

Ответ:

- **Объект `CodeType`**
- Список локальных переменных
- Аргументы по умолчанию
- Имя модуля

Шаг 03:

Миссия 3

Миша экспериментирует с генерацией функций. Он передал в `types.FunctionType` объект `CodeType` и словарь окружения. Что произойдет, если окружение пустое?

Подсказка: Подумайте, где функция ищет глобальные имена.

Ответ:

- **Функция не сможет находить глобальные имена**
- Будет использовать встроенные функции автоматически
- Вернет `None` при вызове
- Произойдет синтаксическая ошибка

Шаг 04:

Миссия 4

Анна сгенерировала функцию из байткода, но забыла передать имя функции в `FunctionType`. Что произойдет?

Подсказка: У функции всегда должно быть имя, даже если оно временное.

Ответ:

- Функция не создастся
- **Будет присвоено имя по умолчанию**
- Произойдет `SyntaxError`
- Функция станет встроенной

Шаг 05:

Миссия 5

Сергей захотел динамически сгенерировать функцию, которая возвращает сумму двух чисел. Какой шаг необязателен в процессе?

Подсказка: Некоторые параметры можно оставить пустыми, и Python справится сам.

Ответ:

- **Передача аргументов по умолчанию**
- Создание объекта `CodeType`
- Создание объекта `FunctionType`
- Передача словаря окружения

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте понятия с их ролями при динамическом создании функций.

Подсказка: Вспомните: один объект хранит байткод, другой связывает его с окружением.

Ответ:

- **CodeType** — Хранит байткод и структуру функции
- **FunctionType** — Создает исполняемую функцию из CodeType
- **Globals** — Определяет доступные глобальные имена
- **Defaults** — Содержит значения аргументов по умолчанию

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте способы динамического создания функций с их описанием.

Подсказка: Есть быстрый путь через `exec`, и есть прямой путь через байткод.

Ответ:

- **exec** — Выполняет строку кода и создает функцию обычным способом
- **lambda** — Создает функцию из выражения, но не через байткод напрямую
- **types.FunctionType** — Создает функцию из CodeType
- **compile** — Преобразует строку в объект CodeType

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте параметры FunctionType с их назначением.

Подсказка: Одни управляют окружением, другие — аргументами.

Ответ:

- **code** — Объект CodeType, содержащий инструкции
- **globals** — Словарь глобальных переменных
- **name** — Имя создаваемой функции
- **defaults** — Кортеж значений по умолчанию

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте инструменты с их результатом.

Подсказка: Один возвращает байткод, другой — исполняемую функцию.

Ответ:

- **compile** — Создает CodeType из текста
- **dis** — Показывает инструкции байткода
- **types.FunctionType** — Создает исполняемую функцию
- **marshal** — Сохраняет и восстанавливает объекты CodeType

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте объекты с их аналогиями.

Подсказка: Представьте: один — чертёж, другой — дом, третий — набор инструментов.

Ответ:

- **CodeType** — Чертёж функции
- **FunctionType** — Готовый дом, построенный по чертежу
- **Globals** — Материалы и инструменты для постройки
- **Defaults** — Стандартные элементы, добавленные по умолчанию

Шаг 11:

Миссия 11

 Сопоставьте сценарии использования динамических функций с их назначением.

Подсказка: Вспомните: иногда нужно ускорить код, а иногда — сгенерировать шаблон.

Ответ:

- **Генерация кода из шаблонов** — Создание функций под конкретные SQL-запросы
- **JIT-компиляция** — Ускорение работы циклов и численных операций
- **Метапрограммирование** — Автоматическая сборка функций из компонентов
- **DSL** — Создание мини-языков внутри Python

Шаг 12:

Миссия 12

Расположите шаги динамического создания функции с помощью `compile` и `FunctionType` в правильном порядке.

Подсказка: Сначала нужен текст, потом байткод, затем — функция.

Ответ:

- Написать строку с кодом функции
- Скомпилировать строку в объект `CodeType`
- Создать `FunctionType` на основе `CodeType`
- Вызвать функцию и проверить результат

Шаг 13:

Миссия 13

Расположите шаги модификации сгенерированной функции.

Подсказка: Изменения делают до вызова.

Ответ:

- Создать функцию через `FunctionType`
- Изменить `defaults` или `globals`
- Сохранить новую функцию в переменную
- Вызвать с аргументами

Шаг 14:

Миссия 14

Расположите шаги работы с `marshal` для сохранения и загрузки `CodeType`.

Подсказка: Сначала сохранить, потом восстановить.

Ответ:

- Создать объект `CodeType`

- Сохранить CodeType с помощью marshal.dump
- Загрузить CodeType обратно с помощью marshal.load
- Создать FunctionType из загруженного CodeType

🔧 Шаг 15:

Миссия 15

Расположите шаги анализа сгенерированной функции при помощи dis .

Подсказка: Сначала функция, потом байткод.

Ответ:

- Создать динамическую функцию
- Передать её в dis.dis
- Получить список инструкций байткода
- Сделать выводы об оптимизации

🔧 Шаг 16:

Миссия 16

Расположите шаги применения динамически созданной функции в мини-DSL.

Подсказка: DSL сначала описывает правила, потом их превращают в код.

Ответ:

- Написать правила DSL
- Сгенерировать код Python из правил
- Скомпилировать код в CodeType
- Создать FunctionType и выполнить

🔧 Шаг 17:

Миссия 17

Расположите шаги создания функции JIT-оптимизации.

Подсказка: JIT сначала анализирует, потом компилирует.

Ответ:

- Проанализировать часто выполняемый код
- Сгенерировать оптимизированный байткод
- Создать CodeType с оптимизациями
- Выполнить через FunctionType

📖 Урок 3 - Мини-JIT на Python концепция и практика

🔧 Шаг 01:

Миссия 1

Разработчик Тимур услышал про JIT-компиляцию и хочет понять её суть. Что является основной идеей JIT в Python?

Подсказка: Вспомните: JIT ускоряет выполнение, но не переписывает программу заново.

Ответ:

- **Компиляция байткода в машинный код во время выполнения**
- Только статическая оптимизация перед запуском
- Хранение всего кода в базе данных

- Переписывание кода пользователем вручную

Шаг 02:

Миссия 2

Анна экспериментирует с ускорением циклов. Как JIT обычно работает с горячими участками кода?

Подсказка: Часто исполняемый код обрабатывается по-особенному.

Ответ:

- **Оптимизирует и компилирует их в более быстрый машинный код**
- Удаляет их полностью
- Всегда интерпретирует их без изменений
- Заменяет их случайными инструкциями

Шаг 03:

Миссия 3

Петя заметил, что JIT-компиляция не всегда даёт прирост. В какой ситуации JIT может быть менее эффективен?

Подсказка: Подумайте про маленькие или редко используемые функции.

Ответ:

- **Когда функция вызывается очень редко**
- Когда функция используется в большом цикле
- Когда код обрабатывает большие данные
- Когда используется простая арифметика

Шаг 04:

Миссия 4

Юля экспериментирует с PyPy. Чем PyPy отличается от CPython?

Подсказка: Вспомните: речь идёт не только о стандартном интерпретаторе.

Ответ:

- **PyPy использует JIT-компилятор для ускорения выполнения**
- PyPy работает только с C++
- PyPy не поддерживает функции
- PyPy всегда медленнее CPython

Шаг 05:

Миссия 5

Стас изучает внутренности мини-JIT. Что первым делом делает JIT перед генерацией машинного кода?

Подсказка: Машинный код не появляется из ниоткуда.

Ответ:

- **Анализирует байткод Python**
- Запускает графический интерфейс
- Сохраняет код на диск
- Случайно изменяет байты памяти

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте элементы JIT-компиляции с их назначением.

Подсказка: Вспомните, какие шаги нужны для превращения байткода в быстрый машинный код.

Ответ:

- **HotSpot** — Выявление часто выполняемых участков кода
- **Интерпретатор** — Выполняет байткод напрямую
- **Компилятор JIT** — Преобразует байткод в машинные инструкции
- **Оптимизатор** — Убирает лишние инструкции и ускоряет работу

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте понятие с правильным описанием.

Подсказка: Подумайте о слоях: от исходного кода до процессора.

Ответ:

- **Исходный код** — Текст программы на Python
- **Байткод** — Инструкции для виртуальной машины Python
- **Машинный код** — Инструкции, понятные процессору
- **JIT** — Связующее звено между байткодом и машинным кодом

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте техники оптимизации JIT с их результатом.

Подсказка: Каждая техника делает выполнение короче или быстрее.

Ответ:

- **Inlining** — Замена вызова функции её телом
- **Loop unrolling** — Повторение тела цикла для ускорения
- **Constant folding** — Вычисление выражений во время компиляции
- **Dead code elimination** — Удаление кода, который никогда не выполняется

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте библиотеку/инструмент с его особенностью.

Подсказка: Эти инструменты часто используют JIT в Python-проектах.

Ответ:

- **Numba** — Компиляция Python-функций в машинный код с помощью LLVM
- **PyPy** — Альтернативный интерпретатор с JIT
- **LLVM** — Инфраструктура для создания компиляторов
- **Cython** — Ускорение Python кода через статическую компиляцию

🔗 Шаг 10:

Миссия 10

📖 Сопоставьте термин JIT с примером его применения.

Подсказка: Представьте, что вы объясняете другу, где применяется каждая идея.

Ответ:

- **Traces** — Запоминание цепочки часто выполняемых инструкций
- **Profiling** — Сбор статистики о работе программы
- **Fallback** — Возврат к интерпретатору при ошибке компиляции
- **Code cache** — Хранение уже скомпилированных фрагментов

Шаг 11:

Миссия 11

Расположите шаги работы мини-JIT в правильном порядке.

Подсказка: Сначала код анализируется, затем компилируется, потом кешируется и выполняется.

Ответ:

- Анализ горячего кода
- Компиляция в машинный код
- Сохранение результата в кэш
- Выполнение машинного кода

Шаг 12:

Миссия 12

Расположите действия программиста при отладке JIT.

Подсказка: Сначала собираем информацию, потом находим узкие места, а затем тестируем изменения.

Ответ:

- Сбор профайлинга
- Поиск горячих функций
- Изменение стратегии компиляции
- Перезапуск программы и проверка

Шаг 13:

Миссия 13

Расположите этапы превращения Python-кода в JIT-оптимизированный код.

Подсказка: Подумайте: от текста до ускоренного исполнения.

Ответ:

- Написание исходного кода
- Преобразование в байткод
- Выявление горячих участков
- JIT-компиляция и оптимизация

Шаг 14:

Миссия 14

Расположите шаги работы оптимизатора в JIT.

Подсказка: Сначала ищутся лишние операции, затем они убираются, и в конце проверяется корректность.

Ответ:

- Поиск неиспользуемого кода
- Удаление неиспользуемого кода
- Сокращение выражений
- Проверка эквивалентности результата

 Шаг 15:**Миссия 15**

Расположите шаги тестирования мини-JIT.

Подсказка: Сначала проект создаётся, потом проверяются функции, далее сравниваются результаты и фиксируются баги.

Ответ:

- Создание минимального прототипа
- Запуск функций под JIT
- Сравнение результатов с интерпретатором
- Фиксация багов и оптимизация

 Шаг 16:**Миссия 16**

Расположите шаги применения JIT к математической библиотеке.

Подсказка: Оптимизация обычно начинается с профайлинга, а заканчивается ускорением.

Ответ:

- Профилирование вычислений
- Выделение горячих функций
- JIT-компиляция этих функций
- Получение ускоренного результата

 Шаг 17:**Миссия 17**

Расположите шаги интеграции мини-JIT в проект.

Подсказка: От идеи до внедрения — шаги логичны и последовательны.

Ответ:

- Определение целей ускорения
- Разработка прототипа JIT
- Интеграция в существующий код
- Тестирование и отладка

 Урок 4 - Практика ускоряем алгоритмы через байткод **Шаг 01:****Миссия 1**

Андрей написал функцию сортировки на Python, но она работает медленно. Что первым делом стоит сделать, чтобы понять, где теряется время?

Подсказка: Иногда достаточно замерить производительность, чтобы увидеть слабое место.

Ответ:

- **Использовать модуль `timeit` для замеров**
- Сразу переписать код на C++
- Добавить больше `print` для отладки
- Удалить все проверки в коде

 **Шаг 02:**

Миссия 2

Программист Маша оптимизирует функцию поиска максимума. Она заметила, что Python часто вызывает одну и ту же встроенную функцию. Как можно ускорить работу?

Подсказка: Подумайте, что быстрее: поиск имени в глобальной области или доступ к локальной переменной?

Ответ:

- **Сохранить ссылку на функцию в локальной переменной**
- Вызвать функцию через `eval`
- Использовать глобальные переменные всегда
- Писать имена функций заглавными буквами

 **Шаг 03:**

Миссия 3

Иван анализирует байткод функции. Он видит частые обращения `LOAD_GLOBAL`. Что это означает?

Подсказка: Эти инструкции связаны с областью видимости переменных.

Ответ:

- **Функция часто обращается к глобальным именам**
- Функция использует слишком много `return`
- Код выполняется в С-интерпретаторе
- Происходит обращение к байтам памяти напрямую

 **Шаг 04:**

Миссия 4

Светлана заметила, что её цикл `for` в Python стал узким местом программы. Что стоит проверить первым делом?

Подсказка: Иногда причина медленности не в самом цикле, а в том, что внутри него.

Ответ:

- **Что именно вызывается внутри цикла**
- Количество пробелов перед циклом
- Сложность алгебры за пределами цикла
- Цвет подсветки синтаксиса в IDE

 **Шаг 05:**

Миссия 5

Егор хочет ускорить выполнение кода с помощью байткода. Какой первый шаг в создании мини-оптимизации?

Подсказка: Сначала нужно увидеть, какие инструкции реально выполняются.

Ответ:

- **Использовать модуль `dis` для анализа байткода**
- Писать весь код в одну строку
- Переписать всё на ассемблер
- Добавить больше комментариев

🔗 Шаг 06:

Миссия 6

Сопоставьте байткод-инструкцию и её назначение.

Подсказка: Подумайте о том, какие операции происходят при выполнении обычных арифметических действий.

Ответ:

- **LOAD_FAST** — Загрузка локальной переменной
- **STORE_FAST** — Сохранение значения в локальную переменную
- **BINARY_ADD** — Сложение двух верхних элементов стека
- **RETURN_VALUE** — Возврат результата функции

🔗 Шаг 07:

Миссия 7

Сопоставьте оптимизационный приём и его эффект.

Подсказка: Вспомните, что можно сделать быстрее: доступ к локалам или глобалам? Вычислять выражение один раз или много?

Ответ:

- **Кэширование результата** — Избегает повторных вычислений
- **Использование локальных переменных** — Ускоряет доступ к значениям
- **Удаление лишних проверок** — Сокращает количество инструкций
- **Замена цикла на генератор** — Уменьшает накладные расходы

🔗 Шаг 08:

Миссия 8

Аня разбирает байткод функции с условием. Сопоставьте инструкцию и её роль.

Подсказка: Условные переходы — это сердце ветвления.

Ответ:

- **COMPARE_OP** — Сравнение двух значений
- **POP_JUMP_IF_FALSE** — Переход при ложном условии
- **POP_JUMP_IF_TRUE** — Переход при истинном условии
- **JUMP_FORWARD** — Безусловный переход вперёд

🔗 Шаг 09:

Миссия 9

Свяжите инструмент и его предназначение при работе с оптимизацией Python.

Подсказка: Вспомните инструменты профилирования и анализа.

Ответ:

- **timeit** — Замер времени выполнения кода

- **cProfile** — Анализ узких мест программы
- **dis** — Декомпиляция байткода
- **line_profiler** — Анализ времени по строкам

🔗 Шаг 10:

Миссия 10

Сопоставьте оптимизацию и пример из практики.

Подсказка: Сфокусируйтесь на реальных случаях из жизни программистов.

Ответ:

- **Избегание глобальных** — Вынес встроенной функции в локальную переменную
- **Кэширование выражений** — Сохранение len списка в переменную
- **Развёртывание цикла** — Ручное повторение операций вместо цикла
- **Использование встроенных функций** — Замена собственного max на встроенный

🔗 Шаг 11:

Миссия 11

Сопоставьте байткод и Python-конструкцию.

Подсказка: Подумайте, как выглядят простые выражения в низкоуровневом виде.

Ответ:

- **LOAD_CONST** — Загрузка числа 5
- **LOAD_NAME** — Обращение к переменной x
- **CALL_FUNCTION** — Вызов функции
- **BUILD_LIST** — Создание списка

🔗 Шаг 12:

Миссия 12

Упорядочите шаги оптимизации кода через байткод.

Подсказка: Сначала ищем проблему, потом анализируем, затем вносим изменения и тестируем.

Ответ:

- Профилирование кода
- Анализ байткода
- Внесение оптимизаций
- Повторный запуск тестов

🔗 Шаг 13:

Миссия 13

Валентин пишет JIT-миникомпилятор. Расставьте шаги в правильном порядке.

Подсказка: JIT всегда начинается с анализа кода, а заканчивается запуском сгенерированного фрагмента.

Ответ:

- Извлечение байткода
- Генерация оптимизированных инструкций
- Сборка нового кода
- Запуск и тестирование

Шаг 14:

Миссия 14

Расположите шаги работы с профилировщиком cProfile.

Подсказка: Не забывайте, что сначала код запускается под профилировщиком, потом идёт анализ.

Ответ:

- Запуск кода с профилировщиком
- Сбор статистики
- Интерпретация отчёта
- Выбор узкого места для оптимизации

Шаг 15:

Миссия 15

Расположите шаги оптимизации цикла for.

Подсказка: Помните, что важно сначала выявить повторные вычисления.

Ответ:

- Выявление повторных вызовов внутри цикла
- Сохранение результата в переменную
- Замена обращений на локальные переменные
- Повторный запуск и проверка скорости

Шаг 16:

Миссия 16

Саша решает оптимизировать вызовы функций. Расставьте шаги.

Подсказка: Оптимизация функции часто связана с заменой глобального вызова на локальный.

Ответ:

- Выявление частых вызовов функции
- Сохранение функции в локальной переменной
- Замена вызовов на локальные
- Запуск и проверка скорости

Шаг 17:

Миссия 17

Финальный квест: расположите шаги оптимизации через анализ байткода.

Подсказка: Представьте, что вы алхимик, превращающий тяжёлый код в лёгкий!

Ответ:

- Сбор исходного байткода
- Выявление неэффективных инструкций
- Замена их на оптимизированные
- Сборка и тестирование

Урок 5 - Лабиринт оптимизация циклов и условий

Шаг 01:

Миссия 1

Программист Саша заметил, что цикл `for` в его проекте работает слишком медленно. Какое первое действие стоит предпринять, чтобы оптимизировать цикл?

Подсказка: Всегда начинай с анализа — не оптимизируй вслепую.

Ответ:

- **Провести замеры времени выполнения цикла**
- Заменить все циклы на рекурсию
- Сразу использовать сторонние библиотеки
- Удалить цикл полностью

Шаг 02:

Миссия 2

Марина тестирует условие `if x in list`, но оно работает медленно при больших списках. Как лучше переписать проверку?

Подсказка: Вспомни разницу в скорости поиска между списками и множествами.

Ответ:

- **Использовать множество: `if x in set`**
- Перебирать вручную элементы списка
- Сортировать список перед проверкой
- Добавить вложенный цикл

Шаг 03:

Миссия 3

Андрей заметил, что в цикле он каждый раз пересчитывает длину списка через `len(data)`. Что лучше сделать?

Подсказка: Иногда выгодно вынести повторяющиеся операции за пределы цикла.

Ответ:

- **Вынести вызов `len(data)` за пределы цикла**
- Заменить цикл на рекурсию
- **Игнорировать проблему** — это не влияет
- Вставить вызов `len(data)` дважды для надёжности

Шаг 04:

Миссия 4

Дима пишет фильтрацию данных в цикле, но его коллега предложил использовать генераторы списков. Почему это может быть быстрее?

Подсказка: Генераторы работают на уровне C и выполняют меньше байткода.

Ответ:

- **Они выполняются быстрее, так как оптимизированы внутри интерпретатора**
- Они всегда используют меньше памяти
- Они автоматически сортируют список
- Они заменяют `if` на более быстрый оператор

🔗 Шаг 05:

Миссия 5

Катя оптимизирует цикл и замечает, что использование `enumerate()` вместо `range(len(...))` работает быстрее. Почему?

Подсказка: Подумай, где создаётся меньше промежуточных объектов.

Ответ:

- **`enumerate()` сразу даёт индекс и значение без лишних обращений**
- `range(len(...))` автоматически кешируется
- `enumerate()` работает только для чисел
- `range(len(...))` меняет тип данных

🔗 Шаг 06:

Миссия 6

📄 Сопоставьте приём оптимизации с его объяснением:

Подсказка: Внимательно подумай, какие действия уменьшают количество инструкций байткода.

Ответ:

- **Замена списка на `set` для поиска** — Позволяет ускорить проверку принадлежности элементов
- **Использование `enumerate` вместо `range(len(...))`** — Снижает количество обращений к списку
- **Вынесение `len()` за пределы цикла** — Избавляет от повторных вызовов функции
- **Генераторы списков** — Оптимизированы на уровне интерпретатора

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте цикловую конструкцию и её оптимизированный вариант:

Подсказка: Иногда простая замена конструкции делает байткод короче.

Ответ:

- **`for i in range(len(data)): item = data[i]`** — `for i, item in enumerate(data)`
- **`for i in range(len(text)): ch = text[i]`** — `for ch in text`
- **`if x in my_list`** — `if x in my_set`
- **`result = [] for x in items: result.append(x*2)`** — `result = [x*2 for x in items]`

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте оптимизацию с причиной её ускорения:

Подсказка: Подумай, что происходит на уровне байткода.

Ответ:

- **Использование локальных переменных** — Доступ быстрее, чем к глобальным
- **Сравнение через `if-elif` вместо множества `if`** — Уменьшает количество проверок
- **Предварительное вычисление выражений** — Избавляет от повторных операций
- **Использование `set` для уникальности** — Убирает необходимость дополнительного цикла

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте инструмент с его назначением в оптимизации:

Подсказка: Некоторые инструменты помогают измерить, а не ускорить напрямую.

Ответ:

- **timeit** — Замер времени выполнения кода
- **dis** — Показ байткода Python-функций
- **cProfile** — Анализ узких мест программы
- **functools.lru_cache** — Сокращение числа повторных вычислений

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте ошибку неоптимального кода с её последствием:

Подсказка: Вспомни примеры, где лишние циклы замедляют выполнение.

Ответ:

- **Вызов len() внутри цикла** — Лишние вычисления
- **Поиск через список вместо set** — Медленный поиск $O(n)$
- **Использование глобальных переменных в цикле** — Замедленный доступ
- **Вложенные циклы без оптимизации** — Квадратичная сложность

🔧 Шаг 11:

Миссия 11

📖 Сопоставьте тип оптимизации с примером её применения:

Подсказка: Каждый приём можно увидеть в конкретном куске кода.

Ответ:

- **Кеширование значений** — Использование `functools.lru_cache`
- **Замена цикла на встроенные функции** — `sum([x for x in data])` → `sum(data)`
- **Использование генераторов** — `(x*x for x in range(1000))`
- **Удаление лишних условий** — `if flag: do_something()`

🔧 Шаг 12:

Миссия 12

📖 Сопоставьте стиль написания цикла с его описанием:

Подсказка: Код может быть одинаково работающим, но по-разному оптимизированным.

Ответ:

- **for x in items: result.append(x*2)** — Медленнее, создаёт лишние обращения к `append`
- **result = [x*2 for x in items]** — Быстрее, использует встроенные механизмы
- **for i, v in enumerate(items)** — Даёт и индекс, и значение
- **for i in range(len(items))** — Устаревший стиль, больше операций

🔧 Шаг 13:

Миссия 13

Расположите шаги оптимизации кода в правильном порядке:

Подсказка: Сначала измерь, затем оптимизируй, а потом проверяй результат.

Ответ:

- Замерить скорость работы
- Внести изменения в код
- Сравнить байткод до и после
- Проверить, что программа работает корректно

Шаг 14:

Миссия 14

Расположите циклы от самого медленного к самому быстрому:

Подсказка: Учти, что встроенные генераторы почти всегда быстрее.

Ответ:

- `for i in range(len(lst)): x = lst[i]`
- `for i, x in enumerate(lst)`
- `for x in lst`
- `[x for x in lst]`

Шаг 15:

Миссия 15

Расставьте инструменты в порядке применения при оптимизации:

Подсказка: Сначала ищем узкие места, потом разбираем байткод, затем оптимизируем.

Ответ:

- `cProfile`
- `dis`
- `timeit`
- Рефакторинг кода

Шаг 16:

Миссия 16

Расположите действия по оптимизации условий в правильной последовательности:

Подсказка: Сначала нужно убрать лишнее, потом упростить, затем измерить.

Ответ:

- Удалить дублирующиеся проверки
- Заменить вложенные `if` на `elif`
- Замерить производительность
- Убедиться в корректности программы

Модуль 3 - Reflection & Introspection

Урок 1 - Инструменты `inspect` и `runtime`-анализ

Шаг 01:

Миссия 1

Программист Лена хочет узнать, какие аргументы принимает её функция в runtime. Какой инструмент Python лучше использовать?

Подсказка: Смотри на модуль, который позволяет инспектировать функции и объекты.

Ответ:

- **inspect.signature**
- dir
- type
- globals

Шаг 02:

Миссия 2

Пётр хочет вывести все атрибуты объекта и методы его класса. Какой метод Python позволяет это сделать?

Подсказка: Подумай о стандартной функции, которая показывает содержимое объекта.

Ответ:

- **dir(obj)**
- help(obj)
- type(obj)
- id(obj)

Шаг 03:

Миссия 3

Маша хочет узнать исходный код функции, чтобы понять, что она делает. Какой инструмент ей поможет?

Подсказка: Есть функция, которая возвращает текст кода объекта.

Ответ:

- **inspect.getsource**
- vars
- repr
- callable

Шаг 04:

Миссия 4

Игорь хочет проверить, является ли объект функцией или методом. Какая функция для этого используется?

Подсказка: Посмотри в модуле inspect функции для проверки типа объектов.

Ответ:

- **inspect.isfunction**
- type(obj) == 'method'
- callable(obj)
- dir(obj)

Шаг 05:

Миссия 5

Оля хочет узнать, какой модуль содержит текущий стек вызовов. Какой инструмент использовать?

Подсказка: Есть стандартный модуль для анализа выполнения в runtime.

Ответ:

- **inspect.stack**
- `traceback.format_stack`
- `sys.modules`
- `dir`

Шаг 06:

Миссия 6

 Сопоставьте инструменты runtime-анализа с их назначением:

Подсказка: Каждый инструмент Python имеет своё специфическое применение.

Ответ:

- **inspect.signature** — Получение информации о параметрах функции
- **inspect.getsource** — Возвращает исходный код объекта
- **inspect.isclass** — Проверка, является ли объект классом
- **inspect.stack** — Возвращает текущий стек вызовов

Шаг 07:

Миссия 7

 Сопоставьте функции inspect с правильными утверждениями:

Подсказка: Подумайте, что каждая функция возвращает.

Ответ:

- **inspect.isfunction** — Возвращает True, если объект является функцией
- **inspect.ismethod** — Возвращает True, если объект является методом
- **inspect.getmodule** — Возвращает модуль, в котором определён объект
- **inspect.getfile** — Возвращает путь к файлу, где определён объект

Шаг 08:

Миссия 8

 Сопоставьте объекты с функциями для их анализа:

Подсказка: Один объект может требовать разный подход для инспекции.

Ответ:

- **Класс** — `inspect.isclass`
- **Функция** — `inspect.isfunction`
- **Метод класса** — `inspect.ismethod`
- **Модуль** — `inspect.getmodule`

Шаг 09:

Миссия 9

 Сопоставьте возможности inspect с примером использования:

Подсказка: Подумайте, какая функция поможет посмотреть код, а какая — параметры.

Ответ:

- **inspect.getsource** — Посмотреть исходный код функции или класса
- **inspect.signature** — Узнать список параметров функции
- **inspect.stack** — Анализ текущего стека вызовов

- **inspect.isbuiltin** — Проверка, является ли объект встроенной функцией

Шаг 10:

Миссия 10

 Сопоставьте тип анализа с инструментом inspect:

Подсказка: Разные аспекты runtime требуют разных инструментов.

Ответ:

- **Параметры функции** — inspect.signature
- **Определение класса** — inspect.isclass
- **Исходный код функции** — inspect.getsource
- **Текущий стек вызовов** — inspect.stack

Шаг 11:

Миссия 11

 Сопоставьте инструмент с его назначением для runtime-анализа:

Подсказка: Подумай о том, как разные функции помогают изучить поведение программы.

Ответ:

- **inspect.isgeneratorfunction** — Проверка, является ли функция генератором
- **inspect.iscoroutinefunction** — Проверка, является ли функция coroutine
- **inspect.getmembers** — Получение всех членов объекта
- **inspect.getmodule** — Определение модуля, в котором находится объект

Шаг 12:

Миссия 12

 Сопоставьте объект с его подходящей функцией inspect:

Подсказка: Один объект может иметь несколько аспектов для анализа.

Ответ:

- **Генератор** — inspect.isgeneratorfunction
- **Coroutine** — inspect.iscoroutinefunction
- **Функция** — inspect.isfunction
- **Метод** — inspect.ismethod

Шаг 13:

Миссия 13

Расположите действия для анализа аргументов функции в правильном порядке:

1. Получить объект функции
2. Импортировать модуль inspect
3. Вызвать inspect.signature()
4. Извлечь параметры и их значения по умолчанию
5. Проверить аннотации типов
6. Использовать полученные данные для динамического вызова функции

 **Подсказка:**

Сначала получаем функцию, потом её сигнатуру и параметры, а в конце анализируем аннотации и используем их.

Ответ:

- Получить объект функции
- Импортировать модуль `inspect`
- Вызвать `inspect.signature()`
- Извлечь параметры и их значения по умолчанию
- Проверить аннотации типов
- Использовать полученные данные для динамического вызова функции

 Шаг 14:

Миссия 14

 Сопоставьте тип объекта с правильной функцией `inspect`

Формулировка: Выберите правильный инструмент для определения типа объекта.

Понятия:

Ответ:

- Генератор
- `Coroutine`
- Функция
- Метод класса

Определения:

1. `inspect.isgeneratorfunction`
2. `inspect.iscoroutinefunction`
3. `inspect.isfunction`
4. `inspect.ismethod`

 Подсказка:

Обратите внимание на то, что генератор возвращает итератор, корутина используется с `await`, метод привязан к объекту.

- **Генератор** — `inspect.isgeneratorfunction`
- **Coroutine** — `inspect.iscoroutinefunction`
- **Функция** — `inspect.isfunction`
- **Метод класса** — `inspect.ismethod`

 Шаг 15:

Миссия 15

Расположите шаги для анализа исходного кода функции:

1. Импортировать модуль `inspect`
2. Выбрать функцию для анализа
3. Вызвать `inspect.getsource()`
4. Вывести исходный код на экран
5. Проанализировать строки и найти паттерны

 Подсказка:

Сначала получаем функцию, затем используем `inspect.getsource()`, а потом разбираем код по строкам.

Ответ:

- Импортировать модуль `inspect`
- Выбрать функцию для анализа
- Вызвать `inspect.getsource()`
- Вывести исходный код на экран
- Проанализировать строки и найти паттерны

🔗 Шаг 16:

Миссия 16

📄 Сопоставьте элементы runtime-анализа с их назначением

Понятия:

Ответ:

- `docstring`
- `locals()`
- `globals()`
- `annotations`

Определения:

1. Содержит документацию функции
2. Показывает локальные переменные функции
3. Показывает глобальные переменные модуля
4. Содержит аннотации типов параметров и возвращаемого значения

🔗 Подсказка:

`docstring` документирует функцию, `locals()` — локальные переменные, `globals()` — глобальные, `annotations` — типы параметров.

- **`docstring`** — Содержит документацию функции
- **`locals()`** — Показывает локальные переменные функции
- **`globals()`** — Показывает глобальные переменные модуля
- **`annotations`** — Содержит аннотации типов параметров и возвращаемого значения

🔗 Шаг 17:

Миссия 17

Расположите шаги runtime-анализа функции в правильном порядке:

1. Определить объект для анализа
2. Проверить тип объекта (функция, класс, метод)
3. Получить сигнатуру функции
4. Посмотреть исходный код
5. Извлечь комментарии и `docstring`
6. Получить список локальных переменных
7. Проанализировать декораторы
8. Собрать метаданные и аннотации типов

🔗 Подсказка:

Начните с определения объекта, затем постепенно изучайте его структуру, параметры, исходный код и дополнительные метаданные.

Ответ:

- Определить объект для анализа
- Проверить тип объекта (функция, класс, метод)
- Получить сигнатуру функции
- Посмотреть исходный код
- Извлечь комментарии и docstring
- Получить список локальных переменных
- Проанализировать декораторы
- Собрать метаданные и аннотации типов

Урок 2 - Динамическая загрузка модулей через importlib

Шаг 01:

Миссия 1

Что делает функция `importlib.import_module()`?

Подсказка: Она позволяет загружать модуль во время выполнения программы.

Ответ:

- **Динамически импортирует модуль по имени**
- Компилирует модуль в байткод
- Удаляет модуль из памяти
- Перезагружает интерпретатор Python

Шаг 02:

Миссия 2

Какой аргумент требуется передать `import_module` для загрузки вложенного модуля?

Подсказка: Используйте точечную нотацию для вложенных модулей.

Ответ:

- **Строку с полным путем через точки (например 'package.submodule')**
- Список имен модулей
- Объект модуля Python
- URL к модулю

Шаг 03:

Миссия 3

 Сопоставьте функции `importlib` с их назначением

Понятия:

Ответ:

- `import_module`
- `reload`
- `util.find_spec`
- `invalidate_caches`

Определения:

1. Импортирует модуль по имени
2. Перезагружает уже загруженный модуль

3. Находит спецификацию модуля
4. Сбрасывает внутренние кэши `importlib`

 Подсказка:

Обратите внимание на разницу между импортом, перезагрузкой и поиском спецификации.

- **`import_module`** — Импортирует модуль по имени
- **`reload`** — Перезагружает уже загруженный модуль
- **`util.find_spec`** — Находит спецификацию модуля
- **`invalidate_caches`** — Сбрасывает внутренние кэши `importlib`

 Шаг 04:

Миссия 4

Расположите шаги для динамического импорта модуля и вызова его функции:

1. Импортировать `importlib`
2. Загрузить модуль через `import_module()`
3. Получить функцию из модуля через `getattr()`
4. Вызвать функцию с необходимыми аргументами

 Подсказка:

Сначала импортируем модуль, потом получаем функцию и только после этого её вызываем.

Ответ:

- Импортировать `importlib`
- Загрузить модуль через `import_module()`
- Получить функцию из модуля через `getattr()`
- Вызвать функцию с необходимыми аргументами

 Шаг 05:

Миссия 5

Что будет, если попытаться импортировать несуществующий модуль через `import_module`?

Подсказка: Подумайте о стандартных исключениях Python при импорте.

Ответ:

- **Возникнет исключение `ModuleNotFoundError`**
- Модуль будет создан автоматически
- Python проигнорирует ошибку и продолжит работу
- Модуль загрузится как пустой объект

 Шаг 06:

Миссия 6

 Сопоставьте методы `importlib.util` с их функционалом

Понятия:

Ответ:

- `spec_from_file_location`
- `module_from_spec`
- `exec_module`
- `invalidate_caches`

Определения:

1. Создает спецификацию модуля из файла
2. Создает объект модуля по спецификации
3. Выполняет код модуля и инициализирует его
4. Очищает внутренние кэши поиска модулей

 **Подсказка:**

Следите за последовательностью: сначала спецификация, затем объект модуля, потом выполнение.

- **spec_from_file_location** — Создает спецификацию модуля из файла
- **module_from_spec** — Создает объект модуля по спецификации
- **exec_module** — Выполняет код модуля и инициализирует его
- **invalidate_caches** — Очищает внутренние кэши поиска модулей

 **Шаг 07:**

Миссия 7

Как проверить, был ли модуль уже загружен в `sys.modules` перед динамическим импортом?

Подсказка: `sys.modules` хранит все загруженные модули.

Ответ:

- **Проверить наличие имени модуля в словаре `sys.modules`**
- Попытаться импортировать модуль и ловить исключение `ImportError`
- Сравнить модуль с `globals()`
- Вызвать `dir()` без аргументов

 **Шаг 08:**

Миссия 8

 Сопоставьте типы исключений при динамическом импорте с их причиной

Понятия:

Ответ:

- `ModuleNotFoundError`
- `ImportError`
- `AttributeError`
- `SyntaxError`

Определения:

1. Модуль не найден в путях поиска
2. Ошибка импорта из-за несовпадения имен или проблем с пакетом
3. Попытка получить функцию или объект, которого нет в модуле
4. Ошибка синтаксиса в загружаемом модуле

 **Подсказка:**

Внимательно различайте ошибки импорта модуля и ошибки при доступе к его содержимому.

- **`ModuleNotFoundError`** — Модуль не найден в путях поиска
- **`ImportError`** — Ошибка импорта из-за несовпадения имен или проблем с пакетом
- **`AttributeError`** — Попытка получить функцию или объект, которого нет в модуле
- **`SyntaxError`** — Ошибка синтаксиса в загружаемом модуле

Шаг 09:

Миссия 9

Что нужно сделать перед перезагрузкой модуля через `importlib.reload()`?

Подсказка: Модуль должен уже быть загружен.

Ответ:

- **Убедиться, что модуль уже импортирован**
- Удалить модуль из `sys.modules`
- Вызвать `exec_module` на новом объекте модуля
- Проверить, что модуль не содержит ошибок синтаксиса

Шаг 10:

Миссия 10

Расположите действия при загрузке модуля через `spec_from_file_location` и `module_from_spec`:

1. Создать спецификацию модуля
2. Создать объект модуля через `module_from_spec`
3. Выполнить код модуля через `exec_module`
4. Добавить модуль в `sys.modules` (при необходимости)

 **Подсказка:**

Сперва спецификация, затем объект, выполнение и регистрация в `sys.modules`.

Ответ:

- Создать спецификацию модуля
- Создать объект модуля через `module_from_spec`
- Выполнить код модуля через `exec_module`
- Добавить модуль в `sys.modules` (при необходимости)

Шаг 11:

Миссия 11

 Сопоставьте атрибуты объекта модуля с их функцией

Понятия:

Ответ:

- `__name__`
- `__file__`
- `__package__`
- `__dict__`

Определения:

1. Имя модуля
2. Путь к файлу модуля
3. Имя пакета, которому принадлежит модуль
4. Словарь всех атрибутов модуля

 **Подсказка:**

Эти атрибуты полезны для анализа загруженных модулей в runtime.

- `__name__` — Имя модуля

- **`__file__`** — Путь к файлу модуля
- **`__package__`** — Имя пакета, которому принадлежит модуль
- **`__dict__`** — Словарь всех атрибутов модуля

🔗 Шаг 12:

Миссия 12

Что делает `importlib.invalidate_caches()`?

Подсказка: Полезно при изменении файлов модулей вне интерпретатора.

Ответ:

- **Очищает кэши поиска модулей**
- Удаляет все загруженные модули
- Перезагружает интерпретатор
- Создаёт новые кэши для модулей

🔗 Шаг 13:

Миссия 13

Что произойдет, если вызвать `importlib.reload()` для модуля, который не был импортирован?

Подсказка: Перезагрузка возможна только для существующих объектов модулей.

Ответ:

- **Будет вызвано исключение `NameError` или `ModuleNotFoundError`**
- Модуль будет автоматически импортирован и перезагружен
- Python проигнорирует вызов и ничего не произойдет
- Создаст новый пустой объект модуля

🔗 Шаг 14:

Миссия 14

📄 Сопоставьте функции `importlib.util` с их назначением

Понятия:

Ответ:

- `spec_from_file_location`
- `module_from_spec`
- `exec_module`
- `invalidate_caches`

Определения:

1. Создает спецификацию модуля на основе пути к файлу
2. Создает объект модуля на основе спецификации
3. Выполняет код модуля
4. Сбрасывает внутренние кэши `importlib`

🔗 **Подсказка:**

Подумайте, в каком порядке нужно подготовить модуль к загрузке и выполнению.

- **`spec_from_file_location`** — Создает спецификацию модуля на основе пути к файлу
- **`module_from_spec`** — Создает объект модуля на основе спецификации
- **`exec_module`** — Выполняет код модуля

- **invalidate_caches** — Сбрасывает внутренние кэши importlib

🔮 Шаг 15:

Миссия 15

Для чего используется getattr() при динамическом импорте модуля?

Подсказка: Модуль уже загружен, нужно получить его содержимое.

Ответ:

- **Получить функцию или переменную из объекта модуля по имени**
- Перезагрузить модуль
- Создать новый модуль с этим именем
- Добавить модуль в sys.modules

🔮 Шаг 16:

Миссия 16

Расположите шаги для динамического импорта модуля из файла и вызова функции:

1. Создать спецификацию модуля через spec_from_file_location
2. Создать объект модуля через module_from_spec
3. Выполнить код модуля через exec_module
4. Получить функцию через getattr
5. Вызвать функцию с нужными аргументами

🔮 **Подсказка:**

Следуйте логике: спецификация → объект → выполнение → доступ к функции → вызов.

Ответ:

- Создать спецификацию модуля через spec_from_file_location
- Создать объект модуля через module_from_spec
- Выполнить код модуля через exec_module
- Получить функцию через getattr
- Вызвать функцию с нужными аргументами

📖 Урок 3 - Hot reload обновление кода без перезапуска

🔮 Шаг 01:

Миссия 1

Что делает importlib.import_module('math')?

Подсказка: Подумайте, что происходит при обычном import.

Ответ:

- **Импортирует модуль math и возвращает его объект**
- Создает новый модуль math
- Удаляет модуль math из sys.modules
- Перезагружает модуль math

🔮 Шаг 02:

Миссия 2

Что делает importlib.reload(module)?

Подсказка: Перегрузка модуля — это не его повторный импорт.

Ответ:

- **Перезагружает уже импортированный модуль**
- Импортирует модуль в первый раз
- Удаляет модуль из памяти
- Создает копию модуля

Шаг 03:

Миссия 3

Как проверить, какие функции определены в модуле?

Подсказка: Используйте стандартные средства Python для инспекции объекта.

Ответ:

- **Использовать `dir(module)`**
- Использовать `import_module(module)`
- Удалить модуль из `sys.modules`
- Вызвать `getattr(module)` без аргументов

Шаг 04:

Миссия 4

Сопоставьте функции `importlib` с их назначением

Понятия:

Ответ:

- `import_module`
- `reload`
- `invalidate_caches`
- `find_spec`

Определения:

1. Импортирует модуль по имени
2. Перезагружает существующий модуль
3. Сбрасывает внутренние кэши импорта
4. Находит спецификацию модуля без его выполнения

 **Подсказка:**

Обратите внимание на цель каждой функции: импорт, перезагрузка, кэш или спецификация.

- **`import_module`** — Импортирует модуль по имени
- **`reload`** — Перезагружает существующий модуль
- **`invalidate_caches`** — Сбрасывает внутренние кэши импорта
- **`find_spec`** — Находит спецификацию модуля без его выполнения

Шаг 05:

Миссия 5

Сопоставьте этапы динамической загрузки модуля через `importlib`

Ответ:

- `spec_from_file_location`

- `module_from_spec`
- `exec_module`
- `getattr`

1. Создает спецификацию модуля по файлу
2. Создает объект модуля на основе спецификации
3. Выполняет код модуля
4. Получает объект функции или переменной модуля по имени

🔗 Подсказка:

Последовательность: спецификация → объект → выполнение → доступ к атрибутам.

- **`spec_from_file_location`** — Создает спецификацию модуля по файлу
- **`module_from_spec`** — Создает объект модуля на основе спецификации
- **`exec_module`** — Выполняет код модуля
- **`getattr`** — Получает объект функции или переменной модуля по имени

🔗 Шаг 06:

Миссия 6

Что происходит при вызове `importlib.invalidate_caches()`?

Подсказка: Кэш используется для ускорения поиска модулей.

Ответ:

- **Сбрасываются внутренние кэши поиска модулей**
- Удаляются все модули из `sys.modules`
- Модули перезагружаются автоматически
- Создается новый модуль в памяти

🔗 Шаг 07:

Миссия 7

Зачем использовать `getattr(module, 'func_name')` после динамического импорта?

Подсказка: Модуль уже загружен, нужно получить конкретный объект.

Ответ:

- **Чтобы получить функцию или переменную модуля по имени**
- Чтобы перезагрузить модуль
- Чтобы создать новый модуль
- Чтобы удалить модуль

🔗 Шаг 08:

Миссия 8

Сопоставьте термины с примерами использования `importlib`

Ответ:

- Динамический импорт
 - Перезагрузка модуля
 - Сброс кэшей
 - Получение функции
1. `module = import_module('os')`
 2. `reload(module)`

3. `invalidate_caches()`
4. `func = getattr(module, 'path')`

🔗 Подсказка:

Каждая функция имеет одно явное применение.

- **Динамический импорт** — `module = import_module('os')`
- **Перезагрузка модуля** — `reload(module)`
- **Сброс кэшей** — `invalidate_caches()`
- **Получение функции** — `func = getattr(module, 'path')`

🔗 Шаг 09:

Миссия 9

Что вернет `importlib.util.module_from_spec(spec)`?

Подсказка: Спецификация уже создана.

Ответ:

- **Объект модуля, готовый к выполнению**
- Только путь к файлу модуля
- Словарь всех функций модуля
- Список атрибутов модуля

🔗 Шаг 10:

Миссия 10

Расположите шаги для вызова функции из внешнего файла через `importlib`:

1. Создать спецификацию модуля `spec_from_file_location`
2. Создать объект модуля `module_from_spec`
3. Выполнить код модуля `exec_module`
4. Получить функцию через `getattr`
5. Вызвать функцию

🔗 Подсказка:

Следуйте логике: подготовка → объект → выполнение → доступ → вызов.

Ответ:

- Создать спецификацию модуля `spec_from_file_location`
- Создать объект модуля `module_from_spec`
- Выполнить код модуля `exec_module`
- Получить функцию через `getattr`
- Вызвать функцию

🔗 Шаг 11:

Миссия 11

Расположите шаги для динамического импорта и вызова функции:

1. Создать спецификацию модуля через `spec_from_file_location`
2. Создать объект модуля через `module_from_spec`
3. Выполнить модуль через `exec_module`
4. Получить функцию через `getattr`
5. Вызвать функцию

🔗 Подсказка:

Логика: подготовка → объект → выполнение → доступ → вызов.

Ответ:

- Создать спецификацию модуля через `spec_from_file_location`
- Создать объект модуля через `module_from_spec`
- Выполнить модуль через `exec_module`
- Получить функцию через `getattr`
- Вызвать функцию

🔗 Шаг 12:

Миссия 12

Что делает `importlib.util.find_spec('module_name')`?

Подсказка: Спецификация нужна для подготовки модуля к загрузке.

Ответ:

- **Возвращает объект спецификации модуля без его выполнения**
- Импортирует модуль и выполняет его код
- Удаляет модуль из `sys.modules`
- Создает пустой модуль в памяти

🔗 Шаг 13:

Миссия 13

Сопоставьте методы `importlib.util` с их назначением

Ответ:

- `spec_from_file_location`
 - `module_from_spec`
 - `exec_module`
 - `find_spec`
1. Создает спецификацию модуля по файлу
 2. Создает объект модуля из спецификации
 3. Выполняет код модуля
 4. Возвращает объект спецификации по имени модуля

🔗 Подсказка:

Запомните: `spec` → `module` → `exec`, а `find_spec` используется отдельно для поиска.

- **`spec_from_file_location`** — Создает спецификацию модуля по файлу
- **`module_from_spec`** — Создает объект модуля из спецификации
- **`exec_module`** — Выполняет код модуля
- **`find_spec`** — Возвращает объект спецификации по имени модуля

🔗 Шаг 14:

Миссия 14

Что произойдет, если вызвать `reload()` на неимпортированном модуле?

Подсказка: `reload` работает только с уже загруженными модулями.

Ответ:

- Будет ошибка, так как модуль не загружен
- Модуль будет автоматически импортирован и перезагружен
- Создается новый модуль в памяти
- Ничего не произойдет

🔗 Шаг 15:

Миссия 15

Расположите действия для получения функции из модуля на диске:

1. Создать спецификацию через `spec_from_file_location`
2. Создать объект модуля через `module_from_spec`
3. Выполнить код модуля `exec_module`
4. Получить функцию через `getattr`

🔗 Подсказка:

Последовательность строго следует логике подготовки и исполнения модуля.

Ответ:

- Создать спецификацию через `spec_from_file_location`
- Создать объект модуля через `module_from_spec`
- Выполнить код модуля `exec_module`
- Получить функцию через `getattr`

🔗 Шаг 16:

Миссия 16

Когда полезно использовать `importlib` для загрузки модулей?

Подсказка: Подумайте о плагинах или динамических расширениях.

Ответ:

- Когда модули нужно загружать динамически во время работы программы
- Для обычного статического импорта всех модулей
- Чтобы удалить модуль из памяти
- Чтобы автоматически перезагружать все модули при старте

🔗 Шаг 17:

Миссия 17

Сопоставьте действия с правильным методом `importlib`

Ответ:

- Динамический импорт модуля
 - Перезагрузка модуля
 - Получение функции по имени
 - Сброс кэшей поиска
1. `import_module('module')`
 2. `reload(module)`
 3. `getattr(module, 'func')`
 4. `invalidate_caches()`

Подсказка:

Каждое действие соответствует отдельной функции `importlib`.

- **Динамический импорт модуля** — `import_module('module')`
- **Перезагрузка модуля** — `reload(module)`
- **Получение функции по имени** — `getattr(module, 'func')`
- **Сброс кэшей поиска** — `invalidate_caches()`

Урок 4 - Практика runtime-анализ API и функций

Шаг 01:

Миссия 1

Что делает функция `dir(obj)` в Python?

Подсказка: Она позволяет увидеть «внутреннюю жизнь» объекта.

Ответ:

- **Возвращает список атрибутов и методов объекта**
- Удаляет все методы объекта
- Изменяет имя объекта
- Создает новый объект с теми же атрибутами

Шаг 02:

Миссия 2

Как с помощью модуля `inspect` узнать список аргументов функции?

Подсказка: Смотрите на сигнатуру функции, а не на её тело.

Ответ:

- **`inspect.signature(func).parameters`**
- `dir(func)`
- `getattr(func)`
- `help(func)`

Шаг 03:

Миссия 3

Что показывает функция `help(func)` для объекта функции?

Подсказка: Подумайте, где Python хранит документацию.

Ответ:

- **Документацию и сигнатуру функции**
- Список всех модулей проекта
- Весь исходный код файла
- Перечень всех переменных в памяти

Шаг 04:

Миссия 4

Сопоставьте объекты runtime-анализа с их назначением:

Ответ:

- `dir(obj)`

- `inspect.signature(func)`
- `help(obj)`
- `getattr(obj, name)`

1. Возвращает список атрибутов объекта
2. Возвращает сигнатуру функции с аргументами
3. Выводит документацию объекта
4. Возвращает значение атрибута по имени

🔗 Подсказка:

Думайте, какая функция «рассматривает» объект, а какая «вызывает» его свойства.

- **`dir(obj)`** — Возвращает список атрибутов объекта
- **`inspect.signature(func)`** — Возвращает сигнатуру функции с аргументами
- **`help(obj)`** — Выводит документацию объекта
- **`getattr(obj, name)`** — Возвращает значение атрибута по имени

🔗 Шаг 05:

Миссия 5

Как проверить, является ли объект функцией в Python?

Подсказка: Смотрите на тип объекта.

Ответ:

- **`isinstance(obj, types.FunctionType)`**
- `dir(obj)`
- `help(obj)`
- `getattr(obj, 'func')`

🔗 Шаг 06:

Миссия 6

Расположите действия при runtime-анализе функции:

1. Импортировать модуль `inspect`
2. Получить объект функции
3. Получить сигнатуру через `inspect.signature`
4. Извлечь список аргументов
5. Использовать `getattr` или другие методы для вызова

🔗 Подсказка:

Сначала подготовка, потом получение объекта, потом изучение и использование.

Ответ:

- Импортировать модуль `inspect`
- Получить объект функции
- Получить сигнатуру через `inspect.signature`
- Извлечь список аргументов
- Использовать `getattr` или другие методы для вызова

🔗 Шаг 07:

Миссия 7

Что возвращает `obj.__doc__` ?

Подсказка: Вспомните, где хранится описание объекта.

Ответ:

- **Документацию (docstring) объекта**
- Список всех атрибутов объекта
- Тип объекта
- Результат выполнения функции

Шаг 08:

Миссия 8

Сопоставьте методы introspection с их назначением:

Ответ:

- `__doc__`
 - `__name__`
 - `__module__`
 - `__annotations__`
1. Документация объекта
 2. Имя объекта
 3. Модуль, где определен объект
 4. Аннотации аргументов функции

Подсказка:

Каждый атрибут хранит разную метаинформацию о функции или классе.

- **`__doc__`** — Документация объекта
- **`__name__`** — Имя объекта
- **`__module__`** — Модуль, где определен объект
- **`__annotations__`** — Аннотации аргументов функции

Шаг 09:

Миссия 9

Для чего используется `callable(obj)` ?

Подсказка: Подумайте, можно ли «вызывать» объект как функцию.

Ответ:

- **Проверка, можно ли вызвать объект как функцию**
- Определение типа объекта
- Получение списка атрибутов
- Извлечение документации объекта

Шаг 10:

Миссия 10

Расположите шаги при динамическом вызове функции через `getattr`:

1. Получить объект модуля
2. Вызвать `getattr(module, 'function_name')`
3. Вызвать функцию через полученный объект
4. Обработать возможные исключения

🔗 Подсказка:

Логика: подготовка → получение → вызов → обработка ошибок.

Ответ:

- Получить объект модуля
- Вызвать `getattr(module, 'function_name')`
- Вызвать функцию через полученный объект
- Обработать возможные исключения

🔗 Шаг 11:

Миссия 11

Расположите шаги при runtime-анализе класса:

1. Импортировать `inspect`
2. Получить объект класса
3. Использовать `dir()` для просмотра атрибутов
4. Использовать `inspect.getmembers()` для детального анализа
5. Проанализировать методы и свойства

🔗 Подсказка:

Сначала подготовка, затем изучение базовых атрибутов, потом расширенный анализ.

Ответ:

- Импортировать `inspect`
- Получить объект класса
- Использовать `dir()` для просмотра атрибутов
- Использовать `inspect.getmembers()` для детального анализа
- Проанализировать методы и свойства

🔗 Шаг 12:

Миссия 12

Что делает `inspect.getmembers(obj)`?

Подсказка: Подумайте, как получить полную информацию об объекте.

Ответ:

- **Возвращает все члены объекта (атрибуты и методы)**
- Удаляет методы объекта
- Возвращает только публичные атрибуты
- Возвращает документацию объекта

🔗 Шаг 13:

Миссия 13

Сопоставьте методы introspection с их назначением:

Ответ:

- `inspect.getmembers()`
- `inspect.isfunction()`
- `inspect.ismethod()`

- `inspect.isclass()`

1. Проверяет, является ли объект функцией
2. Проверяет, является ли объект методом
3. Возвращает все члены объекта
4. Проверяет, является ли объект классом

Подсказка:

Каждая функция модуля `inspect` имеет специфическую цель: проверка или извлечение информации.

- **`inspect.getmembers()`** — Возвращает все члены объекта
- **`inspect.isfunction()`** — Проверяет, является ли объект функцией
- **`inspect.ismethod()`** — Проверяет, является ли объект методом
- **`inspect.isclass()`** — Проверяет, является ли объект классом

Шаг 14:

Миссия 14

Как получить список всех аргументов метода класса?

Подсказка: Смотрите на сигнатуру, а не на `dir()`.

Ответ:

- **`inspect.signature(Class.method).parameters`**
- `dir(Class.method)`
- `Class.__dict__`
- `getattr(Class, 'method')`

Шаг 15:

Миссия 15

Расположите этапы анализа модуля API:

1. Импортировать модуль
2. Использовать `dir()` для первичного осмотра
3. Использовать `inspect.getmembers()` для детального анализа
4. Определить функции и классы
5. Получить документацию и аннотации

Подсказка:

От простого к сложному: сначала обзор, потом подробности.

Ответ:

- Импортировать модуль
- Использовать `dir()` для первичного осмотра
- Использовать `inspect.getmembers()` для детального анализа
- Определить функции и классы
- Получить документацию и аннотации

Шаг 16:

Миссия 16

Для чего нужен `__annotations__` в функции?

Подсказка: Подумайте о типах аргументов и возвращаемого значения.

Ответ:

- **Хранит аннотации типов аргументов и возвращаемого значения**
- Список всех атрибутов функции
- Документацию функции
- Сигнатуру функции без типов

🔮 **Шаг 17:**

Миссия 17

Сопоставьте методы runtime-анализа с их результатом:

Ответ:

- `inspect.getdoc(obj)`
 - `inspect.getfile(obj)`
 - `inspect.getsource(obj)`
 - `inspect.signature(obj)`
1. Возвращает docstring объекта
 2. Возвращает путь к файлу, где определен объект
 3. Возвращает исходный код функции или класса
 4. Возвращает сигнатуру функции

🔮 **Подсказка:**

Каждая функция возвращает разные сведения: документацию, код, файл или сигнатуру.

- **`inspect.getdoc(obj)`** — Возвращает docstring объекта
- **`inspect.getfile(obj)`** — Возвращает путь к файлу, где определен объект
- **`inspect.getsource(obj)`** — Возвращает исходный код функции или класса
- **`inspect.signature(obj)`** — Возвращает сигнатуру функции

📖 **Урок 5 - Лабиринт генерация кода и проверка типов**

🔮 **Шаг 01:**

Миссия 1

Что делает функция `eval()` в Python?

Подсказка: Подумайте о том, как выполнить строку как код.

Ответ:

- **Выполняет строку как Python-код и возвращает результат**
- Проверяет тип объекта
- Создает новый модуль
- Компилирует код в байткод, но не выполняет

🔮 **Шаг 02:**

Миссия 2

Для чего используется функция `compile()`?

Подсказка: Подумайте, как подготовить строку к выполнению позже.

Ответ:

- **Компилирует строку Python-кода в объект кода для последующего выполнения**
- Выполняет код напрямую

- Сохраняет код в файл
- Проверяет синтаксис без компиляции

🔮 Шаг 03:

Миссия 3

Сопоставьте функции генерации кода с их назначением:

Ответ:

- `eval()`
 - `exec()`
 - `compile()`
 - `type()`
1. Возвращает объект кода для выполнения позже
 2. Создает новый объект типа
 3. Выполняет строку как код и возвращает результат
 4. Выполняет строку как код без возвращаемого значения

🔮 Подсказка:

Каждая функция имеет свой сценарий: либо выполнение кода, либо проверка или создание объектов.

- **`eval()`** — Выполняет строку как код и возвращает результат
- **`exec()`** — Выполняет строку как код без возвращаемого значения
- **`compile()`** — Возвращает объект кода для выполнения позже
- **`type()`** — Создает новый объект типа

🔮 Шаг 04:

Миссия 4

Что делает функция `isinstance(obj, cls)`?

Подсказка: Проверьте соответствие объекта типу или классу.

Ответ:

- **Проверяет, принадлежит ли объект `cls` или его подклассу**
- Создает новый класс
- Компилирует объект в байткод
- Возвращает список методов объекта

🔮 Шаг 05:

Миссия 5

Сопоставьте методы проверки типов с их результатом:

Ответ:

- `isinstance(obj, cls)`
 - `issubclass(sub, base)`
 - `type(obj)`
 - `callable(obj)`
1. Проверяет, является ли объект экземпляром класса или подкласса
 2. Проверяет, является ли класс подклассом другого класса
 3. Возвращает тип объекта

4. Проверяет, можно ли вызвать объект как функцию

Подсказка:

Различайте проверку типа и проверку наследования или вызова.

- **isinstance(obj, cls)** — Проверяет, является ли объект экземпляром класса или подкласса
- **issubclass(sub, base)** — Проверяет, является ли класс подклассом другого класса
- **type(obj)** — Возвращает тип объекта
- **callable(obj)** — Проверяет, можно ли вызвать объект как функцию

Шаг 06:

Миссия 6

Как безопасно выполнить строку кода, не возвращая результат?

Подсказка: Используйте функцию, которая подходит для действий без возврата значения.

Ответ:

- **exec()**
- eval()
- compile()
- type()

Шаг 07:

Миссия 7

Расположите этапы безопасной генерации и проверки кода:

1. Написать строку кода
2. Компилировать строку с compile()
3. Проверить типы аргументов с isinstance()
4. Выполнить код с exec() или eval()
5. Обработать результат

Подсказка:

Сначала подготовка кода, потом проверка, затем выполнение и анализ результата.

Ответ:

- Написать строку кода
- Компилировать строку с compile()
- Проверить типы аргументов с isinstance()
- Выполнить код с exec() или eval()
- Обработать результат

Шаг 08:

Миссия 8

Что произойдет, если выполнить eval('2 + 2')?

Подсказка: Подумайте о возвращаемом результате выражения.

Ответ:

- **Вернется значение 4**
- Выведется строка '2 + 2'
- Произойдет ошибка времени выполнения

- Ничего не произойдет

🔮 Шаг 09:

Миссия 9

Сопоставьте методы генерации кода с их безопасностью:

Ответ:

- `eval()`
 - `exec()`
 - `compile()`
 - `ast.literal_eval()`
1. Самый безопасный вариант для вычисления литералов
 2. Выполняет код с возможностью побочных эффектов
 3. Компилирует код, не выполняя его
 4. Выполняет код и возвращает результат, потенциально небезопасно

🔮 Подсказка:

Помните, что `eval()` и `exec()` могут выполнять произвольный код.

- **`eval()`** — Выполняет код и возвращает результат, потенциально небезопасно
- **`exec()`** — Выполняет код с возможностью побочных эффектов
- **`compile()`** — Компилирует код, не выполняя его
- **`ast.literal_eval()`** — Самый безопасный вариант для вычисления литералов

🔮 Шаг 10:

Миссия 10

Что вернет `type([])`?

Подсказка: Смотрите на тип объекта в Python.

Ответ:

-
-
-
-

🔮 Шаг 11:

Миссия 11

Сопоставьте функции проверки объектов с примером:

Ответ:

- `callable()`
 - `isinstance()`
 - `issubclass()`
 - `type()`
1. Функция `foo = lambda x: x; callable(foo)`
 2. `isinstance(42, int)`
 3. `issubclass(bool, int)`
 4. `type('text')`

🔮 Подсказка:

Каждая функция выполняет уникальную проверку: тип, наследование или вызываемость.

- **callable()** — Функция `foo = lambda x: x; callable(foo)`
- **isinstance()** — `isinstance(42, int)`
- **issubclass()** — `issubclass(bool, int)`
- **type()** — `type('text')`

🔮 Шаг 12:

Миссия 12

Что делает `exec('x=5')` в текущем пространстве имен?

Подсказка: Подумайте о присвоении переменной внутри `exec`.

Ответ:

- **Создает переменную `x` со значением 5 в текущем пространстве имен**
- Возвращает значение 5
- Удаляет переменную `x`
- Выдает синтаксическую ошибку

🔮 Шаг 13:

Миссия 13

Расположите этапы выполнения безопасного кода:

1. Написать код как строку
2. Компилировать код через `compile()`
3. Проверить аргументы и типы
4. Выполнить код с `eval()` или `exec()`
5. Обработать и проверить результат

🔮 Подсказка:

Порядок важен: подготовка → проверка → выполнение → обработка результата.

Ответ:

- Написать код как строку
- Компилировать код через `compile()`
- Проверить аргументы и типы
- Выполнить код с `eval()` или `exec()`
- Обработать и проверить результат

🔮 Шаг 14:

Миссия 14

Для чего нужен `ast.literal_eval()`?

Подсказка: Он безопаснее `eval` для литералов.

Ответ:

- **Вычисляет литералы Python безопасно, без выполнения произвольного кода**
- Компилирует весь модуль
- Выполняет код с побочными эффектами
- Создает новый объект типа

Шаг 15:

Миссия 15

Какой модуль нужно импортировать для безопасного преобразования строк в Python-объекты?

Подсказка: Сосредоточьтесь на модулях, которые работают только с литералами.

Ответ:

- **ast**
- os
- sys
- importlib

Шаг 16:

Миссия 16

Расположите этапы безопасного анализа и выполнения динамически сгенерированного кода:

1. Проверить аргументы и типы
2. Составить код как строку
3. Компилировать код через `compile()`
4. Выполнить код с `exec()` или `eval()`
5. Обработать результат и проверить корректность

 **Подсказка:**

Важно соблюдать строгий порядок: подготовка → проверка → компиляция → выполнение → проверка результата.

Ответ:

- Проверить аргументы и типы
- Составить код как строку
- Компилировать код через `compile()`
- Выполнить код с `exec()` или `eval()`
- Обработать результат и проверить корректность

Модуль 4 - Meta-DSL и мини-языки

Урок 1 - Основы создания DSL на Python

Шаг 01:

Миссия 1

Что такое DSL (Domain-Specific Language)?

Подсказка: Это язык, созданный для решения конкретных задач, а не общего программирования.

Ответ:

- **Язык, оптимизированный для конкретной области задач**
- Общий язык программирования типа Python
- Язык разметки HTML
- Язык запросов SQL

🔮 Шаг 02:

Миссия 2

Какой подход удобен для создания мини-языков в Python?

Подсказка: Обратите внимание на возможности языка создавать динамические конструкции.

Ответ:

- **Использование классов и функций для выражения DSL-конструкций**
- Создание отдельного языка с нуля на C
- Использование только глобальных переменных
- Игнорирование типов данных

🔮 Шаг 03:

Миссия 3

Что упрощает чтение и использование DSL для программистов?

Подсказка: Подумайте о том, как сделать синтаксис естественным для пользователя.

Ответ:

- **Простая и понятная абстракция**
- Сложные вложенные конструкции
- Использование только чисел
- Полное копирование синтаксиса Python

🔮 Шаг 04:

Миссия 4

Сопоставьте концепцию DSL с её описанием.

Ответ:

- Мини-язык
 - Встроенный DSL
 - Внешний DSL
 - Абстракция
1. Язык, встроенный в существующий язык программирования
 2. Уровень упрощения и скрытия деталей реализации
 3. Отдельный язык, который взаимодействует с программой через API
 4. Локальный язык для конкретной задачи

Подсказка: Подумайте, где и как используется язык: внутри Python или отдельно.

- **Мини-язык** — Локальный язык для конкретной задачи
- **Встроенный DSL** — Язык, встроенный в существующий язык программирования
- **Внешний DSL** — Отдельный язык, который взаимодействует с программой через API
- **Абстракция** — Уровень упрощения и скрытия деталей реализации

🔮 Шаг 05:

Миссия 5

Сопоставьте типы DSL с их характерными особенностями.

Ответ:

- Командный DSL

- Конфигурационный DSL
- Функциональный DSL
- Скриптовый DSL

1. Позволяет описывать последовательность действий
2. Определяет настройки и параметры
3. Использует функции как основные строительные блоки
4. Позволяет автоматизировать процессы через сценарии

Подсказка: Обратите внимание на назначение DSL: команды, конфигурация или сценарии.

- **Командный DSL** — Позволяет описывать последовательность действий
- **Конфигурационный DSL** — Определяет настройки и параметры
- **Функциональный DSL** — Использует функции как основные строительные блоки
- **Скриптовый DSL** — Позволяет автоматизировать процессы через сценарии

🔮 Шаг 06:

Миссия 6

Что помогает при проектировании мини-языков в Python?

Подсказка: Думайте о динамике и возможностях метапрограммирования.

Ответ:

- **Метаклассы и функции высшего порядка**
- Использование только глобальных переменных
- Жесткое следование стандартным библиотекам
- Полное избегание классов

🔮 Шаг 07:

Миссия 7

Какой метод позволяет превратить строку кода в исполняемый объект Python?

Подсказка: Это встроенная функция для динамической компиляции.

Ответ:

- **compile()**
- eval()
- execfile()
- input()

🔮 Шаг 08:

Миссия 8

Сопоставьте элементы DSL с их назначением.

Ответ:

- Токен
 - Грамматика
 - Интерпретатор
 - Парсер
1. Основные единицы языка (слова, символы)
 2. Правила построения языка
 3. Выполняет код DSL

4. Преобразует текст в структуру данных

Подсказка: Подумайте о том, как текст DSL превращается в выполнение программы.

- **Токен** — Основные единицы языка (слова, символы)
- **Грамматика** — Правила построения языка
- **Интерпретатор** — Выполняет код DSL
- **Парсер** — Преобразует текст в структуру данных

Шаг 09:

Миссия 9

Сопоставьте элементы DSL с их назначением.

Ответ:

- Токен
 - Грамматика
 - Интерпретатор
 - Парсер
1. Основные единицы языка (слова, символы)
 2. Правила построения языка
 3. Выполняет код DSL
 4. Преобразует текст в структуру данных

Подсказка: Подумайте о том, как текст DSL превращается в выполнение программы.

- **Токен** — Основные единицы языка (слова, символы)
- **Грамматика** — Правила построения языка
- **Интерпретатор** — Выполняет код DSL
- **Парсер** — Преобразует текст в структуру данных

Шаг 10:

Миссия 10

Какой инструмент помогает проверить правильность синтаксиса DSL?

Подсказка: Подумайте о том, как программы проверяют корректность кода перед исполнением.

Ответ:

- **Парсер**
- Компилятор C++
- Линтер для CSS
- Сборщик Java

Шаг 11:

Миссия 11

Что обеспечивает гибкость мини-языка в Python?

Подсказка: Подумайте о динамических структурах данных и метапрограммировании.

Ответ:

- **Динамическое создание классов и функций**
- Только глобальные переменные
- Статические структуры данных

- Полная иммутабельность кода

Шаг 12:

Миссия 12

Сопоставьте понятия для работы с кодом DSL.

Ответ:

- `eval()`
- `exec()`
- `compile()`
- `ast.parse()`

1. Выполняет строку Python как выражение
2. Выполняет строку Python как блок кода
3. Компилирует строку в объект кода для последующего выполнения
4. Создаёт дерево синтаксического анализа Python

Подсказка: Обратите внимание на различия между выполнением и компиляцией кода.

- **`eval()`** — Выполняет строку Python как выражение
- **`exec()`** — Выполняет строку Python как блок кода
- **`compile()`** — Компилирует строку в объект кода для последующего выполнения
- **`ast.parse()`** — Создаёт дерево синтаксического анализа Python

Шаг 13:

Миссия 13

Для чего в мини-языках полезно использование функций высшего порядка?

Подсказка: Подумайте о том, как можно передавать поведение в виде объектов.

Ответ:

- **Для передачи и комбинирования поведения внутри DSL**
- Для хранения только числовых значений
- Для работы только с глобальными переменными
- Для создания статических констант

Шаг 14:

Миссия 14

Расположите этапы создания DSL в правильном порядке:

1. Определение синтаксиса
2. Создание парсера
3. Реализация интерпретатора
4. Тестирование DSL

Подсказка: Подумайте, что сначала нужно описать язык, затем разобрать код и выполнить.

Ответ:

- Определение синтаксиса
- Создание парсера
- Реализация интерпретатора
- Тестирование DSL

Шаг 15:

Миссия 15

Что обеспечивает повторное использование кода в мини-языках?

Подсказка: Подумайте о компонентах, которые можно легко комбинировать.

Ответ:

- **Абстракции и функции**
- Жёстко прописанные команды
- Только глобальные переменные
- Статические классы без параметров

Шаг 16:

Миссия 16

Сопоставьте термины, связанные с DSL, с их ролями.

Ответ:

- Лексер
 - Семантика
 - Интерфейс API
 - Тестовый сценарий
1. Разделяет текст на токены
 2. Определяет смысл конструкций DSL
 3. Позволяет программам взаимодействовать с DSL
 4. Проверяет корректность работы DSL

Подсказка: Обратите внимание на этапы: анализ текста, смысл, использование и проверка.

- **Лексер** — Разделяет текст на токены
- **Семантика** — Определяет смысл конструкций DSL
- **Интерфейс API** — Позволяет программам взаимодействовать с DSL
- **Тестовый сценарий** — Проверяет корректность работы DSL

Шаг 17:

Миссия 17

Расположите шаги выполнения кода мини-языка:

1. Парсинг текста DSL
2. Создание структуры данных
3. Интерпретация или выполнение кода
4. Возврат результата пользователю

Подсказка: Подумайте, как текст проходит путь от строки к действию.

Ответ:

- Парсинг текста DSL
- Создание структуры данных
- Интерпретация или выполнение кода
- Возврат результата пользователю

Шаг 18:

Миссия 18

Что важно для поддержки DSL на долгосрочной перспективе?

Подсказка: Подумайте о читаемости и простоте изменений кода.

Ответ:

- **Документация и модульная структура**
- Сложный и запутанный синтаксис
- Жёсткая зависимость от глобальных переменных
- Игнорирование тестирования

Урок 2 - Преобразование DSL в Python AST

Шаг 01:

Миссия 1

Какой модуль Python позволяет создавать и анализировать AST?

Подсказка: Подумайте о стандартной библиотеке для работы с деревьями синтаксиса.

Ответ:

- **ast**
- json
- re
- math

Шаг 02:

Миссия 2

Что выполняет функция `ast.parse()`?

Подсказка: Подумайте о том, как строка Python превращается в структуру данных.

Ответ:

- **Преобразует код Python в дерево AST**
- Выполняет код Python
- Компилирует Python в байткод
- Сохраняет код в файл

Шаг 03:

Миссия 3

Сопоставьте элементы AST с их назначением.

Ответ:

- Module
 - FunctionDef
 - Assign
 - Expr
1. Корневой узел всего скрипта
 2. Определение функции
 3. Присваивание значения переменной
 4. Выражение, выполняемое как инструкция

Подсказка: Подумайте, как строится дерево кода от корня к конкретным действиям.

- **Module** — Корневой узел всего скрипта
- **FunctionDef** — Определение функции
- **Assign** — Присваивание значения переменной
- **Expr** — Выражение, выполняемое как инструкция

🔗 Шаг 04:

Миссия 4

Для чего полезен `ast.dump()`?

Подсказка: Подумайте, как увидеть структуру дерева синтаксиса в удобном виде.

Ответ:

- **Выводит структуру AST в виде строки**
- Выполняет код AST
- Компилирует код Python
- Сохраняет AST в JSON

🔗 Шаг 05:

Миссия 5

Сопоставьте методы AST с их функциями.

Ответ:

- `ast.parse()`
 - `ast.dump()`
 - `ast.NodeVisitor`
 - `ast.NodeTransformer`
1. Преобразует код в AST
 2. Выводит структуру AST
 3. Позволяет обходить узлы дерева
 4. Позволяет модифицировать узлы дерева

Подсказка: Обратите внимание на различие между просмотром и изменением AST.

- **`ast.parse()`** — Преобразует код в AST
- **`ast.dump()`** — Выводит структуру AST
- **`ast.NodeVisitor`** — Позволяет обходить узлы дерева
- **`ast.NodeTransformer`** — Позволяет модифицировать узлы дерева

🔗 Шаг 06:

Миссия 6

Какой объект AST используется для хранения тела функции?

Подсказка: Подумайте, где хранятся инструкции внутри функции.

Ответ:

- **`body`**
- `args`
- `decorator_list`
- `returns`

Шаг 07:

Миссия 7

Расположите этапы обработки DSL кода в AST:

1. Получение текста DSL
2. Преобразование текста в Python код
3. Парсинг Python кода в AST
4. Обход и модификация AST
5. Выполнение Python кода

Подсказка: Подумайте о цепочке от текста до исполнения.

Ответ:

- Получение текста DSL
- Преобразование текста в Python код
- Парсинг Python кода в AST
- Обход и модификация AST
- Выполнение Python кода

Шаг 08:

Миссия 8

Что важно проверять при модификации AST?

Подсказка: Подумайте о корректности и синтаксисе Python после изменений.

Ответ:

- **Синтаксическую корректность и целостность дерева**
- Только производительность кода
- Наличие комментариев в коде
- Форматирование строковых литералов

Шаг 09:

Миссия 9

Сопоставьте типы AST узлов с их назначением.

Ответ:

- Call
 - BinOp
 - Name
 - Constant
1. Вызов функции
 2. Бинарная операция (сложение, умножение и т.д.)
 3. Имя переменной
 4. Константное значение

Подсказка: Подумайте о том, какие узлы представляют выражения, а какие — данные.

- **Call** — Вызов функции
- **BinOp** — Бинарная операция (сложение, умножение и т.д.)
- **Name** — Имя переменной
- **Constant** — Константное значение

🔧 Шаг 10:

Миссия 10

Для чего используют `ast.fix_missing_locations()`?

Подсказка: Подумайте о том, как Python узнает позицию каждой инструкции для компиляции.

Ответ:

- Добавляет отсутствующие координаты строк в AST
- Удаляет лишние комментарии
- Выполняет AST как код
- Компилирует AST в байткод напрямую

🔧 Шаг 11:

Миссия 11

Сопоставьте функции компиляции AST с их действиями.

Ответ:

- `compile()`
 - `eval()`
 - `exec()`
1. Компилирует AST или код Python в объект байткода
 2. Вычисляет выражение и возвращает результат
 3. Выполняет последовательность инструкций без возврата результата

Подсказка: Подумайте о различии между выражениями и инструкциями.

- **`compile()`** — Компилирует AST или код Python в объект байткода
- **`eval()`** — Вычисляет выражение и возвращает результат
- **`exec()`** — Выполняет последовательность инструкций без возврата результата

🔧 Шаг 12:

Миссия 12

Что произойдет, если изменить AST неправильно и выполнить код?

Подсказка: Подумайте о рисках нарушения синтаксиса Python.

Ответ:

- Возникнет **`SyntaxError`** или неправильное поведение программы
- Код выполнится быстрее без ошибок
- Python автоматически исправит AST
- Результат не изменится

🔧 Шаг 13:

Миссия 13

Расположите шаги трансформации DSL в Python AST:

1. Написание DSL сценария
2. Преобразование DSL в Python код
3. Парсинг Python кода в AST
4. Обход и модификация AST
5. Компиляция AST в байткод

6. Выполнение кода

Подсказка: Подумайте о логической цепочке от текста до исполнения.

Ответ:

- Написание DSL сценария
- Преобразование DSL в Python код
- Парсинг Python кода в AST
- Обход и модификация AST
- Компиляция AST в байткод
- Выполнение кода

Шаг 14:

Миссия 14

Что позволяет делать `ast.NodeTransformer`?

Подсказка: Подумайте, как можно изменить код до его выполнения.

Ответ:

- **Модифицировать узлы AST перед компиляцией**
- Исполнять Python код напрямую
- Преобразовывать байткод в AST
- Сохранять AST в файл JSON

Шаг 15:

Миссия 15

Какой метод AST используется для обхода всех узлов дерева без изменения?

Подсказка: Подумайте о разнице между просмотром и трансформацией AST.

Ответ:

- **`ast.NodeVisitor`**
- `ast.NodeTransformer`
- `ast.dump`
- `compile`

Шаг 16:

Миссия 16

Расположите шаги выполнения DSL сценария через AST:

1. Написание DSL кода
2. Преобразование DSL в Python код
3. Парсинг Python кода в AST
4. Модификация AST через `NodeTransformer`
5. Компиляция AST в байткод
6. Выполнение байткода Python

Подсказка: Подумайте о логической цепочке: от текста до исполнения программы.

Ответ:

- Написание DSL кода
- Преобразование DSL в Python код
- Парсинг Python кода в AST

- Модификация AST через NodeTransformer
- Компиляция AST в байткод
- Выполнение байткода Python

Урок 3 - Мини-SQL и математические DSL

Шаг 01:

Миссия 1

Какой оператор в мини-SQL выбирает все строки из таблицы?

Подсказка: Подумайте о ключевом слове, которое обычно используется для выборки.

Ответ:

- **SELECT**
- INSERT
- UPDATE
- DELETE

Шаг 02:

Миссия 2

Какой оператор в мини-SQL используется для фильтрации строк по условию?

Подсказка: Смотрите на слово, которое часто используется после SELECT.

Ответ:

- **WHERE**
- GROUP BY
- ORDER BY
- JOIN

Шаг 03:

Миссия 3

В математическом DSL, какая операция обычно используется для возведения числа в степень?

Подсказка: В Python есть оператор, который выглядит как двойной символ.

Ответ:

- ******
- +
- *
- /

Шаг 04:

Миссия 4

Сопоставьте математические операции в DSL с их результатами:

Понятия:

Ответ:

- Сложение
- Вычитание
- Умножение
- Возведение в степень

Определения:

1. Сумма двух чисел
2. Разница между двумя числами
3. Произведение двух чисел
4. Первое число в степени второго

Подсказка: Каждая операция DSL имеет стандартный аналог в математике.

- **Сложение** — Сумма двух чисел
- **Вычитание** — Разница между двумя числами
- **Умножение** — Произведение двух чисел
- **Возведение в степень** — Первое число в степени второго

Шаг 05:

Миссия 5

Сопоставьте ключевые слова мини-SQL с их функцией:

Ответ:

- SELECT
 - WHERE
 - ORDER BY
 - JOIN
1. Выбор столбцов и строк из таблицы
 2. Фильтрация строк по условию
 3. Сортировка результатов запроса
 4. Объединение двух таблиц по ключу

Подсказка: Ключевые слова часто встречаются в простых SQL запросах.

- **SELECT** — Выбор столбцов и строк из таблицы
- **WHERE** — Фильтрация строк по условию
- **ORDER BY** — Сортировка результатов запроса
- **JOIN** — Объединение двух таблиц по ключу

Шаг 06:

Миссия 6

Какой оператор мини-SQL используется для подсчета количества строк?

Подсказка: Это агрегатная функция.

Ответ:

- **COUNT**
- SUM
- AVG
- MAX

Шаг 07:

Миссия 7

Как в математическом DSL выразить деление с остатком?

Подсказка: В Python есть специальный оператор для получения остатка.

Ответ:

- %
- /
- //
- **

 **Шаг 08:**

Миссия 8

Сопоставьте SQL агрегатные функции с их назначением:

Ответ:

- SUM
- AVG
- MIN
- MAX

1. Сумма значений
2. Среднее значение
3. Минимальное значение
4. Максимальное значение

Подсказка: Агрегатные функции часто применяются в отчетах.

- **SUM** — Сумма значений
- **AVG** — Среднее значение
- **MIN** — Минимальное значение
- **MAX** — Максимальное значение

 **Шаг 09:**

Миссия 9

Расположите шаги выполнения простого SQL запроса:

1. Парсинг запроса
2. Фильтрация строк по условию
3. Выбор столбцов
4. Сортировка результатов
5. Вывод результата

Подсказка: Подумайте о логическом порядке обработки SQL запроса.

Ответ:

- Парсинг запроса
- Фильтрация строк по условию
- Выбор столбцов
- Сортировка результатов
- Вывод результата

 **Шаг 10:**

Миссия 10

Какой оператор мини-SQL используется для объединения условий с помощью логического И?

Подсказка: В SQL есть стандартные логические операторы.

Ответ:

- **AND**
- OR
- NOT
- XOR

Шаг 11:

Миссия 11

Какой оператор мини-SQL используется для объединения результатов из двух таблиц по ключу?

Подсказка: Подумайте о том, как соединить таблицы.

Ответ:

- **JOIN**
- WHERE
- GROUP BY
- SELECT

Шаг 12:

Миссия 12

Сопоставьте математические функции DSL с их применением:

Ответ:

- `abs()`
- `round()`
- `pow()`
- `divmod()`

1. Модуль числа
2. Округление числа
3. Возведение числа в степень
4. Деление с остатком

Подсказка: Каждая функция соответствует стандартной математической операции.

- **`abs()`** — Модуль числа
- **`round()`** — Округление числа
- **`pow()`** — Возведение числа в степень
- **`divmod()`** — Деление с остатком

Шаг 13:

Миссия 13

Сопоставьте SQL ключевые слова с их описанием:

Ответ:

- GROUP BY
- HAVING
- DISTINCT
- LIMIT

1. Группировка строк по значению столбца

2. Фильтрация групп по условию
3. Выбор уникальных значений
4. Ограничение количества возвращаемых строк

Подсказка: Эти ключевые слова помогают формировать отчеты и агрегаты.

- **GROUP BY** — Группировка строк по значению столбца
- **HAVING** — Фильтрация групп по условию
- **DISTINCT** — Выбор уникальных значений
- **LIMIT** — Ограничение количества возвращаемых строк

Шаг 14:

Миссия 14

Какой оператор математического DSL используется для вычисления квадратного корня числа?

Подсказка: Обычно это функция из стандартного модуля math.

Ответ:

- **math.sqrt()**
- pow()
- abs()
- round()

Шаг 15:

Миссия 15

Какой SQL оператор используется для изменения значений в таблице?

Подсказка: Этот оператор изменяет существующие строки.

Ответ:

- **UPDATE**
- INSERT
- DELETE
- SELECT

Шаг 16:

Миссия 16

Расположите шаги выполнения агрегатного запроса SQL:

1. Парсинг запроса
2. Фильтрация строк с WHERE
3. Группировка строк по GROUP BY
4. Применение агрегатных функций (SUM, AVG, COUNT)
5. Фильтрация групп с HAVING
6. Сортировка результатов с ORDER BY
7. Вывод результата

Подсказка: Сначала обрабатываются строки, потом группы, затем сортировка и вывод.

Ответ:

- Парсинг запроса
- Фильтрация строк с WHERE
- Группировка строк по GROUP BY

- Применение агрегатных функций (SUM, AVG, COUNT)
- Фильтрация групп с HAVING
- Сортировка результатов с ORDER BY
- Вывод результата

Шаг 17:

Миссия 17

Расположите шаги преобразования математического выражения DSL в результат:

1. Парсинг выражения
2. Построение AST
3. Проверка типов и валидности
4. Вычисление значения
5. Возврат результата пользователю

Подсказка: Логика похожа на работу интерпретатора: сначала синтаксис, потом вычисление.

Ответ:

- Парсинг выражения
- Построение AST
- Проверка типов и валидности
- Вычисление значения
- Возврат результата пользователю

Урок 4 - Практика генерация кода из DSL

Шаг 01:

Миссия 1

Что такое DSL (Domain-Specific Language)?

Подсказка: Это язык, заточенный под решение конкретной задачи.

Ответ:

- **Язык, специализированный для определённой области**
- Общий язык программирования
- Язык разметки веб-страниц
- Скриптовый язык операционной системы

Шаг 02:

Миссия 2

Какой подход помогает превратить DSL в исполняемый Python-код?

Подсказка: Подумайте о промежуточных представлениях и трансформациях.

Ответ:

- **Генерация AST и последующая компиляция**
- Прямое выполнение текста DSL через `exec()` без проверки
- Перевод DSL в другой DSL
- Интерпретация DSL только как комментариев

Шаг 03:

Миссия 3

Какой инструмент Python помогает работать с синтаксическим деревом кода?

Подсказка: Подумайте о модуле стандартной библиотеки для анализа кода.

Ответ:

- **Модуль ast**
- Модуль os
- Модуль math
- Модуль json

Шаг 04:

Миссия 4

Сопоставьте элементы DSL с их ролью в генерации Python-кода:

Ответ:

- Выражение
 - Оператор
 - Блок кода
 - Переменная
1. Представляет действия или функции
 2. Объединяет инструкции и задаёт последовательность
 3. Хранит данные
 4. Указывает вычисления или операции

Подсказка: Каждое понятие имеет конкретную роль в AST.

- **Выражение** — Указывает вычисления или операции
- **Оператор** — Представляет действия или функции
- **Блок кода** — Объединяет инструкции и задаёт последовательность
- **Переменная** — Хранит данные

Шаг 05:

Миссия 5

Сопоставьте этапы генерации кода из DSL с их описанием:

Ответ:

- Парсинг
 - Построение AST
 - Компиляция
 - Исполнение
1. Преобразование текста DSL в структурированные объекты
 2. Создание синтаксического дерева для анализа
 3. Преобразование AST в байткод Python
 4. Запуск сгенерированного кода

Подсказка: Логика процесса напоминает работу интерпретатора.

- **Парсинг** — Преобразование текста DSL в структурированные объекты
- **Построение AST** — Создание синтаксического дерева для анализа
- **Компиляция** — Преобразование AST в байткод Python
- **Исполнение** — Запуск сгенерированного кода

🔮 Шаг 06:

Миссия 6

Сопоставьте типы узлов AST с их назначением:

Ответ:

- Module
- Expr
- Assign
- Call

1. Корневой узел для файла Python
2. Выполнение выражения
3. Присвоение значения переменной
4. Вызов функции

Подсказка: Каждый узел отражает структуру кода.

- **Module** — Корневой узел для файла Python
- **Expr** — Выполнение выражения
- **Assign** — Присвоение значения переменной
- **Call** — Вызов функции

🔮 Шаг 07:

Миссия 7

Какой метод AST позволяет преобразовать узел обратно в исполняемый код?

Подсказка: Существует стандартная функция в модуле ast для этого.

Ответ:

- **ast.unparse()**
- ast.dump()
- compile()
- exec()

🔮 Шаг 08:

Миссия 8

Что важно проверять перед выполнением кода, сгенерированного из DSL?

Подсказка: Безопасность всегда на первом месте.

Ответ:

- **Типы данных и синтаксис**
- Имя файла исходного DSL
- Комментарий разработчика
- Путь к модулю math

🔮 Шаг 09:

Миссия 9

Расположите шаги генерации кода из DSL в правильном порядке:

1. Парсинг текста DSL
2. Создание AST

3. Проверка типов и валидность
4. Компиляция AST в байткод
5. Исполнение сгенерированного кода
6. Возврат результатов пользователю

Подсказка: Логика похожа на работу интерпретатора: сначала анализ, потом исполнение.

Ответ:

- Парсинг текста DSL
- Создание AST
- Проверка типов и валидность
- Компиляция AST в байткод
- Исполнение сгенерированного кода
- Возврат результатов пользователю

Шаг 10:

Миссия 10

Расположите этапы работы интерпретатора с DSL-кодом для вычисления выражений:

1. Лексический анализ
2. Синтаксический анализ
3. Построение AST
4. Компиляция в байткод
5. Вычисление и возврат результата

Подсказка: Лексический анализ выполняется первым, затем идут синтаксис, дерево и выполнение.

Ответ:

- Лексический анализ
- Синтаксический анализ
- Построение AST
- Компиляция в байткод
- Вычисление и возврат результата

Шаг 11:

Миссия 11

Расположите шаги оптимизации AST перед компиляцией:

1. Удаление ненужных узлов
2. Приведение выражений к стандартной форме
3. Объединение последовательностей операций
4. Проверка типов и валидность
5. Компиляция в байткод

Подсказка: Сначала чистим и упрощаем дерево, затем компилируем.

Ответ:

- Удаление ненужных узлов
- Приведение выражений к стандартной форме
- Объединение последовательностей операций
- Проверка типов и валидность

- Компиляция в байткод

Шаг 12:

Миссия 12

Что позволяет мини-DSL ускорить разработку аналитических задач?

Подсказка: Подумайте о том, что DSL упрощает для программиста.

Ответ:

- **Сокращение количества шаблонного кода и ошибок**
- Увеличение размера байткода
- Замена Python на C
- Автоматическое создание GUI

Шаг 13:

Миссия 13

Сопоставьте элементы мини-DSL с их аналогом в Python:

Ответ:

- SELECT
 - WHERE
 - SUM
 - LET
1. Цикл или фильтрация по условию
 2. Присвоение значения переменной
 3. Функция суммирования
 4. Выбор данных из коллекции

Подсказка: Подумайте, что делает каждая конструкция в SQL и как она преобразуется в Python.

- **SELECT** — Выбор данных из коллекции
- **WHERE** — Цикл или фильтрация по условию
- **SUM** — Функция суммирования
- **LET** — Присвоение значения переменной

Шаг 14:

Миссия 14

Сопоставьте математические конструкции DSL с Python:

Ответ:

- ADD
 - SUB
 - MUL
 - DIV
1. Умножение чисел
 2. Деление чисел
 3. Сложение чисел
 4. Вычитание чисел

Подсказка: Простое соответствие арифметическим операциям.

- **ADD** — Сложение чисел
- **SUB** — Вычитание чисел
- **MUL** — Умножение чисел
- **DIV** — Деление чисел

Шаг 15:

Миссия 15

Что обеспечивает корректность при генерации Python-кода из DSL?

Подсказка: Важна проверка на этапе анализа AST.

Ответ:

- **Валидация синтаксиса и типов**
- Запуск без проверки
- Генерация только комментариев
- Пропуск проверок AST

Шаг 16:

Миссия 16

Расположите этапы преобразования мини-SQL в Python-код:

1. Парсинг SQL-подобного DSL
2. Построение AST
3. Оптимизация AST
4. Компиляция в байткод
5. Исполнение и вывод результатов

Подсказка: Сначала анализируем, затем строим дерево, оптимизируем и выполняем.

Ответ:

- Парсинг SQL-подобного DSL
- Построение AST
- Оптимизация AST
- Компиляция в байткод
- Исполнение и вывод результатов

Шаг 17:

Миссия 17

Расположите шаги выполнения математического DSL для вычислений:

1. Разбор выражений
2. Построение AST
3. Оптимизация выражений
4. Компиляция в байткод
5. Выполнение и возврат результата

Подсказка: Логика аналогична генерации кода из любого DSL.

Ответ:

- Разбор выражений
- Построение AST

- Оптимизация выражений
- Компиляция в байткод
- Выполнение и возврат результата

Урок 5 - Лабиринт создание собственного DSL с AST

Шаг 01:

Миссия 1

Что является первым шагом при создании собственного DSL на Python?

Подсказка: Сначала нужно определить, что будет описывать ваш язык.

Ответ:

- **Определение синтаксиса и семантики DSL**
- Написание тестов сразу
- Компиляция DSL в C
- Добавление GUI для DSL

Шаг 02:

Миссия 2

Какой инструмент в Python помогает работать с AST?

Подсказка: Этот модуль позволяет анализировать и модифицировать синтаксические деревья.

Ответ:

- **Модуль ast**
- Модуль sys
- Модуль os
- Модуль random

Шаг 03:

Миссия 3

Сопоставьте этапы создания DSL с их назначением:

Ответ:

- Парсинг
 - Построение AST
 - Генерация кода
 - Оптимизация
1. Создание внутреннего представления программы
 2. Анализ и проверка синтаксиса
 3. Ускорение выполнения и упрощение кода
 4. Преобразование в исполняемый Python-код

Подсказка: Логика стандартного компилятора: сначала разбор, потом дерево, потом оптимизация и код.

- **Парсинг** — Анализ и проверка синтаксиса
- **Построение AST** — Создание внутреннего представления программы
- **Оптимизация** — Ускорение выполнения и упрощение кода
- **Генерация кода** — Преобразование в исполняемый Python-код

🔮 Шаг 04:

Миссия 4

Для чего используется класс `NodeVisitor` в модуле `ast`?

Подсказка: Он помогает обходить дерево AST.

Ответ:

- **Для обхода и анализа узлов AST**
- Для генерации случайных чисел
- Для создания графического интерфейса
- Для работы с файлами

🔮 Шаг 05:

Миссия 5

Сопоставьте типы узлов AST с их функцией:

Ответ:

- `ast.FunctionDef`
 - `ast.Assign`
 - `ast.BinOp`
 - `ast.If`
1. Определение функции
 2. Присвоение значения переменной
 3. Бинарная операция (сложение, умножение и т.д.)
 4. Условный оператор

Подсказка: Названия типов узлов в `ast` отражают их назначение.

- **`ast.FunctionDef`** — Определение функции
- **`ast.Assign`** — Присвоение значения переменной
- **`ast.BinOp`** — Бинарная операция (сложение, умножение и т.д.)
- **`ast.If`** — Условный оператор

🔮 Шаг 06:

Миссия 6

Что важно учитывать при генерации Python-кода из AST?

Подсказка: Код должен быть корректным и сохранять семантику DSL.

Ответ:

- **Сохранение правильной структуры и последовательности выполнения**
- Добавление случайных функций
- Игнорирование ошибок синтаксиса
- Замена всех переменных на константы

🔮 Шаг 07:

Миссия 7

Расположите шаги создания собственного DSL в логическом порядке:

1. Определение синтаксиса
2. Создание парсера

3. Построение AST
4. Обход AST и генерация Python-кода
5. Тестирование DSL

Подсказка: Сначала определяем язык, потом создаем инструменты и проверяем результат.

Ответ:

- Определение синтаксиса
- Создание парсера
- Построение AST
- Обход AST и генерация Python-кода
- Тестирование DSL

Шаг 08:

Миссия 8

Сопоставьте типы ошибок при работе с AST с их описанием:

Ответ:

- `SyntaxError`
 - `TypeError`
 - `AttributeError`
 - `ValueError`
1. Неправильный синтаксис исходного кода
 2. Некорректная операция над типами данных
 3. Отсутствие атрибута у объекта
 4. Неправильное значение для операции

Подсказка: Ошибки AST аналогичны стандартным Python-ошибкам при выполнении кода.

- **`SyntaxError`** — Неправильный синтаксис исходного кода
- **`TypeError`** — Некорректная операция над типами данных
- **`AttributeError`** — Отсутствие атрибута у объекта
- **`ValueError`** — Неправильное значение для операции

Шаг 09:

Миссия 9

Какой метод используется для компиляции AST в объект кода, который можно выполнить?

Подсказка: Этот метод создает исполняемый Python-объект из AST.

Ответ:

- **`compile()`**
- `exec()`
- `eval()`
- `open()`

Шаг 10:

Миссия 10

Сопоставьте элементы DSL с их Python-эквивалентом:

Ответ:

- DSL-конструкция IF
- DSL-конструкция LOOP
- DSL-конструкция SET
- DSL-конструкция FUNC

1. if условие:
2. for i in ... или while ...
3. variable = value
4. def имя_функции(...):

Подсказка: DSL обычно отображается напрямую в Python-код через AST.

- **DSL-конструкция IF** — if условие:
- **DSL-конструкция LOOP** — for i in ... или while ...
- **DSL-конструкция SET** — variable = value
- **DSL-конструкция FUNC** — def имя_функции(...):

 Шаг 11:

Миссия 11

Что нужно проверять перед выполнением сгенерированного кода DSL?

Подсказка: Любой код должен быть безопасным и корректным.

Ответ:

- **Синтаксис, семантика и безопасность кода**
- Только оформление комментариев
- Наличие GUI
- Совместимость с другими языками

 Шаг 12:

Миссия 12

Сопоставьте методы NodeTransformer с их функционалом:

Ответ:

- visit_Assign
 - visit_BinOp
 - visit_If
 - visit_FunctionDef
1. Изменение присвоений
 2. Модификация бинарных операций
 3. Обработка условных операторов
 4. Изменение функций

Подсказка: NodeTransformer позволяет модифицировать AST на лету.

- **visit_Assign** — Изменение присвоений
- **visit_BinOp** — Модификация бинарных операций
- **visit_If** — Обработка условных операторов
- **visit_FunctionDef** — Изменение функций

Шаг 13:

Миссия 13

Какая техника позволяет визуализировать AST для отладки DSL?

Подсказка: Можно использовать графическое представление или печать дерева.

Ответ:

- **ast.dump() или графические библиотеки**
- time.sleep()
- random.choice()
- os.listdir()

Шаг 14:

Миссия 14

Расположите этапы от идеи DSL до выполнения Python-кода:

1. Определение синтаксиса
2. Парсинг DSL
3. Создание AST
4. Модификация AST через NodeTransformer
5. Компиляция AST в объект кода
6. Выполнение Python-кода

Подсказка: Логика: идея → анализ → дерево → модификации → компиляция → выполнение.

Ответ:

- Определение синтаксиса
- Парсинг DSL
- Создание AST
- Модификация AST через NodeTransformer
- Компиляция AST в объект кода
- Выполнение Python-кода

Шаг 15:

Миссия 15

Что такое «семантическая проверка» при генерации DSL?

Подсказка: Смысл кода должен соответствовать правилам языка.

Ответ:

- **Проверка правильности логики и значений**
- Проверка наличия GUI
- Проверка совместимости с C
- Добавление комментариев

Модуль 5 - Code weaving и шаблоны

Урок 1 - Weaving вставка кода в чужие классы

Шаг 01:

Миссия 1

Что такое code weaving в Python?

Подсказка: Это способ добавить функциональность в уже существующие классы без их изменения напрямую.

Ответ:

- **Вставка кода в существующие классы через декораторы или метаклассы**
- Создание нового класса с нуля
- Удаление методов из класса
- Ручное редактирование исходников чужого класса

Шаг 02:

Миссия 2

Сопоставьте элементы code weaving с их назначением:

Ответ:

- Декоратор метода
 - Метакласс
 - Monkey patching
 - Смешивание классов (Mixin)
1. Добавление функциональности к методу во время его вызова
 2. Изменение поведения класса при создании
 3. Динамическое изменение существующих методов объекта
 4. Повторное использование кода через наследование

Подсказка: Подумайте, как каждый способ влияет на поведение класса.

- **Декоратор метода** — Добавление функциональности к методу во время его вызова
- **Метакласс** — Изменение поведения класса при создании
- **Monkey patching** — Динамическое изменение существующих методов объекта
- **Смешивание классов (Mixin)** — Повторное использование кода через наследование

Шаг 03:

Миссия 3

Какой подход позволяет безопасно добавлять метод в чужой класс без риска перезаписи существующих методов?

Подсказка: Подумайте про метапрограммирование и декораторы.

Ответ:

- **Создание нового метода через декоратор или Mixin**
- Прямое присвоение функции с именем существующего метода
- Редактирование исходника чужого класса
- Удаление старого метода и добавление нового

Шаг 04:

Миссия 4

Сопоставьте примеры code weaving с их результатом:

Ответ:

- Декоратор логирования
- Mixin с методами валидации

- Метакласс с проверкой атрибутов
- Monkey patch метода calculate

1. Каждый вызов метода выводит информацию о параметрах
2. Класс получает методы для проверки данных при наследовании
3. Создание классов автоматически проверяет правильность атрибутов
4. Существующий метод calculate теперь выполняет другую логику

Подсказка: Сосредоточьтесь на том, что изменяет поведение методов и классов.

- **Декоратор логирования** — Каждый вызов метода выводит информацию о параметрах
- **Mixin с методами валидации** — Класс получает методы для проверки данных при наследовании
- **Метакласс с проверкой атрибутов** — Создание классов автоматически проверяет правильность атрибутов
- **Monkey patch метода calculate** — Существующий метод calculate теперь выполняет другую логику

Шаг 05:

Миссия 5

Что важно учитывать при использовании monkey patching?

Подсказка: Этот метод мощный, но может быть опасен.

Ответ:

- **Можно сломать существующий функционал, если не осторожны**
- Он всегда безопасен для любых классов
- Автоматически создает тесты
- Не требует проверки синтаксиса

Шаг 06:

Миссия 6

Сопоставьте способы внедрения кода с примером использования:

Ответ:

- Декоратор
 - Метакласс
 - Mixin
 - Monkey patch
1. Добавление логирования вызовов методов класса
 2. Создание класса с обязательными атрибутами при определении
 3. Повторное использование методов в нескольких классах
 4. Изменение поведения метода в библиотечном классе

Подсказка: Каждый подход имеет уникальный способ воздействия на класс.

- **Декоратор** — Добавление логирования вызовов методов класса
- **Метакласс** — Создание класса с обязательными атрибутами при определении
- **Mixin** — Повторное использование методов в нескольких классах
- **Monkey patch** — Изменение поведения метода в библиотечном классе

🔮 Шаг 07:

Миссия 7

Почему стоит использовать Mixin вместо дублирования кода?

Подсказка: Подумайте о повторном использовании функциональности.

Ответ:

- **Mixin позволяет повторно использовать методы без копирования кода**
- Mixin полностью заменяет родительский класс
- Mixin создает новый процесс выполнения кода
- Mixin запрещает наследование

🔮 Шаг 08:

Миссия 8

Расположите шаги внедрения нового метода через code weaving:

1. Выбор подхода (декоратор, Mixin, метакласс, monkey patch)
2. Написание функции/метода
3. Внедрение функции в целевой класс
4. Тестирование нового метода

Подсказка: Сначала выбираем способ внедрения, потом создаем и проверяем метод.

Ответ:

- Выбор подхода (декоратор, Mixin, метакласс, monkey patch)
- Написание функции/метода
- Внедрение функции в целевой класс
- Тестирование нового метода

🔮 Шаг 09:

Миссия 9

Какой из подходов code weaving лучше использовать для добавления методов сразу во множество классов?

Подсказка: Подумайте о повторном использовании кода через наследование.

Ответ:

- **Mixin**
- Monkey patch
- Декоратор метода
- Редактирование исходников

🔮 Шаг 10:

Миссия 10

Сопоставьте тип code weaving с его особенностью:

Ответ:

- Декоратор
- Mixin
- Метакласс

- Monkey patch

1. Вмешательство во время вызова метода
2. Повторное использование набора методов
3. Контроль над созданием класса
4. Динамическое изменение существующих методов

Подсказка: Подумайте, что работает на уровне метода, класса или объекта.

- **Декоратор** — Вмешательство во время вызова метода
- **Mixin** — Повторное использование набора методов
- **Метакласс** — Контроль над созданием класса
- **Monkey patch** — Динамическое изменение существующих методов

Шаг 11:

Миссия 11

Что позволяет метакласс в контексте code weaving?

Подсказка: Подумайте о создании класса, а не о вызове методов.

Ответ:

- **Контролировать создание класса и его атрибутов**
- Изменять методы только одного объекта
- Повторно использовать функции в нескольких модулях
- Автоматически запускать тесты

Шаг 12:

Миссия 12

Сопоставьте инструменты code weaving с их типичными примерами:

Ответ:

- Декоратор
 - Mixin
 - Метакласс
 - Monkey patch
1. Добавление проверки входных данных при вызове метода
 2. Повторное использование методов для нескольких классов в проекте
 3. Автоматическая проверка структуры класса при создании
 4. Изменение поведения метода в библиотеке без изменения исходника

Подсказка: Подумайте, что работает на уровне вызова метода, создания класса или изменения существующих методов.

- **Декоратор** — Добавление проверки входных данных при вызове метода
- **Mixin** — Повторное использование методов для нескольких классов в проекте
- **Метакласс** — Автоматическая проверка структуры класса при создании
- **Monkey patch** — Изменение поведения метода в библиотеке без изменения исходника

Шаг 13:

Миссия 13

Какой подход позволяет добавлять код только во время выполнения, а не при определении класса?

Подсказка: Подумайте о динамическом изменении объектов.

Ответ:

- **Monkey patch**
- Метакласс
- Mixin
- Декоратор

Шаг 14:

Миссия 14

Сопоставьте типы code weaving с их преимуществами:

Ответ:

- Декоратор
 - Mixin
 - Метакласс
 - Monkey patch
1. Добавление функциональности без изменения исходного кода метода
 2. Повторное использование набора методов в нескольких классах
 3. Контроль структуры класса при создании
 4. Внесение изменений в существующий метод во время выполнения

Подсказка: Подумайте, на каком уровне каждый инструмент действует.

- **Декоратор** — Добавление функциональности без изменения исходного кода метода
- **Mixin** — Повторное использование набора методов в нескольких классах
- **Метакласс** — Контроль структуры класса при создании
- **Monkey patch** — Внесение изменений в существующий метод во время выполнения

Шаг 15:

Миссия 15

Почему code weaving считается мощным инструментом?

Подсказка: Он позволяет менять поведение классов и методов без изменения их исходников.

Ответ:

- **Позволяет внедрять функциональность динамически**
- Автоматически исправляет ошибки кода
- Создает новые процессы выполнения без классов
- Заменяет все методы в проекте без проверки

Шаг 16:

Миссия 16

Расположите шаги безопасного использования monkey patch:

1. Определение метода для изменения
2. Создание альтернативной функции
3. Временная замена метода на новую функцию
4. Тестирование нового поведения
5. Возврат к исходной функции при необходимости

Подсказка: Сначала подготавливаем, потом тестируем, потом контролируем откат.

Ответ:

- Определение метода для изменения
- Создание альтернативной функции
- Временная замена метода на новую функцию
- Тестирование нового поведения
- Возврат к исходной функции при необходимости

🔧 **Шаг 17:**

Миссия 17

Как минимизировать риск при внедрении code weaving в чужие классы?

Подсказка: Подумайте о тестировании и изоляции изменений.

Ответ:

- **Писать тесты и использовать изоляцию изменений**
- Изменять код без проверки
- Внедрять все изменения сразу во все классы
- Игнорировать документацию библиотеки

📖 **Урок 2 - Генерация методов из схем и моделей**

🔧 **Шаг 01:**

Миссия 1

Аня пишет генератор методов для своей ORM-системы. Какой подход поможет ей создать методы без дублирования кода?

Подсказка: Подумайте о том, как функции можно динамически создавать на основе шаблонов.

Ответ:

- **Использование фабрики функций, генерирующей методы из схем**
- Ручное написание каждого метода
- Копирование методов из старых классов
- Создание методов через глобальные переменные

🔧 **Шаг 02:**

Миссия 2

Борис создает шаблон методов для работы с API. Какой параметр лучше передавать в шаблон для генерации метода?

Подсказка: Подумайте о том, что у всех методов будет одинаковая структура, но различная логика.

Ответ:

- **Имя метода и URL для запроса**
- Цвет интерфейса
- Размер шрифта в IDE
- Системное время компьютера

🔧 **Шаг 03:**

Миссия 3

Катя хочет автоматически создавать методы для всех полей модели. Какой способ оптимален?
Подсказка: Думайте о метаклассах и их возможностях для динамического создания методов.

Ответ:

- **Использовать метакласс, который генерирует методы при создании класса**
- Создавать методы вручную через IDE
- Копировать методы из другой модели
- Хардкодить каждый метод в исходном коде

🔮 Шаг 04:

Миссия 4

Сопоставьте элементы генерации методов с их описанием.

Подсказка: Каждый элемент выполняет уникальную роль, обратите внимание на контекст использования.

Ответ:

- **Шаблон метода** — Общий каркас функции, который можно использовать для генерации нескольких методов
- **Фабрика методов** — Функция или объект, создающий методы на основе шаблона
- **Метакласс** — Класс, создающий другие классы и их методы динамически
- **Параметризация** — Передача данных, влияющих на поведение сгенерированного метода

🔮 Шаг 05:

Миссия 5

Сопоставьте типы методов с их применением при генерации.

Подсказка: Думайте о различиях между методами класса, экземпляра и статическими методами.

Ответ:

- **Метод экземпляра** — Используется для работы с конкретным объектом
- **Метод класса** — Используется для работы с классом в целом
- **Статический метод** — Не зависит от экземпляра или класса, может быть вызван напрямую

🔮 Шаг 06:

Миссия 6

Сопоставьте шаги создания метода через фабрику.

Подсказка: Важно соблюдать порядок: сначала шаблон, потом параметры, потом генерация.

Ответ:

- **Выбор шаблона** — Определяем, какой каркас метода нужен
- **Передача параметров** — Определяем уникальные значения для метода
- **Генерация метода** — Создаем реальную функцию с нужной логикой
- **Присвоение классу** — Метод становится частью класса и доступен для объектов

🔮 Шаг 07:

Миссия 7

Сопоставьте термины из Bytecode алхимии с их описанием.

Подсказка: Bytecode — это промежуточный язык между вашим Python-кодом и машиной.

Ответ:

- **Динамическая генерация** — Создание функций или методов во время выполнения программы
- **Декоратор** — Функция, которая изменяет поведение другой функции
- **Метапрограммирование** — Программирование, которое изменяет код программ на лету

 Шаг 08:

Миссия 8

Расположите шаги генерации метода через шаблон в правильном порядке:

1. Определение шаблона метода
2. Передача параметров для генерации
3. Генерация метода
4. Присвоение метода классу

Подсказка: Подумайте, что сначала нужно создать каркас, потом заполнить его данными, потом использовать.

Ответ:

- Определение шаблона метода
- Передача параметров для генерации
- Генерация метода
- Присвоение метода классу

 Шаг 09:

Миссия 9

Упорядочите процесс создания методов для всех полей модели:

1. Итерируем по полям модели
2. Генерируем метод через шаблон
3. Присваиваем метод классу
4. Тестируем созданные методы

Подсказка: Сначала перебираем данные, затем создаём и присваиваем методы, и только потом проверяем.

Ответ:

- Итерируем по полям модели
- Генерируем метод через шаблон
- Присваиваем метод классу
- Тестируем созданные методы

 Шаг 10:

Миссия 10

Расположите этапы использования метакласса для генерации методов:

1. Создание метакласса
2. Определение шаблонов методов внутри метакласса
3. Создание класса с использованием метакласса
4. Методы автоматически добавляются в класс

Подсказка: Метакласс сначала создаётся, потом им пользуются, и методы генерируются автоматически.

Ответ:

- Создание метакласса
- Определение шаблонов методов внутри метакласса
- Создание класса с использованием метакласса
- Методы автоматически добавляются в класс

Шаг 11:

Миссия 11

Дмитрий хочет оптимизировать генерацию методов для сложной модели. Что ему поможет ускорить разработку?

Подсказка: Используйте механизм, который позволяет создавать методы автоматически без ручного копирования.

Ответ:

- **Декораторы и фабрики методов**
- Ручное написание каждого метода
- Использование глобальных переменных для кода
- Копирование старого кода из проектов

Шаг 12:

Миссия 12

Оля хочет сгенерировать методы с разной логикой для API. Какой подход правильный?

Подсказка: Подумайте о том, как можно хранить различия отдельно от общего каркаса.

Ответ:

- **Создать общий шаблон и передавать функции-логики как параметры**
- Писать все функции вручную
- Использовать один метод для всего API
- Игнорировать различия в логике

Шаг 13:

Миссия 13

Сопоставьте тип шаблонов методов с их характерной чертой:

Подсказка: Каждый шаблон решает конкретную задачу.

Ответ:

- **CRUD-шаблон** — Методы для создания, чтения, обновления и удаления объектов
- **API-шаблон** — Методы для взаимодействия с внешними сервисами
- **Вспомогательный шаблон** — Методы, используемые внутри других методов

Шаг 14:

Миссия 14

Сопоставьте термины метапрограммирования с примерами:

Подсказка: Примеры показывают применение метапрограммирования в действии.

Ответ:

- **Декоратор** — Функция, которая добавляет логирование к другому методу
- **Фабрика методов** — Создает методы на лету для всех полей модели
- **Метакласс** — Автоматически добавляет методы при создании класса

 **Шаг 15:**

Миссия 15

Расположите этапы генерации методов для большого проекта:

1. Определение шаблонов методов
2. Создание фабрик для генерации
3. Генерация методов
4. Присвоение методам классов
5. Тестирование и отладка

Подсказка: Сначала планируем, потом создаём механизмы генерации, затем генерируем и присваиваем, и только в конце проверяем.

Ответ:

- Определение шаблонов методов
- Создание фабрик для генерации
- Генерация методов
- Присвоение методам классов
- Тестирование и отладка

 **Шаг 16:**

Миссия 16

Расположите шаги динамической генерации методов с учетом сложной логики и тестирования:

1. Определение шаблона метода
2. Определение функции логики
3. Передача функции логики в шаблон
4. Генерация метода
5. Присвоение метода классу
6. Тестирование и отладка

Подсказка: Сначала создаём шаблон и логику, затем объединяем их для генерации метода, присваиваем и проверяем работу.

Ответ:

- Определение шаблона метода
- Определение функции логики
- Передача функции логики в шаблон
- Генерация метода
- Присвоение метода классу
- Тестирование и отладка

Урок 3 - Декораторы и прокси для динамики

 **Шаг 01:**

Миссия 1

Саша хочет логировать вызовы функций в своём проекте. Какой инструмент Python поможет добавить логи к функциям без изменения их кода?

Подсказка: Подумайте о способе обернуть функцию другим объектом для изменения поведения.

Ответ:

- **Декоратор**
- Метакласс
- Контекстный менеджер
- Глобальная переменная

Шаг 02:

Миссия 2

Ирина создаёт декоратор, который проверяет права пользователя перед выполнением функции. Что должен делать такой декоратор?

Подсказка: Думайте о том, как можно выполнить проверку до вызова основной функции.

Ответ:

- **Проверять права и вызывать функцию только при успешной проверке**
- Менять код функции внутри декоратора
- Удалять аргументы функции
- Создавать новый класс вместо функции

Шаг 03:

Миссия 3

Петя хочет измерять время выполнения функций. Какой подход наиболее гибкий?

Подсказка: Подумайте, как можно добавить поведение функции без её изменения.

Ответ:

- **Использовать декоратор для оборачивания функции**
- Вставлять код таймера в каждую функцию вручную
- Менять глобальную переменную с временем
- Создавать новый модуль с функциями измерения

Шаг 04:

Миссия 4

Сопоставьте виды декораторов с их назначением.

Подсказка: Каждый декоратор решает специфическую задачу: логирование, проверка или изменение поведения.

Ответ:

- **Логирующий декоратор** — Добавляет запись о вызове функции в журнал
- **Декоратор проверки прав** — Проверяет доступ пользователя перед выполнением функции
- **Декоратор таймера** — Измеряет время выполнения функции
- **Декоратор кэширования** — Сохраняет результаты функции для повторного использования

Шаг 05:

Миссия 5

Сопоставьте типы прокси с их применением.

Подсказка: Прокси может контролировать доступ, кэшировать данные или логировать вызовы.

Ответ:

- **Защитный прокси** — Ограничивает доступ к объекту
- **Логирующий прокси** — Записывает вызовы методов
- **Кэширующий прокси** — Сохраняет результаты вычислений для ускорения

Шаг 06:

Миссия 6

Сопоставьте шаги применения декоратора к функции.

Подсказка: Декоратор сначала создаёт новую функцию-обёртку, затем возвращает её вместо оригинала.

Ответ:

- **Создание функции-обёртки** — Функция, которая добавляет новое поведение
- **Вызов декоратора** — Декоратор применяется к исходной функции
- **Возврат новой функции** — Исходная функция заменяется обёрткой

Шаг 07:

Миссия 7

Сопоставьте методы прокси с их особенностями.

Подсказка: Прокси может перехватывать вызовы, проверять условия и менять результат.

Ответ:

- **`__getattr__`** — Перехватывает обращение к атрибутам объекта
- **`__setattr__`** — Контролирует запись значений атрибутов
- **`__call__`** — Позволяет объекту вести себя как функция

Шаг 08:

Миссия 8

Расположите шаги создания логирующего декоратора в правильном порядке:

1. Создание функции-обёртки
2. Добавление логики записи в журнал
3. Вызов исходной функции
4. Возврат функции-обёртки

Подсказка: Сначала создаём обёртку, потом выполняем действия, затем возвращаем новую функцию.

Ответ:

- Создание функции-обёртки
- Добавление логики записи в журнал
- Вызов исходной функции
- Возврат функции-обёртки

Шаг 09:

Миссия 9

Упорядочьте этапы применения защитного прокси:

1. Создание класса прокси
2. Определение условий доступа
3. Перехват вызова метода объекта
4. Выполнение метода только при успешной проверке

Подсказка: Сначала создаём прокси, потом проверяем условия перед вызовом реального метода.

Ответ:

- Создание класса прокси
- Определение условий доступа
- Перехват вызова метода объекта
- Выполнение метода только при успешной проверке

Шаг 10:

Миссия 10

Расположите этапы создания кэширующего декоратора:

1. Создание словаря для хранения результатов
2. Создание функции-обёртки
3. Проверка наличия результата в словаре
4. Вызов исходной функции при отсутствии кэша
5. Сохранение результата в словарь
6. Возврат результата

Подсказка: Логика кэширования сначала проверяет данные, затем вызывает функцию при необходимости и сохраняет результат.

Ответ:

- Создание словаря для хранения результатов
- Создание функции-обёртки
- Проверка наличия результата в словаре
- Вызов исходной функции при отсутствии кэша
- Сохранение результата в словарь
- Возврат результата

Шаг 11:

Миссия 11

Маша хочет комбинировать несколько декораторов на одной функции. Что нужно учитывать?

Подсказка: Порядок применения декораторов влияет на поведение функции.

Ответ:

- Декораторы применяются сверху вниз, порядок важен
- Декораторы всегда применяются одновременно, порядок неважен
- Можно применять только один декоратор на функцию
- Декораторы нужно писать в отдельном файле

Шаг 12:

Миссия 12

Денис создает прокси для удалённого сервиса. Какой подход безопаснее?

Подсказка: Прокси должен контролировать доступ и обработку данных.

Ответ:

- **Реализовать защитный прокси с проверкой аргументов и прав**
- Вызывать сервис напрямую без проверки
- Передавать данные через глобальные переменные
- Игнорировать ошибки сервиса

Шаг 13:

Миссия 13

Сопоставьте свойства прокси с их примерами:

Подсказка: Каждый прокси имеет уникальное поведение.

Ответ:

- **Логирующий прокси** — перехватывает вызовы и записывает их в журнал
- **Защитный прокси** — ограничивает вызовы методов при отсутствии прав
- **Кэширующий прокси** — сохраняет результаты вычислений для ускорения

Шаг 14:

Миссия 14

Упорядочьте шаги комбинированного декоратора с проверкой и логированием:

1. Создание функции-обёртки
2. Проверка условий доступа
3. Логирование вызова
4. Вызов исходной функции
5. Возврат результата

Подсказка: Сначала проверяем доступ, потом логируем вызов, затем выполняем функцию.

Ответ:

- Создание функции-обёртки
- Проверка условий доступа
- Логирование вызова
- Вызов исходной функции
- Возврат результата

Шаг 15:

Миссия 15

Расположите шаги создания кэширующего прокси:

1. Создание прокси-класса
2. Создание структуры для хранения результатов
3. Перехват вызова метода объекта
4. Проверка наличия результата в кэше
5. Вызов метода и сохранение результата при отсутствии
6. Возврат результата

Подсказка: Прокси сначала перехватывает вызов, проверяет кэш, и только при отсутствии данных вызывает реальный метод и сохраняет результат.

Ответ:

- Создание прокси-класса
- Создание структуры для хранения результатов
- Перехват вызова метода объекта
- Проверка наличия результата в кэше
- Вызов метода и сохранение результата при отсутствии
- Возврат результата

 Шаг 16:

Миссия 16

Расположите этапы применения нескольких прокси и декораторов к одной функции:

1. Создание функции-обёртки декоратора
2. Применение к функции логирующего декоратора
3. Применение защитного прокси
4. Применение кэширующего декоратора
5. Вызов исходной функции через цепочку прокси и декораторов
6. Возврат результата

Подсказка: Сначала создаются обёртки, затем они комбинируются, после чего выполняется функция через все слои контроля и модификации.

Ответ:

- Создание функции-обёртки декоратора
- Применение к функции логирующего декоратора
- Применение защитного прокси
- Применение кэширующего декоратора
- Вызов исходной функции через цепочку прокси и декораторов
- Возврат результата

 Урок 4 - Практика автоматическое логирование функций

 Шаг 01:

Миссия 1

Аня хочет автоматически логировать вызовы функций в своём проекте. Что ей лучше использовать?

Подсказка: Подумайте, какой механизм позволяет обернуть функцию и добавить поведение без изменения её кода.

Ответ:

- **Декоратор**
- Контекстный менеджер
- Метакласс
- Глобальная переменная

 Шаг 02:

Миссия 2

Игорь хочет логировать аргументы и результаты функции. Какой подход наиболее удобный?

Подсказка: Подумайте о способе перехватить вызов и добавить дополнительное поведение.

Ответ:

- **Использовать декоратор для функции**
- Вставлять логирование вручную в каждую функцию
- Создать глобальный логгер без изменения функций
- Менять исходный код функций автоматически

🔮 **Шаг 03:**

Миссия 3

Марина хочет, чтобы её логирование было гибким: включать или отключать его по настройкам. Как лучше это реализовать?

Подсказка: Декоратор может принимать параметры для изменения поведения.

Ответ:

- **Создать параметризованный декоратор**
- Использовать отдельную глобальную переменную для всех функций
- Менять функции вручную при каждом изменении настроек
- Создать новый класс для каждой функции

🔮 **Шаг 04:**

Миссия 4

Сопоставьте типы логирования с их применением.

Подсказка: Подумайте, какие данные нужно фиксировать: вызовы, аргументы или результаты.

Ответ:

- **Логирование вызовов** — Фиксирует факт вызова функции
- **Логирование аргументов** — Записывает переданные функции параметры
- **Логирование результатов** — Сохраняет возвращаемые значения функции
- **Логирование ошибок** — Фиксирует исключения, возникшие при вызове

🔮 **Шаг 05:**

Миссия 5

Сопоставьте элементы декоратора с их функцией.

Подсказка: Каждая часть декоратора выполняет конкретную задачу.

Ответ:

- **Функция-обёртка** — Добавляет новое поведение к исходной функции
- **Вызов исходной функции внутри обёртки** — Сохраняет функциональность оригинала
- **Возврат обёртки из декоратора** — Подменяет исходную функцию на расширенную

🔮 **Шаг 06:**

Миссия 6

Сопоставьте типы логирования с форматом хранения данных.

Подсказка: Разные типы данных лучше хранить в разных форматах.

Ответ:

- **Короткие вызовы функций** — Текстовый лог
- **Большие данные и результаты** — JSON или база данных
- **Ошибки и исключения** — Отдельный файл ошибок

- **Отладочная информация** — Консоль или временные файлы

Шаг 07:

Миссия 7

Сопоставьте способы вызова функции с особенностями логирования.

Подсказка: Логирование можно добавлять как до, так и после вызова функции.

Ответ:

- **Вызов функции напрямую** — Без дополнительного логирования
- **Вызов через декоратор** — Аргументы и результаты фиксируются автоматически
- **Вызов через прокси** — Можно контролировать доступ и логировать вызовы

Шаг 08:

Миссия 8

Расположите шаги создания простого логирующего декоратора:

1. Создание функции-обёртки
2. Запись логов вызова функции
3. Вызов исходной функции
4. Возврат результата
5. Возврат функции-обёртки из декоратора

Подсказка: Сначала создаётся обёртка, затем выполняется логирование и вызов функции, после чего возвращается результат и сама обёртка.

Ответ:

- Создание функции-обёртки
- Запись логов вызова функции
- Вызов исходной функции
- Возврат результата
- Возврат функции-обёртки из декоратора

Шаг 09:

Миссия 9

Расположите шаги создания декоратора для логирования времени выполнения функции:

1. Создание функции-обёртки
2. Запись времени начала выполнения
3. Вызов исходной функции
4. Запись времени окончания выполнения
5. Вычисление продолжительности и запись в лог
6. Возврат результата функции
7. Возврат функции-обёртки из декоратора

Подсказка: Сначала создаём обёртку, фиксируем время до и после вызова функции, затем сохраняем результат.

Ответ:

- Создание функции-обёртки
- Запись времени начала выполнения
- Вызов исходной функции

- Запись времени окончания выполнения
- Вычисление продолжительности и запись в лог
- Возврат результата функции
- Возврат функции-обёртки из декоратора

🔮 Шаг 10:

Миссия 10

Пётр хочет, чтобы логирование автоматически работало для всех функций в модуле. Что ему поможет?

Подсказка: Подумайте о способе обернуть все функции без изменения каждой вручную.

Ответ:

- **Использовать метапрограммирование или цикл по функциям модуля**
- Менять код каждой функции вручную
- Создавать отдельный скрипт для каждой функции
- Использовать глобальные переменные для логирования

🔮 Шаг 11:

Миссия 11

Какой подход позволит логировать только определённые функции, а остальные оставить без изменений?

Подсказка: Подумайте о возможности маркировки функций для логирования.

Ответ:

- **Применять декоратор только к нужным функциям**
- Включать логирование через глобальный флаг для всех функций
- Создавать отдельные модули для логирования
- Переопределять функции через прокси для всех функций подряд

🔮 Шаг 12:

Миссия 12

Сопоставьте уровни логирования с их содержанием.

Подсказка: Подумайте, какие данные логируются для разных уровней.

Ответ:

- **DEBUG** — Подробная информация для отладки, включая аргументы и результаты
- **INFO** — Основные события программы, вызовы функций
- **WARNING** — Предупреждения о возможных проблемах
- **ERROR** — Фиксирует ошибки и исключения

🔮 Шаг 13:

Миссия 13

Сопоставьте элементы логирования с их описанием.

Подсказка: Разные элементы логирования выполняют разные функции.

Ответ:

- **Логгер** — Объект, который записывает сообщения в заданный формат
- **Хэндлер** — Определяет, куда отправлять логи (файл, консоль)
- **Форматтер** — Определяет, как выглядит запись лога

- **Уровень логирования** — Определяет важность сообщений для записи

Шаг 14:

Миссия 14

Расположите шаги применения параметризованного логирующего декоратора:

1. Определение декоратора с параметрами
2. Приём параметров и возврат функции-обёртки
3. Вызов исходной функции внутри обёртки
4. Запись логов с учётом параметров
5. Возврат результата функции
6. Применение декоратора с конкретными параметрами к функции

Подсказка: Сначала создаём декоратор с параметрами, затем применяем его к функции, внутри обёртки выполняем вызов и логирование.

Ответ:

- Определение декоратора с параметрами
- Приём параметров и возврат функции-обёртки
- Вызов исходной функции внутри обёртки
- Запись логов с учётом параметров
- Возврат результата функции
- Применение декоратора с конкретными параметрами к функции

Шаг 15:

Миссия 15

Расположите шаги логирования с обработкой исключений:

1. Создание функции-обёртки
2. Вызов исходной функции внутри try
3. Запись логов успеха выполнения функции
4. Обработка исключения и запись лога ошибки
5. Возврат результата функции или повторный выброс исключения
6. Возврат функции-обёртки из декоратора

Подсказка: Логи ошибок записываются в блоке except, успешные вызовы — в try.

Ответ:

- Создание функции-обёртки
- Вызов исходной функции внутри try
- Запись логов успеха выполнения функции
- Обработка исключения и запись лога ошибки
- Возврат результата функции или повторный выброс исключения
- Возврат функции-обёртки из декоратора

Шаг 16:

Миссия 16

Расположите шаги применения цепочки декораторов для логирования, кэширования и проверки доступа:

1. Создание функций-декораторов (логирование, кэш, проверка доступа)

2. Нанесение декораторов на функцию в нужном порядке
3. Вызов функции через цепочку декораторов
4. Выполнение логирования и проверок в правильной последовательности
5. Возврат результата функции

Подсказка: Декораторы выполняются в порядке нанесения, начиная с верхнего.

Ответ:

- Создание функций-декораторов (логирование, кэш, проверка доступа)
- Нанесение декораторов на функцию в нужном порядке
- Вызов функции через цепочку декораторов
- Выполнение логирования и проверок в правильной последовательности
- Возврат результата функции

Шаг 17:

Миссия 17

Расположите шаги полного процесса автоматического логирования функций в проекте:

1. Определение декораторов для логирования, ошибок и времени выполнения
2. Применение декораторов к выбранным функциям
3. Вызов функций через декораторы
4. Запись логов в файлы, консоль или базу
5. Анализ и визуализация логов при необходимости

Подсказка: Сначала создаются и применяются декораторы, затем функция вызывается через них, логи сохраняются и анализируются.

Ответ:

- Определение декораторов для логирования, ошибок и времени выполнения
- Применение декораторов к выбранным функциям
- Вызов функций через декораторы
- Запись логов в файлы, консоль или базу
- Анализ и визуализация логов при необходимости

Урок 5 - Лабиринт runtime-вставка методов в классы

Шаг 01:

Миссия 1

Аня работает с библиотекой, где нужно добавить метод в класс во время выполнения. Какой подход позволит ей это сделать?

Подсказка: В Python можно динамически изменять классы и объекты.

Ответ:

- Присвоить функцию классу через атрибут
- Переписать весь класс в файле
- Использовать importlib без изменений класса

Шаг 02:

Миссия 2

Владислав хочет добавить метод в экземпляр класса без изменения других объектов этого класса. Как лучше действовать?

Подсказка: Подумайте, как привязать функцию к отдельному объекту, а не ко всему классу.

Ответ:

- **Использовать `types.MethodType` для привязки метода к объекту**
- Добавить метод в сам класс, затронув все экземпляры
- Создать новый класс с методом и заменить объект

Шаг 03:

Миссия 3

Маша пишет плагин, который должен динамически расширять класс утилиты. Что важно помнить при вставке методов в runtime?

Подсказка: Методы должны корректно работать с экземплярами и сохранять контекст `self`.

Ответ:

- **Методы должны принимать `self` как первый аргумент**
- Методы можно писать без `self`, Python автоматически добавит
- Методы не должны возвращать значение, иначе runtime не вставит их

Шаг 04:

Миссия 4

 Сопоставьте термины с правильными определениями в контексте динамической вставки методов

Ответ:

- Monkey patching
 - Метод экземпляра
 - Метод класса
 - Функция как атрибут класса
1. Изменение или добавление методов в класс во время выполнения программы.
 2. Метод, привязанный к конкретному объекту через `self`.
 3. Метод, который принимает `cls` вместо `self` и работает с классом.
 4. Функция, присвоенная классу, становится доступной всем экземплярам.

Подсказка: Подумайте, кто получает метод: один объект или весь класс.

- **Monkey patching** — Изменение или добавление методов в класс во время выполнения программы.
- **Метод экземпляра** — Метод, привязанный к конкретному объекту через `self`.
- **Метод класса** — Метод, который принимает `cls` вместо `self` и работает с классом.
- **Функция как атрибут класса** — Функция, присвоенная классу, становится доступной всем экземплярам.

Шаг 05:

Миссия 5

 Сопоставьте способы добавления метода с их особенностями

Ответ:

- Добавление метода напрямую в класс
- Добавление метода через `types.MethodType` к объекту
- Использование декоратора `@classmethod`

- Использование декоратора `@staticmethod`
1. Метод доступен всем экземплярам класса.
 2. Метод доступен только конкретному объекту.
 3. Метод получает класс как первый аргумент `cls`.
 4. Метод не получает ни `self`, ни `cls`.

Подсказка: Важно, кто и как использует метод — экземпляр или класс.

- **Добавление метода напрямую в класс** — Метод доступен всем экземплярам класса.
- **Добавление метода через `types.MethodType` к объекту** — Метод доступен только конкретному объекту.
- **Использование декоратора `@classmethod`** — Метод получает класс как первый аргумент `cls`.
- **Использование декоратора `@staticmethod`** — Метод не получает ни `self`, ни `cls`.

Шаг 06:

Миссия 6

 Сопоставьте объекты и подходящие способы динамического расширения функционала

Ответ:

- Объект пользовательского интерфейса
 - Объект базы данных
 - Объект для математических вычислений
 - Утилитарный объект с глобальными методами
1. МетодType для отдельного экземпляра
 2. Динамическое добавление методов в класс с тестами
 3. Добавление функций как статических методов
 4. Monkey patching с осторожностью

Подсказка: Подумайте, что должно быть уникально для объекта, а что можно расширить глобально.

- **Объект пользовательского интерфейса** — МетодType для отдельного экземпляра
- **Объект базы данных** — Динамическое добавление методов в класс с тестами
- **Объект для математических вычислений** — Добавление функций как статических методов
- **Утилитарный объект с глобальными методами** — Monkey patching с осторожностью

Шаг 07:

Миссия 7

Расположите шаги добавления метода в класс во время выполнения в правильном порядке:

1. Присвоение функции как атрибут класса
2. Создание функции с `self` (или `cls`) в аргументах
3. Тестирование нового метода
4. Вызов метода через объект или класс

Подсказка: Сначала создаем функцию, потом прикрепляем, затем проверяем и используем.

Ответ:

- Создание функции с `self` (или `cls`) в аргументах
- Присвоение функции как атрибут класса

- Тестирование нового метода
- Вызов метода через объект или класс

Шаг 08:

Миссия 8

Расположите шаги безопасного monkey patching:

1. Создание резервной копии оригинального метода
2. Написание нового метода с логикой
3. Присвоение нового метода классу
4. Тестирование, что оригинальная функциональность сохранена

Подсказка: Сначала сохраняем старое, потом добавляем новое, потом проверяем.

Ответ:

- Создание резервной копии оригинального метода
- Написание нового метода с логикой
- Присвоение нового метода классу
- Тестирование, что оригинальная функциональность сохранена

Шаг 09:

Миссия 9

Когда стоит использовать статический метод вместо метода экземпляра?

Подсказка: Подумайте, нужен ли объект для работы метода.

Ответ:

- **Когда метод не зависит от состояния объекта или класса**
- Когда метод работает только с конкретным экземпляром
- Когда метод должен изменять все экземпляры класса

Шаг 10:

Миссия 10

 Сопоставьте ошибки при динамическом добавлении методов с последствиями

Ответ:

- Забыли self в методе
 - Добавили метод только для одного объекта, хотели для всех
 - Переписали метод класса без резервной копии
 - Добавили метод с несовместимыми аргументами
1. TypeError при вызове метода
 2. Другие экземпляры не видят метод
 3. Потеря прежней функциональности
 4. Ошибка при передаче аргументов

Подсказка: Проверьте синтаксис и кто использует метод, чтобы избежать ошибок.

- **Забыли self в методе** — TypeError при вызове метода
- **Добавили метод только для одного объекта, хотели для всех** — Другие экземпляры не видят метод
- **Переписали метод класса без резервной копии** — Потеря прежней функциональности
- **Добавили метод с несовместимыми аргументами** — Ошибка при передаче аргументов

Шаг 11:

Миссия 11

Расположите шаги добавления метода в экземпляр объекта:

1. Создание функции с `self`
2. Импорт `types` и использование `MethodType`
3. Присвоение метода объекту
4. Вызов метода через объект

Подсказка: Функция сначала существует отдельно, потом привязывается к объекту.

Ответ:

- Создание функции с `self`
- Импорт `types` и использование `MethodType`
- Присвоение метода объекту
- Вызов метода через объект

Шаг 12:

Миссия 12

Игорь хочет добавить метод, который будет доступен и текущему экземпляру, и всем новым объектам класса. Как это лучше сделать?

Подсказка: Подумайте, что влияет на все объекты, а что только на один.

Ответ:

- **Присвоить функцию как атрибут класса**
- Присвоить функцию только экземпляру через `MethodType`
- Использовать `staticmethod` для всех функций

Шаг 13:

Миссия 13

 Сопоставьте способы добавления методов с ожидаемым результатом:

Ответ:

- Присвоение функции как атрибут класса
 - Использование `MethodType` для объекта
 - `staticmethod`
 - `classmethod`
1. Метод виден всем экземплярам и будущим объектам класса
 2. Метод доступен только конкретному объекту
 3. Метод не использует `self` или `cls`
 4. Метод получает класс как первый аргумент

Подсказка: Подумайте, кто и как будет использовать новый метод.

- **Присвоение функции как атрибут класса** — Метод виден всем экземплярам и будущим объектам класса
- **Использование `MethodType` для объекта** — Метод доступен только конкретному объекту
- **`staticmethod`** — Метод не использует `self` или `cls`
- **`classmethod`** — Метод получает класс как первый аргумент

Шаг 14:

Миссия 14

Расположите шаги безопасного добавления метода к библиотечному классу:

1. Создание резервной копии оригинального метода
2. Разработка нового метода с нужной логикой
3. Присвоение нового метода классу
4. Проверка работоспособности всех функций класса

Подсказка: Сначала сохраняем старое, потом добавляем новое, затем тестируем.

Ответ:

- Создание резервной копии оригинального метода
- Разработка нового метода с нужной логикой
- Присвоение нового метода классу
- Проверка работоспособности всех функций класса

Шаг 15:

Миссия 15

Алекс хочет добавить метод в объект, но случайно переписал метод класса. Что произошло?

Подсказка: Разделяйте методы экземпляров и методы класса, чтобы избежать побочных эффектов.

Ответ:

- **Была потеря прежней функциональности для всех экземпляров класса**
- Метод добавился только к одному объекту, как планировалось
- Python автоматически сохранил старый метод

Шаг 16:

Миссия 16

 Сопоставьте ситуации с правильными методами Python для runtime-вставки:

Ответ:

- Добавление метода только для одного экземпляра
 - Метод нужен всем экземплярам и новым объектам класса
 - Метод не использует self и cls
 - Метод должен работать с самим классом
1. `types.MethodType`
 2. Присвоение функции как атрибут класса
 3. `staticmethod`
 4. `classmethod`

Подсказка: Сначала определите область действия метода — объект, класс или все экземпляры.

- **Добавление метода только для одного экземпляра** — `types.MethodType`
- **Метод нужен всем экземплярам и новым объектам класса** — Присвоение функции как атрибут класса
- **Метод не использует self и cls** — `staticmethod`
- **Метод должен работать с самим классом** — `classmethod`

Шаг 17:

Миссия 17

Расположите шаги добавления метода к библиотечному объекту с проверкой безопасности:

1. Создание нового метода
2. Сохранение оригинального метода (резервная копия)
3. Присвоение нового метода объекту или классу
4. Запуск тестов и проверка совместимости

Подсказка: Сначала готовим новый метод, сохраняем старый, затем добавляем и проверяем.

Ответ:

- Создание нового метода
- Сохранение оригинального метода (резервная копия)
- Присвоение нового метода объекту или классу
- Запуск тестов и проверка совместимости

Модуль 6 - Интеграция с внешними мирами

Урок 1 - Python AST → Cython ускорение кода

Шаг 01:

Миссия 1

 У программиста Алисы есть скрипт на Python, который работает медленно. Она думает использовать Cython. Что даст ей наибольший выигрыш?

Подсказка: Cython лучше всего ускоряет численные расчёты и циклы.

Ответ:

- **Оптимизация циклов и численных операций**
- Добавление комментариев
- Увеличение количества функций print()
- Слияние всех функций в одну

Шаг 02:

Миссия 2

 Программист Боб решил превратить часть кода Python в Cython. Что он должен сделать первым?

Подсказка: Начало всегда связано с подготовкой исходного файла.

Ответ:

- **Создать файл с расширением .pyx**
- Удалить все функции из кода
- Сжать код в архив zip
- Переименовать все переменные на латинский

Шаг 03:

Миссия 3

 Студент Ким хочет ускорить свою функцию факториала. Какой приём в Cython поможет ему больше всего?

Подсказка: Подумайте о строгих типах переменных.

Ответ:

- **Объявление переменных с типами `cdef int`**
- Добавление больше `print()`
- Удаление `return`
- Использование глобальных переменных без типов

Шаг 04:

Миссия 4

🕒 Программистка Дина сравнила Python и Cython. Где ускорение будет заметнее всего?

Подсказка: Сравните работу с циклами и строками.

Ответ:

- **В больших численных циклах**
- В печати текста
- В создании списков строк
- В подключении модулей

Шаг 05:

Миссия 5

✂ Разработчик Егор пишет AST-преобразователь, чтобы автоматически превращать Python-функции в Cython. Что в AST нужно изменить?

Подсказка: Вспомните про добавление информации о типах.

Ответ:

- **Добавить узлы для объявлений переменных с типами**
- Удалить все функции
- Заменить все идентификаторы на числа
- Убрать все `return`

Шаг 06:

Миссия 6

🔗 Сопоставьте элементы Python AST с их назначением при трансляции в Cython.

Подсказка: Вспомните, что каждое дерево описывает структуру кода.

Ответ:

- **FunctionDef** — Описание функции в дереве
- **Assign** — Присваивание значения переменной
- **For** — Описание цикла перебора
- **BinOp** — Операция сложения, вычитания и т.п.

Шаг 07:

Миссия 7

🧠 Тимур автоматизирует генерацию Cython-кода. Сопоставьте шаги процесса с их смыслом.

Подсказка: Подумайте: сначала дерево, потом трансформация, затем генерация.

Ответ:

- **Парсинг Python кода в AST** — Создание абстрактного синтаксического дерева

- **Трансформация AST** — Изменение дерева для добавления типов
- **Генерация кода Cython** — Создание исходного .pyx файла
- **Компиляция** — Преобразование в машинный код

🔮 Шаг 08:

Миссия 8

Разработчица Лана тестирует свой конвертер Python→Cython. Сопоставьте типы данных с их оптимизированными аналогами.

Подсказка: Обратите внимание на числовые типы.

Ответ:

- **int** — cdef int
- **float** — cdef double
- **list** — Cython dynamic array или typed memoryview
- **dict** — Стандартный объект Python (без ускорения)

🔮 Шаг 09:

Миссия 9

📊 Вика работает с большим массивом данных. Сопоставьте библиотеку или инструмент с его назначением при ускорении Python кода.

Подсказка: Некоторые инструменты работают через Cython.

Ответ:

- **NumPy** — Оптимизированные массивы и операции
- **Cython** — Компиляция Python-кода в C
- **PyPy** — JIT-компилятор для Python
- **AST** — Внутреннее представление структуры кода

🔮 Шаг 10:

Миссия 10

🔧 Артём ускоряет функцию обработки чисел. Сопоставьте приём оптимизации с его эффектом.

Подсказка: Помните, что не все оптимизации универсальны.

Ответ:

- **Использование cdef int** — Сильное ускорение в циклах
- **Использование глобальных переменных** — Может замедлить выполнение
- **Typed memoryviews** — Быстрый доступ к массивам
- **Удаление типов** — Нет ускорения

🔮 Шаг 11:

Миссия 11

Разработчик Никита проверяет, где оптимизация через Cython даёт результат. Сопоставьте задачу с результатом.

Подсказка: Там, где больше математики — больше ускорения.

Ответ:

- **Обработка изображений** — Заметное ускорение благодаря числовым операциям
- **Работа с JSON** — Ускорение незначительное
- **Численные интегралы** — Большой прирост скорости

- **Печать строк** — Практически без изменений

Шаг 12:

Миссия 12

📋 Расставьте шаги компиляции Cython-кода в правильном порядке.

Подсказка: Сначала создаётся .рух, потом строится расширение, а затем выполняется компиляция.

Ответ:

- Написание кода в файле .рух
- Создание setup.py
- Запуск компиляции через `python setup.py build_ext --inplace`
- Импорт полученного модуля

Шаг 13:

Миссия 13

🕒 Мила проверяет оптимизацию. Расставьте шаги эксперимента по порядку.

Подсказка: Сначала пишем тест, затем измеряем скорость, а потом оптимизируем.

Ответ:

- Написать тестовую функцию
- Измерить время выполнения на Python
- Переписать функцию с типами в Cython
- Сравнить время выполнения

Шаг 14:

Миссия 14

⚙️ Расположите этапы работы AST-транслятора Python→Cython.

Подсказка: Подумайте о порядке: анализ → трансформация → генерация.

Ответ:

- Разбор исходного кода в AST
- Добавление аннотаций типов
- Генерация Cython-кода
- Компиляция

Шаг 15:

Миссия 15

🏗️ Иван строит пайплайн ускорения кода. Расположите действия.

Подсказка: Подумайте: подготовка → преобразование → оптимизация → запуск.

Ответ:

- Подготовить Python код
- Трансформировать в AST
- Добавить Cython-типы
- Скомпилировать и протестировать

🔧 Шаг 16:

Миссия 16

✳ Команда стартапа хочет встроить ускоренный модуль. Расставьте шаги интеграции.

Подсказка: Сначала локальная проверка, потом внедрение в проект.

Ответ:

- Скомпилировать Cython-модуль
- Протестировать локально
- Добавить в основной проект
- Развернуть на сервере

🔧 Шаг 17:

Миссия 17

🦋 Финальная проверка: расположите шаги полного цикла Python→Cython→ускорение.

Подсказка: Представьте, что вы ведёте проект от начала до конца.

Ответ:

- Написание Python кода
- Трансформация в AST
- Генерация Cython-кода
- Компиляция и запуск
- Измерение ускорения

📖 Урок 2 - Транспилиция AST → Rust

🔧 Шаг 01:

Миссия 1

🧑 Алиса услышала про Rust и решила попробовать транспилицию Python AST в Rust. Что является первой целью такого подхода?

Подсказка: Подумайте, зачем программисты вообще выбирают Rust для критичных частей.

Ответ:

- **Ускорение и безопасность кода**
- Увеличение количества print()
- Добавление новых багов
- Соккрытие логики в длинных функциях

🔧 Шаг 02:

Миссия 2

🦋 Бобу нужно подготовить Python-код для трансляции в Rust. С чего он начнёт?

Подсказка: Транслятору нужно видеть структуру кода, а не строки текста.

Ответ:

- **Построение AST Python**
- Переписывание кода вручную на Rust
- Добавление случайных комментариев
- Сжатие файла в zip

🔮 Шаг 03:

Миссия 3

⚡ Ким хочет ускорить Python-функцию через Rust. Какой участок кода наиболее выгодно перенести?

Подсказка: Rust особенно хорош там, где много вычислений.

Ответ:

- **Длинные численные циклы**
- Вывод на экран
- Форматирование строк
- Создание словарей

🔮 Шаг 04:

Миссия 4

🕒 Дина сравнивает Python и Rust. В чём главное отличие при управлении памятью?

Подсказка: Вспомните про borrow checker.

Ответ:

- **Rust контролирует владение и заимствование памяти**
- Python автоматически управляет через GC
- Rust не управляет памятью вовсе
- Python требует вручную освобождать память

🔮 Шаг 05:

Миссия 5

✂ Егор пишет транpiler AST→Rust. Что ему нужно изменить в AST перед генерацией Rust-кода?

Подсказка: Обратите внимание на строгую типизацию Rust.

Ответ:

- **Добавить информацию о типах переменных**
- Удалить все функции
- Заменить идентификаторы числами
- Убрать все return

🔮 Шаг 06:

Миссия 6

🔑 Сопоставьте элементы Python AST с их назначением при генерации Rust-кода.

Подсказка: Каждый узел AST соответствует конструкции в исходном языке.

Ответ:

- **FunctionDef** — Объявление функции
- **Assign** — Присваивание переменной
- **For** — Цикл перебора
- **If** — Условное ветвление

🔮 Шаг 07:

Миссия 7

🧠 Тимур автоматизирует генерацию Rust-кода. Сопоставьте шаги процесса с их назначением.
Подсказка: Подумайте: сначала дерево, потом трансформация, потом генерация.

Ответ:

- **Парсинг Python** — Создание AST
- **Анализ типов** — Определение нужных аннотаций
- **Генерация Rust-кода** — Создание .rs файла
- **Компиляция Rust** — Построение бинаря

🔮 Шаг 08:

Миссия 8

Лана хочет оптимизировать транспилатор. Сопоставьте Python-типы с Rust-аналогами.
Подсказка: Вспомните базовые примитивы Rust.

Ответ:

- **int** — i32
- **float** — f64
- **str** — String
- **list** — Vec

🔮 Шаг 09:

Миссия 9

📖 Вика анализирует, где стоит применять Rust. Сопоставьте задачу с результатом применения.
Подсказка: Там, где математика — там ускорение.

Ответ:

- **Численные расчёты** — Сильное ускорение
- **Парсинг JSON** — Умеренное ускорение
- **Работа со строками** — Скорость почти как в Python
- **Многопоточность** — Надёжное и безопасное управление

🔮 Шаг 10:

Миссия 10

🔧 Артём тестирует разные способы интеграции Python и Rust. Сопоставьте инструмент с его назначением.

Подсказка: Эти инструменты часто встречаются в продакшне.

Ответ:

- **PyO3** — Создание Python-биндингов к Rust-коду
- **maturin** — Сборка и публикация пакетов
- **Rust FFI** — Вызов функций Rust напрямую
- **AST-транспилатор** — Автоматическая генерация Rust-кода

🔮 Шаг 11:

Миссия 11

Никита сопоставляет конструкции Python и Rust. Найдите пары.
Подсказка: Обратите внимание на синтаксис циклов и условий.

Ответ:

- **for i in range(n)** — for i in 0..n
- **if x > 0:** — if x > 0 {
- **def foo():** — fn foo() {
- **return x** — return x;

Шаг 12:

Миссия 12

📖 Расположите шаги транспиляции Python→Rust в правильном порядке.

Подсказка: Сначала строим дерево, потом меняем его, затем генерируем код.

Ответ:

- Парсинг Python в AST
- Добавление информации о типах
- Генерация Rust-кода
- Компиляция Rust

Шаг 13:

Миссия 13

💡 Мила проверяет скорость работы. Расставьте шаги эксперимента.

Подсказка: Сначала пишем тест, потом измеряем Python, затем Rust.

Ответ:

- Написать тестовую функцию
- Измерить время выполнения на Python
- Транспилировать и скомпилировать в Rust
- Сравнить результаты

Шаг 14:

Миссия 14

⚙️ Иван строит пайплайн AST→Rust. Расставьте шаги.

Подсказка: Анализ → трансформация → генерация → интеграция.

Ответ:

- Разбор Python в AST
- Трансформация узлов под Rust
- Генерация Rust-файла
- Интеграция в проект

Шаг 15:

Миссия 15

🚀 Стартап внедряет Rust-модуль. Расположите действия.

Подсказка: Сначала тесты локально, потом публикация.

Ответ:

- Скомпилировать Rust-модуль
- Протестировать локально
- Добавить в Python-проект

- Развернуть на сервере

Шаг 16:

Миссия 16

✖ Команда интегрирует AST-транспилатор. Расположите шаги.

Подсказка: Представьте реальный рабочий процесс.

Ответ:

- Подготовить Python код
- Запустить транспилатор AST→Rust
- Скомпилировать Rust-код
- Вызвать из Python

Шаг 17:

Миссия 17

🎯 Финальный квест: полный цикл Python→AST→Rust. Расставьте шаги.

Подсказка: Подумайте о последовательности от исходного кода до результата.

Ответ:

- Написать Python код
- Построить AST
- Добавить типы
- Сгенерировать Rust
- Скомпилировать и протестировать

Урок 3 - Python → JavaScript через AST (опционально)

Шаг 01:

Миссия 1

Разработчик Вика хочет запустить Python-код прямо в браузере через JavaScript. Что является первым шагом на пути к этому?

Подсказка: Нужно сначала превратить Python-код в удобное дерево.

Ответ:

- **Разобрать Python-код в AST (абстрактное синтаксическое дерево)**
- Сразу писать обертку на C++
- Просто скопировать код Python в файл .js
- Запустить Python через терминал

Шаг 02:

Миссия 2

Артём написал функцию на Python и хочет её выполнить в браузере. Какой формат кода нужно получить после трансляции?

Подсказка: Браузеры понимают только один язык из списка.

Ответ:

- **JavaScript-код**
- C#-код
- Python bytecode

- Ruby-код

🔗 Шаг 03:

Миссия 3

Валя хочет ускорить трансляцию Python → JavaScript. Какой инструмент поможет анализировать Python-код на уровне синтаксиса?

Подсказка: Этот инструмент встроен в стандартную библиотеку Python.

Ответ:

- **Модуль ast**
- Модуль os
- Модуль random
- Модуль sys

🔗 Шаг 04:

Миссия 4

Игорь заметил, что Python использует отступы, а JavaScript — фигурные скобки. Как поступить при транспиляции?

Подсказка: Думайте о способах сохранить логику, меняя синтаксис.

Ответ:

- **Преобразовать отступы Python в фигурные скобки JavaScript**
- Игнорировать отступы и надеяться на удачу
- Заменить отступы на табы
- Оставить отступы как есть

🔗 Шаг 05:

Миссия 5

Аня использует Python-функцию `print()`. Какой аналог должен появиться после трансляции в JavaScript?

Подсказка: Вспомните, как в браузере выводят текст в консоль.

Ответ:

- **`console.log()`**
- `alert()`
- `printf()`
- `document.write()`

🔗 Шаг 06:

Миссия 6

Сопоставьте конструкции Python с их аналогами в JavaScript.

Подсказка: Вспомните, что циклы и условия в обоих языках похожи, но есть различия в синтаксисе.

Ответ:

- **if x > 0:** — `if (x > 0) {`
- **for i in range(5):** — `for (let i = 0; i < 5; i++) {`
- **while x < 10:** — `while (x < 10) {`
- **print(x)** — `console.log(x)`

🔮 Шаг 07:

Миссия 7

Сопоставьте встроенные типы Python с их аналогами в JavaScript.

Подсказка: Числа и строки совпадают по сути, но у списков и словарей есть свои аналоги.

Ответ:

- **int** — Number
- **str** — String
- **list** — Array
- **dict** — Object

🔮 Шаг 08:

Миссия 8

Сопоставьте операторы Python и их аналоги в JavaScript.

Подсказка: Внимание на сравнения: в Python их пишут иначе, чем в JavaScript.

Ответ:

- **==** — ===
- **!=** — !==
- **and** — &&
- **or** — ||

🔮 Шаг 09:

Миссия 9

Сопоставьте ключевые слова Python и их аналоги в JavaScript.

Подсказка: Некоторые совпадают, другие переводятся иначе.

Ответ:

- **def** — function
- **None** — null
- **True** — true
- **False** — false

🔮 Шаг 10:

Миссия 10

Сопоставьте библиотеки/функции Python с их аналогами в JavaScript.

Подсказка: Подумайте, где чаще всего работают со случайностями и временем.

Ответ:

- **random.randint()** — Math.floor(Math.random() * n)
- **len(arr)** — arr.length
- **time.time()** — Date.now()
- **str(x)** — String(x)

🔮 Шаг 11:

Миссия 11

Сопоставьте обработку ошибок в Python и JavaScript.

Подсказка: Обратите внимание на ключевые слова, они схожи, но есть различия в деталях.

Ответ:

- **try:** — try {
- **except Exception as e:** — catch (e) {
- **finally:** — finally {
- **raise Exception()** — throw new Error()

Шаг 12:

Миссия 12

Сопоставьте способы работы со строками.

Подсказка: Индексация и конкатенация в обоих языках похожи, но методы отличаются.

Ответ:

- **s[0]** — s.charAt(0)
- **s + t** — s + t
- **s.upper()** — s.toUpperCase()
- **s.lower()** — s.toLowerCase()

Шаг 13:

Миссия 13

Расположите шаги трансляции Python → JavaScript в правильном порядке.

Подсказка: Вспомните: сначала нужно разобрать код, потом построить дерево, затем преобразовать и вывести JavaScript.

Ответ:

- Разбор Python-кода в AST
- Преобразование AST в промежуточное представление
- Генерация JavaScript-кода
- Запуск полученного кода в браузере

Шаг 14:

Миссия 14

Расположите этапы тестирования транспилятора.

Подсказка: Логично сначала написать тесты, затем прогнать транспилятор, потом сравнить результаты.

Ответ:

- Написание тестов для Python-кода
- Запуск транспилятора Python → JavaScript
- Сравнение вывода Python и JavaScript
- Исправление ошибок и доработка

Шаг 15:

Миссия 15

Упорядочьте шаги при добавлении поддержки новой конструкции Python в транспилятор.

Подсказка: Сначала надо обновить анализатор, потом генерацию кода, и только после — протестировать.

Ответ:

- Добавить разбор конструкции в AST
- Реализовать генерацию соответствующего JavaScript-кода
- Написать тесты на новую конструкцию
- Прогнать тесты и проверить корректность

💡 Шаг 16:

Миссия 16

Расставьте шаги оптимизации транспилятора.

Подсказка: Сначала нужно собрать метрики, потом найти узкие места, затем переписать и снова измерить.

Ответ:

- Сбор статистики работы
- Поиск узких мест
- Оптимизация генерации кода
- Повторное измерение производительности

💡 Шаг 17:

Миссия 17

Расположите шаги публикации транспилятора как open-source проекта.

Подсказка: Репозиторий, документация и сообщество — ключевые шаги.

Ответ:

- Создание репозитория на GitHub
- Добавление README с инструкциями
- Публикация примеров использования
- Общение с сообществом и приём pull request'ов

📖 Урок 4 - Практика mini-пайплайн Python → Cython

💡 Шаг 01:

Миссия 1

Ты написал простой Python-скрипт и хочешь ускорить его с помощью Cython. С чего лучше начать?

Подсказка: Перед оптимизацией нужно подготовить основу — превратить обычный код в тот, что Cython сможет понять.

Ответ:

- **Сохранить файл с расширением .pyx**
- Сразу подключить GPU
- Удалить все функции и оставить только print
- Заменить Python на Rust

💡 Шаг 02:

Миссия 2

Программист Дима добавил аннотации типов в код для Cython. Какие типы помогут ускорить работу численных вычислений?

Подсказка: Думай о примитивных числах, а не о сложных структурах.

Ответ:

- **int**
- **double**
- list
- dict

Шаг 03:

Миссия 3

Аня скомпилировала модуль с помощью Cython, но он не ускорился. В чём может быть причина?

Подсказка: Если не ограничить Python-объекты, Cython не сможет разогнаться.

Ответ:

- **Не добавлены статические типы**
- Файл назван .cpp
- Использован Python 3.10
- Слишком короткий код

Шаг 04:

Миссия 4

Саша хочет скомпилировать Cython-файл. Какой инструмент ему пригодится?

Подсказка: Этот инструмент превращает Python-исходники в расширения для Python на C.

Ответ:

- **setuptools с использованием cythonize**
- conda install speed
- python run.py
- npm build

Шаг 05:

Миссия 5

Программист Лена ускоряет функцию умножения матриц. Что лучше всего поможет?

Подсказка: Представь, что нужно избавиться от Python-циклов.

Ответ:

- **Использовать memomview и указать типы элементов**
- Добавить больше print для отладки
- Перенести код в Excel
- Добавить комментарии на английском

Шаг 06:

Миссия 6

 Сопоставьте ключевые шаги пайплайна Python → Cython.

Подсказка: Подумай, в каком порядке всё происходит: подготовка кода, компиляция, использование.

Ответ:

- **Создание .pyx файла** — Написать код, который будет обрабатываться Cython

- **Добавление типов** — Указать `cdef` и `memoryview` для оптимизации
- **Компиляция** — Использовать `cythonize` для сборки
- **Импорт модуля** — Запускать функции как обычные Python-модули

🔗 Шаг 07:

Миссия 7

Сопоставь тип Cython с его назначением.

Подсказка: Числа, строки и массивы ведут себя по-разному.

Ответ:

- **`cdef int`** — Для целых чисел
- **`cdef double`** — Для чисел с плавающей точкой
- **`cdef char*`** — Для строк в стиле C
- **`memoryview`** — Для работы с массивами без копирования

🔗 Шаг 08:

Миссия 8

Сопоставьте команду с её действием.

Подсказка: Эти команды часто встречаются в `setup.py`.

Ответ:

- **`cythonize`** — Компиляция `.pyx` в C
- **`Extension`** — Описание модуля для сборки
- **`build_ext`** — Сборка расширений
- **`import модуль`** — Запуск кода в Python

🔗 Шаг 09:

Миссия 9

Сопоставь возможные ошибки с причинами.

Подсказка: Ошибки часто связаны с несовместимыми типами или отсутствием файлов.

Ответ:

- **`ModuleNotFoundError`** — Файл не скомпилирован или не установлен
- **`TypeError` при вызове функции** — Несовпадение типов аргументов
- **`SyntaxError` в `.pyx`** — Неверный синтаксис Cython
- **`ImportError: DLL load failed`** — Несовместимость с компилятором или библиотекой

🔗 Шаг 10:

Миссия 10

Сопоставьте оптимизацию с её эффектом.

Подсказка: Вспомни, что ускоряет циклы, а что — работу с массивами.

Ответ:

- **Использование `cdef`** — Снижает накладные расходы Python-объектов
- **Применение `memoryview`** — Ускоряет работу с массивами
- **`nogil`-блок** — Позволяет выполнять код параллельно
- **`prange`** — Параллельные циклы

🔮 Шаг 11:

Миссия 11

Сопоставьте персонажа и его цель в проекте.

Подсказка: Каждый герой оптимизирует под свою задачу.

Ответ:

- **Аня** — Хочет ускорить циклы в численных расчетах
- **Дима** — Добавляет поддержку многопоточности
- **Саша** — Работает с большими массивами
- **Лена** — Собирает модуль через setup.py

🔮 Шаг 12:

Миссия 12

Сопоставьте Python-концепцию с её аналогом в Cython.

Подсказка: Некоторые конструкции переходят почти без изменений, другие требуют явных указаний.

Ответ:

- **def функция** — Обычная Python-функция
- **cdef функция** — Функция для Cython с типами
- **cpdef функция** — Можно вызвать из Python и Cython
- **with nogil** — Блок без глобальной блокировки интерпретатора

🔮 Шаг 13:

Миссия 13

Расставь шаги сборки Cython-модуля в правильном порядке.

Подсказка: Сначала подготовка кода, затем компиляция, потом использование.

Ответ:

- Создание .pyx файла
- Написание setup.py с cythonize
- Компиляция модуля
- Импорт и запуск модуля в Python

🔮 Шаг 14:

Миссия 14

Расставь действия Димы, который хочет оптимизировать работу с массивами.

Подсказка: Он сначала пишет код, потом указывает типы, затем тестирует.

Ответ:

- Написать Python-функцию
- Добавить memoryview
- Скомпилировать модуль
- Протестировать скорость

🔮 Шаг 15:

Миссия 15

Расположи шаги оптимизации циклов с помощью prange.

Подсказка: Сначала импортируем, потом описываем типы, затем запускаем параллельность.

Ответ:

- Импортировать prange
- Описать типы переменных
- Написать цикл с prange
- Запустить с nogil

Шаг 16:

Миссия 16

Расставь шаги по отладке Cython-кода.

Подсказка: Ошибки лучше ловить постепенно: компиляция, тесты, профилирование.

Ответ:

- Скомпилировать модуль
- Запустить тесты
- Использовать cython -a для анализа
- Оптимизировать проблемные места

Шаг 17:

Миссия 17

Расположи действия Лены при переносе проекта на Cython.

Подсказка: Она идёт от самого простого к более глубоким оптимизациям.

Ответ:

- Создать .pyx файл
- Добавить cdef и cpdef
- Оптимизировать циклы с prange
- Убрать GIL в критичных местах

Урок 5 - Лабиринт транслятор Python → Rust

Шаг 01:

Миссия 1

Алексей пишет транслятор Python → Rust и хочет ускорить простой цикл. Что в Rust будет более эффективно для итерации по массиву?

Подсказка: Подумай о том, что в Rust ценится работа без лишних аллокаций.

Ответ:

- **Использование for item in &array**
- Перебор индексов с array.get(i)
- Создание копии массива перед циклом
- Использование глобальной переменной для хранения индекса

Шаг 02:

Миссия 2

Марина заметила, что Python-список при трансляции в Rust лучше заменить на...

Подсказка: Этот тип — стандарт для динамических массивов в Rust.

Ответ:

- HashMap
- **Vec**
- Box
- Option

🔮 Шаг 03:

Миссия 3

Иван переводит Python-функцию `factorial(n)`. Как лучше в Rust объявить возвращаемый тип?
Подсказка: В Python `int` бесконечно растёт, но в Rust нужно ограничение.

Ответ:

- **`fn factorial(n: u64) -> u64`**
- `fn factorial(n: int) -> int`
- `fn factorial(n: f64) -> f64`
- `fn factorial(n) -> usize`

🔮 Шаг 04:

Миссия 4

Света переводит конструкцию Python `try/except`. В Rust аналогом будет...
Подсказка: Вспомни `enum` для работы с ошибками.

Ответ:

- **Result**
- Option
- `panic!()`
- `unwrap()`

🔮 Шаг 05:

Миссия 5

Дима хочет транслировать Python-словарь. Какой тип Rust ближе всего?
Подсказка: Это структура из `std::collections`.

Ответ:

- **HashMap**
- `Vec<(K, V)>`
- BTreeSet
- `Option<(K, V)>`

🔮 Шаг 06:

Миссия 6

📖 Задача: сопоставьте ключевые концепции Python и их эквиваленты в Rust.
Подсказка: В Rust всё жёстко типизировано.

Ответ:

- **Python list** — Vec
- **Python dict** — HashMap
- **Python None** — `Option::None`

- **Python Exception** — Result

🔮 Шаг 07:

Миссия 7

Сопоставьте Python-операции с их безопасными аналогами в Rust.

Подсказка: В Rust часто нужно явно указывать обработку ошибок.

Ответ:

- **x = arr[0]** — arr.get(0)
- **a + b** — a.checked_add(b)
- **file.read()** — fs::read()
- **raise Exception** — Err(e)

🔮 Шаг 08:

Миссия 8

Сопоставьте Python-паттерны с типичными стилями Rust.

Подсказка: Rust требует явности и контроля памяти.

Ответ:

- **Глобальные переменные** — Локальные переменные в ограниченной области
- **Динамическая типизация** — Статическая строгая типизация
- **Ленивое отношение к ошибкам** — Обязательная обработка Result
- **Списки разной длины** — Vec с capacity

🔮 Шаг 09:

Миссия 9

Соотнесите Python-модули и их стандартные Rust-аналоги.

Подсказка: std в Rust охватывает базовые операции.

Ответ:

- **os.path** — std::path
- **random** — rand crate
- **math** — std::f64
- **json** — serde_json

🔮 Шаг 10:

Миссия 10

Соотнесите Python-конструкции с особенностями владения в Rust.

Подсказка: Подумай о borrow checker.

Ответ:

- **Python pass by reference** — &T (borrow)
- **Python pass by value** — T (move)
- **Копирование объектов** — Clone trait
- **Оставление ссылки без хозяина** — Ошибка borrow checker

🔮 Шаг 11:

Миссия 11

Расположите шаги трансляции Python-функции в Rust в правильном порядке.

Подсказка: Сначала пишем сигнатуру, потом тело, а в конце — тесты.

Ответ:

- Определить сигнатуру функции с типами
- Реализовать тело функции
- Добавить обработку ошибок
- Написать тесты

Шаг 12:

Миссия 12

Расставьте шаги при замене Python list на Rust Vec.

Подсказка: В Rust нужен тип и capacity.

Ответ:

- Выбрать тип элементов
- Создать Vec::new() или Vec::with_capacity()
- Заполнить значениями
- Выполнить операции push/pop

Шаг 13:

Миссия 13

Последовательность перевода Python dict в HashMap.

Подсказка: Нужно указать ключи и значения явно.

Ответ:

- Импортировать HashMap
- Создать let mut map = HashMap::new()
- Вставить пары ключ-значение
- Использовать методы get/insert

Шаг 14:

Миссия 14

Расставьте этапы обработки ошибок Python → Rust.

Подсказка: В Rust ошибка — это Result.

Ответ:

- Определить потенциальное исключение
- Обозначить Result
- Вернуть Err(e) при ошибке
- Обработать Ok(v) в вызывающем коде

Шаг 15:

Миссия 15

Определите правильный порядок шагов при трансляции Python-модуля в crate Rust.

Подсказка: crate — это аналог пакета.

Ответ:

- Создать Cargo.toml

- Создать `src/lib.rs` или `src/main.rs`
- Написать код модуля
- Собрать и протестировать `crate`

🔧 Шаг 16:

Миссия 16

Расставьте шаги от Python-кода до готового бинарника Rust.

Подсказка: Не забудьте про компиляцию.

Ответ:

- Написать Python-код
- Транслировать в Rust
- Скомпилировать с `cargo build`
- Запустить бинарник

📖 Модуль 7 - Продвинутая оптимизация и байткод ⚡

📖 Урок 1 - Оптимизация циклов и функций на байткоде

🔧 Шаг 01:

Миссия 1

Программист Артём заметил, что цикл `for` в Python работает медленно при большом количестве итераций. Что в байткоде чаще всего тормозит такие циклы?

Подсказка: Подумайте, что в Python требует постоянной проверки типов и вызова методов даже внутри простого цикла.

Ответ:

- **Постоянные вызовы операций `LOAD_FAST` и `COMPARE_OP`**
- Недостаток кэш-памяти процессора
- Применение глобальных переменных вместо локальных
- Использование синтаксиса `list comprehension`

🔧 Шаг 02:

Миссия 2

Когда Марина смотрела байткод функции, она заметила частые `LOAD_GLOBAL`. Что это значит?

Подсказка: Вспомните, чем отличаются локальные переменные от глобальных при доступе к ним.

Ответ:

- **Переменные берутся из глобальной области видимости, что медленнее**
- Функция не имеет аргументов
- Используется встроенная оптимизация Python
- Происходит обращение к кешированным значениям

🔧 Шаг 03:

Миссия 3

Разработчик Дима переписал цикл с `range` на `itertools.count` и заметил изменения в байткоде. Почему?

Подсказка: Обратите внимание, что `range` создаёт объект последовательности, а `count` — итератор.

Ответ:

- **Потому что `count` возвращает итератор без создания списка значений**
- Потому что `count` компилируется в машинный код
- Потому что `range` быстрее благодаря встроенной оптимизации
- Потому что `itertools` автоматически использует Rust-модуль

Шаг 04:

Миссия 4

Юлия анализирует байткод функции и хочет уменьшить количество `CALL_FUNCTION`. Что чаще всего помогает?

Подсказка: Подумайте о встроенных функциях Python, которые можно заменить более простыми выражениями.

Ответ:

- **Использовать выражения вместо вызова встроенных функций**
- Добавить больше комментариев в код
- Вызывать функцию через `eval()`
- Заменить функции на классы

Шаг 05:

Миссия 5

Разработчик Илья заметил, что `list comprehension` работает быстрее обычного цикла. Почему?

Подсказка: Взгляните на количество инструкций в байткоде для обоих вариантов.

Ответ:

- **Потому что `comprehension` компилируется в компактный байткод**
- Потому что `comprehension` использует встроенный Rust-модуль
- Потому что `comprehension` создаёт глобальные переменные
- Потому что `comprehension` всегда выполняется параллельно

Шаг 06:

Миссия 6

 Задача: Сопоставьте байткод-инструкции с их назначением.

Подсказка: Вспомните, что отвечает за загрузку переменных, что — за вызов функций, а что — за арифметику.

Ответ:

- **`LOAD_FAST`** — Загрузка локальной переменной
- **`LOAD_GLOBAL`** — Доступ к глобальной переменной или функции
- **`CALL_FUNCTION`** — Вызов функции
- **`BINARY_ADD`** — Выполнение сложения

Шаг 07:

Миссия 7

 Задача: Сопоставьте оптимизационные приёмы с их эффектом.

Подсказка: Подумайте, где важна область видимости переменных, а где — сокращение вызовов функций.

Ответ:

- **Использовать локальные переменные** — Снижает количество LOAD_GLOBAL
- **Вынести вычисления за пределы цикла** — Уменьшает количество повторных инструкций
- **Заменить вызовы функций встроенными операторами** — Уменьшает количество CALL_FUNCTION
- **Использовать генераторы вместо списков** — Снижает затраты на хранение и создание объектов

🔮 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте тип цикла с его особенностью в байткоде.

Подсказка: Вспомните, какие инструкции управляют итерацией и какие — генерацией значений.

Ответ:

- **for с range** — Создаёт объект последовательности
- **for с генератором** — Работает через протокол итератора
- **while** — Использует инструкции JUMP_ABSOLUTE или POP_JUMP_IF_FALSE
- **list comprehension** — Компилируется в отдельную функцию с RETURN_VALUE

🔮 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте программистский приём с наблюдением в байткоде.

Подсказка: Следите, какие инструкции появляются в коде при оптимизации.

Ответ:

- **Использование локальных переменных** — Увеличение числа LOAD_FAST
- **Обращение к глобальной константе** — Появление LOAD_GLOBAL
- **Арифметическая операция** — Появление BINARY_* инструкций
- **Вызов функции** — CALL_FUNCTION в байткоде

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте способ оптимизации функций с результатом в байткоде.

Подсказка: Иногда один шаг уменьшает число инструкций, а иногда меняет их тип.

Ответ:

- **Замена lambda на def** — Создание именованной функции вместо объекта на лету
- **Использование констант вместо вычислений** — LOAD_CONST вместо BINARY_*
- **Избегание eval()** — Уменьшает количество вызовов функций и интерпретации строк
- **Инлайнинг маленьких функций** — Снижает число CALL_FUNCTION

🔮 Шаг 11:

Миссия 11

Расположите шаги анализа байткода функции по оптимизации циклов.

Подсказка: Сначала нужно собрать байткод, потом определить слабые места, затем оптимизировать и снова проверить.

Ответ:

- Скомпилировать функцию в байткод
- Найти часто повторяющиеся инструкции
- Переписать код с оптимизациями
- Снова проверить байткод на улучшения

Шаг 12:

Миссия 12

Расположите шаги при оптимизации функции с глобальными переменными.

Подсказка: Вспомните: перенос переменных в локальную область снижает затраты, но важно проверить корректность работы.

Ответ:

- Определить использование глобальных переменных
- Перенести их в локальные переменные
- Переписать функцию
- Проверить результат работы и байткод

Шаг 13:

Миссия 13

Расставьте шаги для оптимизации list comprehension.

Подсказка: Обратите внимание: comprehension создаёт функцию, которую можно изучить через байткод.

Ответ:

- Написать comprehension
- Скомпилировать и изучить байткод
- Сравнить с обычным циклом
- Выбрать более быстрый вариант

Шаг 14:

Миссия 14

Определите порядок шагов при оптимизации while цикла.

Подсказка: Сначала нужно понять условие, затем проверить инструкции прыжков и переписать их.

Ответ:

- Посмотреть условие цикла
- Изучить POP_JUMP_IF_FALSE
- Упростить условие
- Проверить байткод после оптимизации

Шаг 15:

Миссия 15

Расположите шаги анализа функций с рекурсией.

Подсказка: Рекурсия добавляет вызовы функций, и важно понять, можно ли заменить их на цикл.

Ответ:

- Определить рекурсивный вызов
- Посмотреть CALL_FUNCTION в байткоде
- Заменить рекурсию на цикл
- Проверить улучшение скорости

Шаг 16:

Миссия 16

Определите порядок шагов при оптимизации генераторов.

Подсказка: Генераторы создают меньше объектов, но их тоже можно анализировать через байткод.

Ответ:

- Создать генератор
- Изучить его байткод
- Сравнить с list comprehension
- Выбрать оптимальный вариант

Урок 2 - Паттерны runtime-оптимизации

Шаг 01:

Миссия 1

Андрей оптимизирует сервис логирования. Какой приём runtime-оптимизации чаще всего используют, чтобы избежать лишнего вычисления аргументов функции при отключённых логах?

Подсказка: Подумайте о ленивых вычислениях и проверках перед вызовом.

Ответ:

- **Оборачивать вызовы проверкой условия перед вычислением аргументов**
- Увеличивать количество логов для контроля
- Вызывать все функции всегда, независимо от уровня логирования
- Переписать логирование на SQL-запросы

Шаг 02:

Миссия 2

Марина ускоряет обработку данных. Какой способ уменьшить количество аллокаций в Python эффективнее?

Подсказка: Вспомните про использование объектов один раз вместо пересоздания в цикле.

Ответ:

- **Создавать объект один раз и переиспользовать его**
- Каждый раз заново создавать объект внутри цикла
- Удалять объекты с помощью del на каждой итерации
- Вызывать gc.collect() на каждом шаге

Шаг 03:

Миссия 3

Илья оптимизирует функцию суммирования. Что быстрее в Python при большом количестве чисел?

Подсказка: Вспомните про встроенные функции на C-уровне.

Ответ:

- **Использовать встроенную функцию `sum()`**
- Писать цикл `for` с `+=` вручную
- Реализовать собственную рекурсию
- Хранить числа в строках и конкатенировать

Шаг 04:

Миссия 4

Светлана анализирует код и замечает частые вызовы `len()` в цикле. Что лучше сделать?

Подсказка: Сократите количество одинаковых операций.

Ответ:

- **Сохранить результат `len()` в переменной перед циклом**
- Вызывать `len()` на каждой итерации
- Использовать `while True` и `break`
- Переписать цикл в виде рекурсии

Шаг 05:

Миссия 5

Алексей хочет оптимизировать доступ к локальным переменным. Что Python делает для ускорения?

Подсказка: Сравните доступ к локальным и глобальным переменным.

Ответ:

- **Использует массив ячеек для локальных переменных**
- Хранит локальные переменные в глобальном словаре
- Каждый раз ищет локальную переменную в `sys.modules`
- Вызывает `eval()` для каждой локальной переменной

Шаг 06:

Миссия 6

 Сопоставьте оптимизационные паттерны с их назначением.

Подсказка: Каждый паттерн решает конкретную проблему производительности.

Ответ:

- **Loop unrolling** — Снижение количества итераций цикла путём развертывания шагов
- **Memoization** — Кэширование результатов функции для повторного использования
- **Inlining** — Замена вызова функции её телом для уменьшения накладных расходов
- **Constant folding** — Вычисление константных выражений на этапе компиляции

Шаг 07:

Миссия 7

 Сопоставьте техники работы с памятью с их эффектами.

Подсказка: Оптимизация памяти часто уменьшает накладные расходы при выполнении кода.

Ответ:

- **Object pooling** — Переиспользование готовых объектов вместо пересоздания
- **String interning** — Хранение одинаковых строк в единственном экземпляре
- **Arena allocation** — Выделение памяти блоками для снижения фрагментации
- **Reference counting** — Автоматическое освобождение объектов при достижении нуля ссылок

Шаг 08:

Миссия 8

 Сопоставьте оптимизационные инструменты с их особенностями.

Подсказка: Подумайте, чем отличаются jit-компиляторы от профайлеров.

Ответ:

- **PyPy** — JIT-компилятор, ускоряющий Python-код во время выполнения
- **cProfile** — Инструмент для измерения времени выполнения функций
- **line_profiler** — Профайлер, замеряющий время по строкам кода
- **Numba** — Компилятор Python-функций в машинный код с помощью LLVM

Шаг 09:

Миссия 9

 Сопоставьте оптимизационные практики с их примером.

Подсказка: Представьте, как программист избегает лишней работы во время исполнения.

Ответ:

- **Lazy evaluation** — Отложенное вычисление выражения до необходимости
- **Short-circuit evaluation** — Остановка проверки логического выражения при определённом результате
- **Loop fusion** — Объединение двух циклов в один для снижения накладных расходов
- **Dead code elimination** — Удаление кода, который никогда не выполняется

Шаг 10:

Миссия 10

 Сопоставьте байткод-операции с их действием.

Подсказка: Вспомните базовые инструкции Python VM.

Ответ:

- **LOAD_CONST** — Загрузка константы на стек
- **LOAD_FAST** — Загрузка локальной переменной на стек
- **CALL_FUNCTION** — Вызов функции с аргументами
- **RETURN_VALUE** — Возврат результата из функции

Шаг 11:

Миссия 11

 Сопоставьте виды оптимизации с их примером.

Подсказка: Некоторые оптимизации работают во время выполнения, другие — на этапе компиляции.

Ответ:

- **Runtime specialization** — Выбор оптимального пути исполнения на основе типов в рантайме
- **Bytecode peephole optimization** — Замена последовательности инструкций на более эффективную
- **Inlining** — Замена вызова функции её телом
- **Caching** — Хранение результатов вычислений для ускорения доступа

 Шаг 12:

Миссия 12

Расположите шаги оптимизации функции с помощью мемоизации в правильном порядке.
Подсказка: Сначала добавьте кэш, потом проверку, а уже затем сохранение результата.

Ответ:

- Создание словаря для кэша
- Проверка наличия результата в кэше
- Вычисление функции при отсутствии в кэше
- Сохранение результата в кэш

 Шаг 13:

Миссия 13

Расположите шаги анализа байткода функции в правильном порядке.
Подсказка: Сначала нужно получить объект функции, затем — байткод, потом разобрать и проанализировать.

Ответ:

- Получение объекта функции
- Доступ к атрибуту `**code**`
- Чтение `co_code`
- Декодирование байткода в инструкции

 Шаг 14:

Миссия 14

Упорядочите шаги оптимизации цикла с помощью unrolling.
Подсказка: Подумайте: сначала анализируем, затем разворачиваем цикл, и только потом проверяем результат.

Ответ:

- Анализ структуры цикла
- Определение количества шагов для развёртки
- Развёртывание тела цикла вручную
- Тестирование и проверка результата

 Шаг 15:

Миссия 15

Упорядочите шаги оптимизации с помощью JIT-компиляции.
Подсказка: Сначала профилирование, потом генерация кода, затем замена и запуск.

Ответ:

- Профилирование горячих участков кода
- Генерация машинного кода
- Замена интерпретируемого кода на скомпилированный
- Запуск оптимизированного кода

 Шаг 16:

Миссия 16

Расставьте шаги оптимизации функции через inlining.

Подсказка: Обратите внимание: нужно найти вызов, заменить его телом, затем протестировать.

Ответ:

- Нахождение места вызова функции
- Подстановка тела функции на место вызова
- Удаление лишних вызовов
- Проверка корректности исполнения

 Урок 3 - Генерация self-optimizing функций

 Шаг 01:

Миссия 1

Андрей пишет функцию, которая сама выбирает оптимальный алгоритм сортировки в зависимости от размера списка. Что это за приём?

Подсказка: В таких функциях логика выбора встроена прямо в их исполнение.

Ответ:

- **Self-optimizing функция**
- Декоратор без аргументов
- Метакласс
- Функция-хелпер

 Шаг 02:

Миссия 2

Катя заметила, что её функция часто вызывает один и тот же участок кода. Она решила переписать её так, чтобы после первого вызова заменять себя на более эффективную реализацию. Как называется этот паттерн?

Подсказка: Функция модифицирует саму себя после первого вызова.

Ответ:

- **Lazy self-replacement**
- Tail call optimization
- Мемоизация
- Динамическая диспетчеризация

 Шаг 03:

Миссия 3

Вася хочет встроить в функцию выбор между NumPy и обычным Python в зависимости от того, установлен ли NumPy. Какой подход лучше использовать?

Подсказка: Подумай о том, как функция может адаптироваться к окружению.

Ответ:

- **Self-optimizing функция с проверкой окружения**
- Фиксированный импорт в начале программы
- Использование try/except внутри каждого вызова
- Заранее прописанный fallback без проверки

🔮 Шаг 04:

Миссия 4

Программист Лена создала функцию, которая при первом запуске измеряет время исполнения разных стратегий и выбирает самую быструю. Что она реализовала?

Подсказка: Это динамическая оптимизация на основе бенчмарков.

Ответ:

- **Adaptive benchmarking**
- Constant folding
- Static dispatch
- Duck typing

🔮 Шаг 05:

Миссия 5

Игорь встроил в свою функцию механику замены кода в ****globals**** после анализа первого аргумента. Что это за техника?

Подсказка: Глобальное пространство функций тоже можно менять.

Ответ:

- **Self-modifying globals**
- Memoization
- Inlining
- Метакласс с перегрузкой

🔮 Шаг 06:

Миссия 6

Сопоставьте паттерн self-optimizing функций с их описанием.

Паттерны:

Ответ:

- Lazy self-replacement
- Adaptive benchmarking
- Dynamic dispatch
- Memoization

Описание:

1. Функция измеряет разные стратегии и выбирает самую быструю.
2. Функция заменяет себя оптимизированной версией после первого вызова.
3. Функция выбирает реализацию в зависимости от типа аргументов или окружения.
4. Функция запоминает результат, чтобы не пересчитывать заново.

Подсказка: Вспомните, какая техника работает с кешем, а какая — с выбором стратегии.

- **Lazy self-replacement** — Функция заменяет себя оптимизированной версией после первого вызова.
- **Adaptive benchmarking** — Функция измеряет разные стратегии и выбирает самую быструю.
- **Dynamic dispatch** — Функция выбирает реализацию в зависимости от типа аргументов или окружения.
- **Memoization** — Функция запоминает результат, чтобы не пересчитывать заново.

🔮 Шаг 07:

Миссия 7

Свяжите примеры кода с подходящим паттерном оптимизации.

Ответ:

- `if x in cache: return cache[x]`
- `if isinstance(x, str): ... else: ...`
- `globals()['f'] = fast_version`
- `t0=time(); test_strategies(); pick_fastest()`

1. Dynamic dispatch
2. Adaptive benchmarking
3. Memoization
4. Self-modifying globals

Подсказка: Посмотрите на ключевые слова: `cache`, `isinstance`, `globals`, `time`.

- **`if x in cache: return cache[x]`** — Memoization
- **`if isinstance(x, str): ... else: ...`** — Dynamic dispatch
- **`globals()['f'] = fast_version`** — Self-modifying globals
- **`t0=time(); test_strategies(); pick_fastest()`** — Adaptive benchmarking

🔮 Шаг 08:

Миссия 8

Сопоставьте разработчиков и их любимые оптимизационные трюки.

Ответ:

- Андрей
 - Катя
 - Вася
 - Лена
1. Adaptive benchmarking
 2. Lazy self-replacement
 3. Self-modifying globals
 4. Dynamic dispatch по окружению

Подсказка: Вспомните, кто тестировал стратегии, а кто заменял себя первой же оптимизацией.

- **Андрей** — Dynamic dispatch по окружению
- **Катя** — Lazy self-replacement
- **Вася** — Self-modifying globals
- **Лена** — Adaptive benchmarking

🔮 Шаг 09:

Миссия 9

Сопоставьте технический термин с его риском.

Ответ:

- Self-modifying globals
- Lazy self-replacement
- Adaptive benchmarking
- Memoization

1. Может вызвать рост памяти из-за хранения результатов.
2. Может сделать отладку сложнее, так как код меняется на лету.
3. Может тратить время на бенчмаркинг при первом запуске.
4. Может неожиданно переписать функции в глобальном пространстве.

Подсказка: Думайте о типичных побочных эффектах.

- **Self-modifying globals** — Может неожиданно переписать функции в глобальном пространстве.
- **Lazy self-replacement** — Может сделать отладку сложнее, так как код меняется на лету.
- **Adaptive benchmarking** — Может тратить время на бенчмаркинг при первом запуске.
- **Memoization** — Может вызвать рост памяти из-за хранения результатов.

🔮 Шаг 10:

Миссия 10

Сопоставьте ситуации с подходящим паттерном оптимизации.

Ответ:

- Функция должна быстро реагировать на разные типы аргументов.
- Функция должна работать быстрее после первого вызова.
- Функция должна хранить результаты для повторного использования.
- Функция должна найти лучшую стратегию на старте.

1. Lazy self-replacement
2. Dynamic dispatch
3. Memoization
4. Adaptive benchmarking

Подсказка: Подумайте: один паттерн кэширует, другой тестирует стратегии, третий переключает логику.

- **Функция должна быстро реагировать на разные типы аргументов.** — Dynamic dispatch
- **Функция должна работать быстрее после первого вызова.** — Lazy self-replacement
- **Функция должна хранить результаты для повторного использования.** — Memoization
- **Функция должна найти лучшую стратегию на старте.** — Adaptive benchmarking

🔮 Шаг 11:

Миссия 11

Сопоставьте инструмент Python и его роль в self-optimizing функциях.

Ответ:

- `functools.lru_cache`
 - `globals()`
 - `timeit`
 - `isinstance()`
1. Используется для динамической диспетчеризации по типам.
 2. Используется для хранения и доступа к глобальным объектам.
 3. Используется для кэширования результатов функции.
 4. Используется для измерения производительности.

Подсказка: Эти инструменты напрямую связаны с паттернами.

- **`functools.lru_cache`** — Используется для кэширования результатов функции.
- **`globals()`** — Используется для хранения и доступа к глобальным объектам.
- **`timeit`** — Используется для измерения производительности.
- **`isinstance()`** — Используется для динамической диспетчеризации по типам.

 Шаг 12:

Миссия 12

Расставьте шаги создания функции, которая заменяет себя оптимизированной версией после первого вызова.

1. Задать функцию-обёртку.
2. Определить оптимизированную функцию.
3. При первом вызове заменить глобальную ссылку на оптимизированную.
4. Вернуть результат оптимизированной функции.

Подсказка: Сначала нужна оболочка, потом логика замены, и только в конце вызов новой версии.

Ответ:

- Задать функцию-обёртку.
- Определить оптимизированную функцию.
- При первом вызове заменить глобальную ссылку на оптимизированную.
- Вернуть результат оптимизированной функции.

 Шаг 13:

Миссия 13

Расположите шаги создания кэширующей self-optimizing функции в правильном порядке.

1. Проверка: есть ли результат в кэше?
2. Вычисление результата, если кэш пуст.
3. Сохранение результата в кэше.
4. Возврат результата пользователю.

Подсказка: Логика похожа на библиотеку: сначала ищем готовый ответ на полке, если нет — создаём и ставим обратно.

Ответ:

- Проверка: есть ли результат в кэше?
- Вычисление результата, если кэш пуст.
- Сохранение результата в кэше.

- Возврат результата пользователю.

Шаг 14:

Миссия 14

Упорядочите шаги при оптимизации функции через байткод-инъекцию.

1. Извлечение исходного байткода функции.
2. Изменение инструкций для ускорения.
3. Создание новой функции на основе изменённого байткода.
4. Подмена старой функции новой.

Подсказка: Думайте, как хирург: сначала достать объект, потом изменить, затем вернуть улучшенную версию.

Ответ:

- Извлечение исходного байткода функции.
- Изменение инструкций для ускорения.
- Создание новой функции на основе изменённого байткода.
- Подмена старой функции новой.

Шаг 15:

Миссия 15

Расположите шаги JIT-компиляции функции в правильной последовательности.

1. Детектировать горячий участок кода.
2. Скомпилировать этот участок в машинный код.
3. Сохранить скомпилированный блок.
4. Выполнять его вместо интерпретации.

Подсказка: Представьте: сначала находят горячее место, потом делают быстрый «ярлык» в машинный код, и уже используют его.

Ответ:

- Детектировать горячий участок кода.
- Скомпилировать этот участок в машинный код.
- Сохранить скомпилированный блок.
- Выполнять его вместо интерпретации.

Шаг 16:

Миссия 16

Расставьте этапы генерации self-optimizing функции с fallback-механизмом.

1. Определение основной стратегии выполнения.
2. Добавление запасного варианта (fallback).
3. Реализация проверки условий выполнения.
4. Выбор между основной и fallback-стратегией во время запуска.

Подсказка: Сначала проектируется главный путь, потом «план Б», а в коде нужно уметь переключаться.

Ответ:

- Определение основной стратегии выполнения.
- Добавление запасного варианта (fallback).

- Реализация проверки условий выполнения.
- Выбор между основной и fallback-стратегией во время запуска.

🔗 Шаг 17:

Миссия 17

Расположите этапы оптимизации функции с профилированием.

1. Сбор статистики выполнения функции.
2. Анализ собранных данных.
3. Выбор оптимальной стратегии исполнения.
4. Применение выбранной стратегии при последующих вызовах.

Подсказка: Сначала наблюдаем, потом анализируем, после — выбираем, и лишь затем действуем.

Ответ:

- Сбор статистики выполнения функции.
- Анализ собранных данных.
- Выбор оптимальной стратегии исполнения.
- Применение выбранной стратегии при последующих вызовах.

📖 Урок 4 - Практика автоматическое кеширование и JIT

🔗 Шаг 01:

Миссия 1

Вася пишет сервис и замечает, что функция вычисляет одно и то же снова и снова. Какой первый шаг к автоматическому кешированию?

Подсказка: Подумайте о сохранении результата функции для повторного использования.

Ответ:

- **Использовать декоратор @lru_cache**
- Переписать функцию на C++
- Добавить глобальные переменные
- Создать новую БД для хранения результатов

🔗 Шаг 02:

Миссия 2

Программист Оля добавила кеширование. Как проверить, что кеш реально работает?

Подсказка: Важно убедиться, что повторные вызовы стали быстрее.

Ответ:

- **Замерить время первого и повторного вызова**
- Посмотреть на размер стека вызовов
- Удалить кеш и сравнить результат
- Сравнить с кодом на другом языке

🔗 Шаг 03:

Миссия 3

Разработчик Петр включил кеширование, но замечает, что память быстро заполняется. Что можно сделать?

Подсказка: Иногда кеш не должен хранить всё подряд, а лишь ограниченное количество элементов.

Ответ:

- **Задать `maxsize` в `@lru_cache`**
- Отключить кеширование
- Сохранить всё в JSON-файле
- Переписать код на ассемблере

Шаг 04:

Миссия 4

Аня изучает JIT-компиляцию Python. Какую задачу она решает?

Подсказка: JIT уменьшает время выполнения за счёт преобразования в машинный код на лету.

Ответ:

- **Ускорение выполнения программ**
- Добавление новых библиотек
- Уменьшение размера кода
- Упрощение отладки

Шаг 05:

Миссия 5

Дмитрий сравнивает обычный интерпретатор Python и JIT-решение. В чём ключевое отличие?

Подсказка: Интерпретатор выполняет построчно, JIT старается компилировать блоки кода.

Ответ:

- **JIT компилирует байткод в машинный код во время работы**
- JIT уменьшает количество импортов
- Интерпретатор запускается быстрее JIT
- JIT хранит результаты только в кеше

Шаг 06:

Миссия 6

Катя использует `pumba` для JIT-компиляции. Какой синтаксис ей нужен?

Подсказка: Вспомните о специальном декораторе для функции.

Ответ:

- **`@jit`**
- `@cache`
- `@optimize`
- `@compile`

Шаг 07:

Миссия 7

Фрилансер Иван хочет проверить ускорение после JIT. Что ему лучше сравнить?

Подсказка: Замеры времени до и после оптимизации дадут объективную картину.

Ответ:

- **Время работы функции до и после применения JIT**

- Размер кеша
- Количество импортированных модулей
- Количество строк кода

🔧 Шаг 08:

Миссия 8

Программист Артём использует кеширование, но его код сильно нагружен параллельными потоками. Что важно учесть?

Подсказка: При многопоточности стоит подумать о синхронизации доступа.

Ответ:

- **Потокобезопасность кеша**
- Размер виртуального окружения
- Скорость установки библиотек
- Формат логов

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте ситуации с подходящими решениями кеширования:

Подсказка: Важно учитывать частоту вызовов и ограничения памяти.

Ответ:

- **Функция вызывается очень часто** — Использовать `@lru_cache`
- **Кеш быстро заполняет память** — Ограничить размер кеша
- **Результаты должны храниться между перезапусками** — Сохранение кеша на диск
- **Много потоков используют одну функцию** — Использовать потокобезопасные структуры

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте технологии с их особенностями оптимизации:

Подсказка: Обратите внимание: PyPy и numba используют разные подходы к JIT.

Ответ:

- **PyPy** — Встроенный JIT-интерпретатор
- **Numba** — Компиляция функций в машинный код с помощью LLVM
- **CPython** — Стандартный интерпретатор без JIT
- **Cython** — Компиляция Python в C для ускорения

🔧 Шаг 11:

Миссия 11

📖 Сопоставьте оптимизацию с её описанием:

Подсказка: Вспомните, какие приёмы помогают ускорить Python-код.

Ответ:

- **JIT-компиляция** — Компиляция байткода в машинный код во время работы
- **Кеширование** — Сохранение результатов для повторного использования
- **Профилирование** — Измерение времени работы разных частей кода
- **Оптимизация алгоритма** — Снижение асимптотической сложности

🔗 Шаг 12:

Миссия 12

📖 Сопоставьте ошибки с причинами, которые могут возникнуть при кешировании:

Подсказка: Подумайте, что может сломать работу кеша.

Ответ:

- **Не кешируется список** — Аргументы функции должны быть хэшируемыми
- **Кеш не обновляется** — Не задано время жизни (TTL)
- **Кеш занимает слишком много памяти** — Не ограничен maxsize
- **Функция не ускоряется** — Редкие вызовы не дают эффекта

🔗 Шаг 13:

Миссия 13

📖 Сопоставьте программиста с его задачей оптимизации:

Подсказка: Каждая задача связана с разным инструментом.

Ответ:

- **Вася** — Добавил `@lru_cache` для ускорения вычислений
- **Оля** — Проверяет профилирование после оптимизации
- **Иван** — Переписал часть кода с использованием `numba`
- **Катя** — Тестирует PyPy для запуска проекта

🔗 Шаг 14:

Миссия 14

Расположите шаги при использовании кеширования:

Подсказка: Сначала нужно подготовить функцию, потом включить кеш и проверить.

Ответ:

- Написать функцию
- Добавить декоратор `@lru_cache`
- Запустить функцию несколько раз
- Сравнить время выполнения

🔗 Шаг 15:

Миссия 15

Расположите шаги для проверки эффективности JIT:

Подсказка: Нельзя измерять ускорение без базовой метрики.

Ответ:

- Написать функцию для теста
- Измерить время работы без JIT
- Применить JIT-компиляцию
- Измерить время работы с JIT

🔗 Шаг 16:

Миссия 16

Расположите этапы оптимизации Python-программы:

Подсказка: Сначала ищем узкие места, затем применяем оптимизации.

Ответ:

- Профилирование программы
- Определение медленных функций
- Применение кеширования и JIT
- Повторное тестирование

Урок 5 - Лабиринт байткод-оптимизация полного модуля

Шаг 01:

Миссия 1

Разработчик Лена заметила, что её модуль работает медленно из-за частых вызовов одной функции. Какой способ оптимизации стоит использовать в первую очередь?

Подсказка: Подумайте о сохранении результатов повторных вычислений.

Ответ:

- **Использовать @lru_cache для функции**
- Переписать функцию на C++
- Добавить print для отладки
- Создать отдельный модуль для функции

Шаг 02:

Миссия 2

Программист Алекс хочет проверить, ускорилась ли функция после кеширования. Что нужно сделать?

Подсказка: Сравните время работы до и после оптимизации.

Ответ:

- **Измерить время работы функции до и после кеша**
- Посмотреть на количество строк кода
- Сравнить результат с другой библиотекой
- Удалить кеш и проверить повторно

Шаг 03:

Миссия 3

Разработчик Ирина заметила, что кеш занимает слишком много памяти. Как это исправить?

Подсказка: Можно ограничить количество элементов в кеше.

Ответ:

- **Задать maxsize в @lru_cache**
- Удалить кеширование
- Сохранять кеш на диск
- Переписать код на ассемблере

Шаг 04:

Миссия 4

Фрилансер Олег изучает JIT-компиляцию. Какую задачу она решает?

Подсказка: JIT ускоряет выполнение программы, преобразуя байткод в машинный код на лету.

Ответ:

- **Ускорение выполнения функций**
- Снижение размера исходного кода
- Автоматическая отладка ошибок
- Замена интерпретатора на другой язык

Шаг 05:

Миссия 5

Разработчик Степан хочет понять разницу между CPython и PyPy. В чём основное отличие?

Подсказка: Один интерпретатор использует JIT, другой нет.

Ответ:

- **PyPy включает JIT-компиляцию, CPython — нет**
- CPython быстрее PyPy
- PyPy требует меньше памяти для кеша
- CPython автоматически оптимизирует байткод

Шаг 06:

Миссия 6

Программистка Лилия использует numba для ускорения функций. Какой декоратор нужно применить?

Подсказка: Существуют специальные декораторы для JIT-компиляции функций.

Ответ:

- **@jit**
- @cache
- @optimize
- @fast

Шаг 07:

Миссия 7

Фрилансер Сергей хочет проверить ускорение после JIT. Что нужно сделать?

Подсказка: Сравнивать нужно время выполнения функции до и после оптимизации.

Ответ:

- **Измерить время работы функции до и после JIT**
- Посчитать количество байткодов
- Посмотреть размер модуля
- Сравнить с другим языком

Шаг 08:

Миссия 8

 Сопоставьте ситуации с оптимальными решениями оптимизации:

Подсказка: Подумайте о частоте вызовов, памяти и многопоточности.

Ответ:

- **Функция вызывается часто** — Использовать @lru_cache
- **Кеш быстро заполняет память** — Задать maxsize

- **Результаты должны храниться между перезапусками** — Сохранение кеша на диск
- **Много потоков используют функцию** — Применить потокобезопасный кеш

🔮 Шаг 09:

Миссия 9

📖 Сопоставьте инструменты оптимизации с их особенностями:

Подсказка: PyPy и numba используют разные подходы к ускорению.

Ответ:

- **PyPy** — JIT-интерпретатор для ускорения Python-кода
- **Numba** — Компиляция отдельных функций в машинный код через LLVM
- **Cython** — Компиляция Python в C для ускорения
- **CPython** — Стандартный интерпретатор без JIT

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте оптимизацию с её назначением:

Подсказка: Вспомните, какие приёмы ускоряют выполнение функций.

Ответ:

- **JIT-компиляция** — Преобразование байткода в машинный код во время работы
- **Кеширование** — Сохранение результатов для повторного использования
- **Профилирование** — Измерение времени выполнения частей кода
- **Оптимизация алгоритма** — Снижение сложности операций

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте программиста с его задачей по оптимизации:

Подсказка: Каждому герою соответствует свой инструмент оптимизации.

Ответ:

- **Вася** — Добавил `@lru_cache` для ускорения функции
- **Оля** — Проверяет профилирование кода
- **Иван** — Использует numba для компиляции функций
- **Катя** — Тестирует PyPy для всего модуля

🔮 Шаг 12:

Миссия 12

Расположите шаги при использовании кеширования в правильном порядке:

Подсказка: Сначала пишем функцию, потом включаем кеш и проверяем результат.

Ответ:

- Написание функции
- Добавление декоратора `@lru_cache`
- Запуск функции несколько раз
- Сравнение времени выполнения

🔮 Шаг 13:

Миссия 13

Расположите шаги проверки эффективности JIT:

Подсказка: Без исходного замера нельзя увидеть ускорение.

Ответ:

- Написание тестовой функции
- Замер времени без JIT
- Применение JIT-компиляции
- Замер времени с JIT

🔮 Шаг 14:

Миссия 14

Расположите этапы оптимизации модуля Python:

Подсказка: Сначала анализируем, затем применяем оптимизации и тестируем.

Ответ:

- Профилирование модуля
- Определение медленных функций
- Применение кеширования и JIT
- Повторное тестирование

🔮 Шаг 15:

Миссия 15

Расположите этапы работы программиста Артёма над оптимизацией:

Подсказка: Он сначала написал код, затем заметил узкие места, применил инструменты и протестировал.

Ответ:

- Написание исходного кода
- Замер производительности
- Применение кеширования
- Применение JIT
- Финальное тестирование

🔮 Шаг 16:

Миссия 16

Расположите шаги комплексной оптимизации модуля:

Подсказка: Сначала выявляем проблемы, затем применяем все доступные методы ускорения.

Ответ:

- Профилирование модуля
- Анализ медленных функций
- Применение кеширования
- JIT-компиляция
- Тестирование производительности

Шаг 17:

Миссия 17

Расположите действия программиста после выявления медленных функций в модуле:

Подсказка: Сначала анализируем, потом применяем оптимизации и проверяем результат.

Ответ:

- Замер производительности модуля
- Анализ медленных функций
- Применение кеширования
- Применение JIT
- Повторное тестирование

Модуль 8 - Храм Архимага

Урок 1 - Финальный проект генератор и оптимизатор кода

Шаг 01:

Миссия 1

Маг-программист Алексей создает генератор кода. Он хочет, чтобы функции выполнялись быстрее. Что ему стоит использовать в первую очередь?

Подсказка: Подумайте о сохранении результатов повторных вычислений.

Ответ:

- **Добавить кеширование через @lru_cache**
- Писать вручную оптимизированный C-код
- Увеличить количество комментариев
- Создать отдельный модуль для каждой функции

Шаг 02:

Миссия 2

Программистка Мария хочет ускорить генератор кода при множественных вызовах функций. Какой инструмент поможет?

Подсказка: Смотрите на библиотеки для JIT-компиляции и кеширования.

Ответ:

- **Использовать numba и декоратор @jit**
- Переписать весь код на Java
- Добавить многопоточность без кеша
- Создать генератор через eval

Шаг 03:

Миссия 3

Маг-программист Сергей замечает, что кеш занимает слишком много памяти. Что делать?

Подсказка: Кеш можно ограничивать, чтобы не расходовать лишние ресурсы.

Ответ:

- **Установить maxsize в @lru_cache**
- Удалить кеширование полностью

- Сохранять кеш на диск без лимита
- Переписать функции на ассемблер

🔧 Шаг 04:

Миссия 4

Разработчик Ирина хочет применить JIT для генератора. Какую цель она преследует?

Подсказка: JIT ускоряет функции, преобразуя байткод в машинный код.

Ответ:

- **Ускорить выполнение функций генератора**
- Снизить размер исходного кода
- Добавить автоматическую отладку
- Перевести код в другой язык

🔧 Шаг 05:

Миссия 5

Фрилансер Павел сравнивает CPython и PyPy. В чём ключевое отличие для генератора кода?

Подсказка: Один из интерпретаторов использует JIT, другой нет.

Ответ:

- **PyPy использует JIT, CPython — нет**
- CPython быстрее PyPy
- PyPy требует меньше памяти
- CPython автоматически оптимизирует байткод

🔧 Шаг 06:

Миссия 6

Программистка Лилия хочет ускорить отдельные функции генератора. Какой инструмент она применит?

Подсказка: Для отдельных функций существует специализированный JIT-декоратор.

Ответ:

- **Декоратор @jit из numba**
- @lru_cache
- @async
- @optimize

🔧 Шаг 07:

Миссия 7

📄 Сопоставьте ситуации генератора с оптимальным решением:

Подсказка: Смотрите на частоту вызовов, память и потокобезопасность.

Ответ:

- **Функция вызывается часто** — Применить @lru_cache
- **Кеш быстро растёт** — Установить maxsize
- **Данные должны сохраняться между сессиями** — Сохранять кеш на диск
- **Много потоков используют функции** — Использовать потокобезопасный кеш

Шаг 08:

Миссия 8

 Сопоставьте инструменты генератора кода с их особенностями:

Подсказка: Каждый инструмент ускоряет код по-разному.

Ответ:

- **PyPy** — JIT-интерпретатор для всего модуля
- **Numba** — Компиляция отдельных функций через LLVM
- **Cython** — Компиляция Python в C
- **CPython** — Стандартный интерпретатор без JIT

Шаг 09:

Миссия 9

 Сопоставьте оптимизацию с её назначением:

Подсказка: Разные приёмы ускоряют код разными способами.

Ответ:

- **JIT-компиляция** — Преобразует байткод в машинный код во время работы
- **Кеширование** — Сохраняет результаты для повторного использования
- **Профилирование** — Измеряет время выполнения частей кода
- **Оптимизация алгоритма** — Снижает сложность операций

Шаг 10:

Миссия 10

 Сопоставьте программиста с его стратегией оптимизации:

Подсказка: Каждый герой выбирает свой метод ускорения.

Ответ:

- **Артём** — Добавил `@lru_cache` для функций генератора
- **Оля** — Проверяет профилирование кода
- **Иван** — Использует `numba` для отдельных функций
- **Катя** — Тестирует PyPy для всего модуля

Шаг 11:

Миссия 11

Расположите шаги внедрения кеширования в генератор:

Подсказка: Сначала пишем функцию, потом включаем кеш и проверяем результат.

Ответ:

- Написание функции
- Добавление `@lru_cache`
- Запуск функции несколько раз
- Сравнение времени выполнения

Шаг 12:

Миссия 12

Расположите шаги проверки ускорения функций после JIT:

Подсказка: Замеряем до и после применения JIT.

Ответ:

- Создание тестовой функции
- Замер времени без JIT
- Применение JIT-компиляции
- Замер времени с JIT

Шаг 13:

Миссия 13

Расположите шаги комплексной оптимизации генератора:

Подсказка: Сначала выявляем узкие места, затем применяем методы ускорения и тестируем.

Ответ:

- Профилирование модуля
- Анализ медленных функций
- Применение кеширования
- JIT-компиляция
- Повторное тестирование

Шаг 14:

Миссия 14

Расположите этапы работы программиста над финальным проектом:

Подсказка: Сначала пишем код, затем анализируем, применяем оптимизации и проверяем результат.

Ответ:

- Написание исходного кода
- Замер производительности
- Применение кеширования
- Применение JIT
- Финальное тестирование

Шаг 15:

Миссия 15

Расположите этапы внедрения генератора в проект:

Подсказка: Сначала пишем и тестируем код, затем оптимизируем.

Ответ:

- Написание генератора
- Тестирование базового функционала
- Оптимизация функций через кеш
- Оптимизация функций через JIT
- Финальное тестирование

Шаг 16:

Миссия 16

Расположите шаги комплексной оптимизации финального модуля генератора:

Подсказка: Анализируем, применяем все методы ускорения и тестируем результат.

Ответ:

- Профилирование всего модуля
- Выявление медленных функций
- Применение кеширования
- Применение JIT
- Повторное тестирование

 Шаг 17:

Миссия 17

Расположите действия программиста после выявления медленных функций финального генератора:

Подсказка: Сначала анализируем, затем применяем оптимизации и проверяем результат.

Ответ:

- Замер производительности модуля
- Анализ медленных функций
- Применение кеширования
- Применение JIT
- Финальное тестирование

 Урок 2 - Интеграция DSL и динамических модулей

 Шаг 01:

Миссия 1

Программистка Алина создает DSL для конфигурации генератора кода. Что поможет ускорить работу модуля на первом этапе?

Подсказка: Подумайте о кэшировании часто повторяющихся команд DSL.

Ответ:

- **Встроить кеширование в интерпретатор DSL**
- Переписать DSL на C
- Добавить длинные комментарии в DSL
- Сделать DSL полностью статическим

 Шаг 02:

Миссия 2

Инженер Борис хочет динамически подключать модули Python к DSL. Что лучше использовать для безопасности и контроля версий?

Подсказка: Подумайте о механизмах загрузки модулей и ограничении доступа.

Ответ:

- **Использовать importlib с ограничением namespace**
- Просто делать exec на код модулей
- Переписать модуль в Cython
- Добавить глобальные переменные для всех модулей

 Шаг 03:

Миссия 3

Маг-программист Виктор наблюдает, что динамические модули замедляют DSL. Какой метод ускорения стоит применить первым?

Подсказка: Подумайте о компиляции повторяющихся команд в байткод.

Ответ:

- **Применить JIT-компиляцию команд DSL**
- Увеличить количество логов
- Сохранить все модули в одном файле
- Разделить DSL на более мелкие файлы

Шаг 04:

Миссия 4

Программистка Дарья создает генератор модулей. Она хочет, чтобы DSL корректно интерпретировал новые конструкции. Что нужно сделать?

Подсказка: Думайте о расширяемости синтаксиса DSL и обработке новых узлов.

Ответ:

- **Добавить парсер для новых конструкций DSL**
- Игнорировать новые конструкции
- Переписать DSL на другой язык
- Использовать глобальные eval для всех команд

Шаг 05:

Миссия 5

Инженер Егор хочет интегрировать несколько динамических модулей без конфликтов. Что поможет?

Подсказка: Подумайте о namespace и управлении зависимостями.

Ответ:

- **Использовать отдельные namespace для каждого модуля**
- Загружать все модули в globals()
- Объединить все функции в один файл
- Игнорировать версии модулей

Шаг 06:

Миссия 6

 Сопоставьте задачу интеграции с оптимальным решением:

Подсказка: Смотрите на частоту изменений, сложность и безопасность.

Ответ:

- **Частые изменения DSL** — Использовать динамическую загрузку модулей
- **Большие модули** — Разделить на подмодули
- **Конфликт имён** — Использовать отдельные namespace
- **Безопасность исполнения** — Ограничить доступ через importlib

Шаг 07:

Миссия 7

 Сопоставьте метод ускорения с его эффектом на DSL:

Подсказка: Разные подходы оптимизируют по-разному.

Ответ:

- **JIT-компиляция команд** — Ускоряет выполнение повторяющихся команд
- **Кеширование результатов** — Сохраняет результаты для повторного использования
- **Асинхронная обработка** — Позволяет параллельно выполнять независимые команды
- **Статическая проверка** — Снижает вероятность ошибок до исполнения

Шаг 08:

Миссия 8

 Сопоставьте динамический модуль с его преимуществом:

Подсказка: Каждый улучшает DSL по-своему.

Ответ:

- **Модуль генерации кода** — Автоматизирует создание повторяющихся функций
- **Модуль проверки типов** — Позволяет выявлять ошибки на лету
- **Модуль визуализации** — Отображает структуру DSL для отладки
- **Модуль конфигурации** — Упрощает настройку параметров

Шаг 09:

Миссия 9

 Сопоставьте программиста с его подходом к интеграции DSL:

Подсказка: Каждый выбирает метод, который подходит для его модуля.

Ответ:

- **Алексей** — Добавляет JIT-компиляцию для часто вызываемых команд
- **Марина** — Использует кеширование для повторяющихся результатов
- **Дмитрий** — Разделяет модули по namespace
- **Ольга** — Ограничивает динамические импорты для безопасности

Шаг 10:

Миссия 10

Расположите шаги интеграции нового динамического модуля в DSL:

Подсказка: Сначала пишем модуль, проверяем работу, затем подключаем и тестируем снова.

Ответ:

- Создание нового модуля
- Проверка работы модуля отдельно
- Подключение к DSL
- Тестирование полной системы

Шаг 11:

Миссия 11

Расположите этапы оптимизации DSL с динамическими модулями:

Подсказка: Сначала анализируем, затем применяем ускорения и проверяем результат.

Ответ:

- Профилирование DSL
- Идентификация медленных команд
- Добавление кеширования

- Применение JIT
- Финальное тестирование

Шаг 12:

Миссия 12

Расположите шаги безопасного подключения динамических модулей:

Подсказка: Сначала проверяем модуль, потом изолируем namespace и подключаем.

Ответ:

- Проверка модуля на ошибки
- Изоляция в отдельный namespace
- Подключение к основной системе
- Тестирование интеграции

Шаг 13:

Миссия 13

Расположите шаги оптимизации DSL с новыми динамическими модулями:

Подсказка: Сначала профилируем, затем применяем ускорение и проверяем корректность.

Ответ:

- Профилирование выполнения DSL
- Идентификация медленных команд
- Добавление кеширования
- Применение JIT
- Финальное тестирование

Шаг 14:

Миссия 14

Расположите этапы интеграции DSL с внешним API:

Подсказка: Сперва анализируем API, потом создаем адаптер и проверяем работу.

Ответ:

- Анализ API
- Создание адаптера
- Подключение к DSL
- Тестирование интеграции

Шаг 15:

Миссия 15

Расположите этапы тестирования динамических модулей DSL:

Подсказка: Сначала проверяем модуль автономно, затем интегрируем и тестируем систему.

Ответ:

- Автономное тестирование модуля
- Интеграция в DSL
- Тестирование всей системы
- Ревью кода

Шаг 16:

Миссия 16

Программистка Светлана хочет уменьшить время запуска DSL с динамическими модулями. Что окажется наиболее эффективным?

Подсказка: Подумайте о сокращении повторяющихся вычислений при запуске.

Ответ:

- **Предварительное кеширование часто используемых команд**
- Добавление логов на каждый шаг
- Полная перекомпиляция каждого модуля при запуске
- Игнорирование оптимизации

Шаг 17:

Миссия 17

Инженер Николай хочет отследить эффективность оптимизации DSL после добавления JIT и кеширования. Как лучше оценить результаты?

Подсказка: Смотрите на время выполнения и использование ресурсов.

Ответ:

- **Сравнить профилирование до и после оптимизации**
- Оценивать по количеству строк кода
- Измерять только размер бинарного файла
- Сравнить количество импортов

Урок 3 - Hot reload + runtime модификации

Шаг 01:

Миссия 1

Вы работаете над проектом и хотите обновлять код без перезапуска программы. Что нужно использовать?

Подсказка: Иногда полезно видеть изменения в реальном времени.

Ответ:

- **Hot reload**
- Статическая компиляция
- Ручной перезапуск программы
- Добавление print во все функции

Шаг 02:

Миссия 2

Программист вносит правки в функцию и хочет сразу проверить результат. Что нужно сделать?

Подсказка: Обновления должны отражаться без полной перезагрузки приложения.

Ответ:

- **Использовать механизм hot reload**
- Закомментировать старый код и запустить заново
- Перезапустить сервер вручную
- Пересобрать проект через Makefile

Шаг 03:

Миссия 3

Вы хотите изменить поведение метода на лету без изменения исходного файла. Как это сделать?

Подсказка: Python позволяет модифицировать функции во время выполнения.

Ответ:

- **Присвоить новой функции старое имя метода**
- Изменить байткод вручную
- Создать новый модуль и перезапустить
- Использовать глобальные переменные для хранения старой функции

Шаг 04:

Миссия 4

В проекте несколько разработчиков. Как безопасно вносить runtime-изменения в общий модуль?

Подсказка: Подумайте о том, чтобы не ломать работу других функций.

Ответ:

- **Использовать monkey patching в локальной копии**
- Изменять модуль напрямую на проде
- Перезапускать всех пользователей
- Удалять старые функции без резервной копии

Шаг 05:

Миссия 5

Какой инструмент поможет отслеживать изменения кода в реальном времени при разработке веб-приложений?

Подсказка: Часто используется в сочетании с фреймворками типа Flask или Django.

Ответ:

- **Hot reload сервер**
- Традиционный отладчик
- Система сборки Make
- Проверка статички через линтер

Шаг 06:

Миссия 6

Сопоставьте подход с его эффектом при runtime модификациях.

Формулировка: Соотнесите действия и их результаты.

Действия:

Ответ:

- Monkey patching функции
- Использование decorators
- Перезапись метода класса
- Reload модуля

Эффекты:

1. Изменение поведения функции без изменения исходного файла
2. Добавление функциональности к существующей функции
3. Моментальное обновление всех экземпляров класса
4. Перезагрузка модуля с применением последних изменений

🔮 Подсказка:

Подумайте, что делает каждый инструмент на уровне кода и объектов.

- **Monkey patching функции** — Изменение поведения функции без изменения исходного файла
- **Использование decorators** — Добавление функциональности к существующей функции
- **Перезапись метода класса** — Моментальное обновление всех экземпляров класса
- **Reload модуля** — Перезагрузка модуля с применением последних изменений

🔮 Шаг 07:

Миссия 7

Сопоставьте тип изменения кода с возможной проблемой.

Типы изменений:

Ответ:

- Динамическая замена функции
- Hot reload модуля
- Изменение глобальной переменной
- Monkey patching объекта

Проблемы:

1. Конфликты с другими частями программы
2. Непредсказуемое поведение при повторном вызове
3. Обновления могут быть потеряны при перезапуске
4. Нарушение инкапсуляции

🔮 Подсказка:

Подумайте о том, как каждое изменение влияет на глобальное состояние и объекты.

- **Динамическая замена функции** — Непредсказуемое поведение при повторном вызове
- **Hot reload модуля** — Обновления могут быть потеряны при перезапуске
- **Изменение глобальной переменной** — Конфликты с другими частями программы
- **Monkey patching объекта** — Нарушение инкапсуляции

🔮 Шаг 08:

Миссия 8

Сопоставьте инструмент Python с его применением для hot reload.

Инструменты:

Ответ:

- `importlib.reload`
- `setattr`
- `decorators`
- `sys.modules`

Применение:

1. Перезагрузка модуля с новыми функциями
2. Динамическая замена метода или атрибута
3. Добавление поведения к существующим функциям
4. Доступ к уже загруженным модулям

🔗 Подсказка:

Подумайте, какие функции стандартной библиотеки позволяют работать с объектами и модулями на лету.

- **importlib.reload** — Перезагрузка модуля с новыми функциями
- **setattr** — Динамическая замена метода или атрибута
- **decorators** — Добавление поведения к существующим функциям
- **sys.modules** — Доступ к уже загруженным модулям

🔗 Шаг 09:

Миссия 9

Сопоставьте подход к runtime модификации с уровнем риска.

Подходы:

Ответ:

- Перезапись методов стандартной библиотеки
- Hot reload собственного модуля
- Динамическая замена глобальной переменной
- Использование monkey patch на сторонней библиотеке

Риски:

1. Высокий риск сломать систему
2. **Средний риск** — возможны конфликты
3. **Низкий риск** — изменения локализованы
4. **Средний риск** — поведение может отличаться от ожиданий

🔗 Подсказка:

Подумайте о том, насколько изменения затрагивают внешний код и глобальное состояние.

- **Перезапись методов стандартной библиотеки** — Высокий риск сломать систему
- **Hot reload собственного модуля** — Низкий риск — изменения локализованы
- **Динамическая замена глобальной переменной** — Средний риск — возможны конфликты
- **Использование monkey patch на сторонней библиотеке** — Средний риск — поведение может отличаться от ожиданий

🔗 Шаг 10:

Миссия 10

Сопоставьте методы тестирования runtime модификаций с их назначением.

Методы:

Ответ:

- Unit тесты
- Integration тесты
- Логирование изменений

- Проверка состояния объектов после изменений

Назначение:

1. Проверка отдельных функций на корректность
2. Контроль взаимодействия между модулями после изменений
3. Фиксация динамических изменений для отладки
4. Обнаружение неожиданных эффектов модификации

Подсказка:

Разные уровни тестирования помогают снизить риск ошибок при runtime изменениях.

- **Unit тесты** — Проверка отдельных функций на корректность
- **Integration тесты** — Контроль взаимодействия между модулями после изменений
- **Логирование изменений** — Фиксация динамических изменений для отладки
- **Проверка состояния объектов после изменений** — Обнаружение неожиданных эффектов модификации

Шаг 11:

Миссия 11

Расположите шаги безопасного применения hot reload:

1. Создание резервной копии текущего состояния модуля
2. Применение изменений в тестовой среде
3. Проверка корректности работы после изменений
4. Обновление модуля в рабочем приложении

Подсказка:

Подумайте: сначала нужно защититься, затем проверить, и только после этого внедрять.

Ответ:

- Создание резервной копии текущего состояния модуля
- Применение изменений в тестовой среде
- Проверка корректности работы после изменений
- Обновление модуля в рабочем приложении

Шаг 12:

Миссия 12

Расположите шаги безопасного monkey patching:

1. Создать локальную копию функции
2. Внести изменения в копию
3. Присвоить новое имя оригинальной функции
4. Протестировать изменения в изолированной среде

Подсказка:

Сначала изолируем изменения, потом заменяем оригинал и тестируем.

Ответ:

- Создать локальную копию функции
- Внести изменения в копию
- Присвоить новое имя оригинальной функции
- Протестировать изменения в изолированной среде

Шаг 13:

Миссия 13

Расположите действия при динамическом обновлении метода класса:

1. Создать новую версию метода
2. Сохранить старый метод на случай отката
3. Присвоить новый метод классу
4. Протестировать экземпляры класса

Подсказка:

Сначала подготавливаем новый код, сохраняем старый, затем меняем и проверяем.

Ответ:

- Создать новую версию метода
- Сохранить старый метод на случай отката
- Присвоить новый метод классу
- Протестировать экземпляры класса

Шаг 14:

Миссия 14

Расположите шаги безопасного обновления класса через monkey patch:

1. Определить целевой метод
2. Создать новую функцию с улучшениями
3. Назначить новый метод классу
4. Протестировать поведение всех экземпляров

Подсказка:

Подумайте о том, чтобы изменения были изолированы и протестированы до применения на всех объектах.

Ответ:

- Определить целевой метод
- Создать новую функцию с улучшениями
- Назначить новый метод классу
- Протестировать поведение всех экземпляров

Шаг 15:

Миссия 15

Расположите шаги отладки hot reload приложения:

1. Включить логирование изменений
2. Внести корректировки в код
3. Перезагрузить модуль через hot reload
4. Проверить новые функции через тесты

Подсказка:

Сначала фиксируем изменения, затем корректируем, обновляем и проверяем работу кода.

Ответ:

- Включить логирование изменений

- Внести корректировки в код
- Перезагрузить модуль через hot reload
- Проверить новые функции через тесты

🔮 Шаг 16:

Миссия 16

Расположите действия при обновлении глобальных переменных в runtime:

1. Создать резервные копии текущих значений
2. Внести изменения в тестовой среде
3. Применить изменения в рабочем приложении
4. Отследить влияние на все зависимости

🔮 Подсказка:

Сначала сохраняем старые значения, тестируем изменения и отслеживаем последствия перед внедрением.

Ответ:

- Создать резервные копии текущих значений
- Внести изменения в тестовой среде
- Применить изменения в рабочем приложении
- Отследить влияние на все зависимости

🔮 Шаг 17:

Миссия 17

Расположите последовательность безопасного обновления нескольких модулей в runtime:

1. Создать резервные копии всех модулей
2. Обновить один модуль и протестировать
3. Последовательно обновить оставшиеся модули
4. Протестировать всю систему на интеграцию

🔮 Подсказка:

Сначала сохраняем, потом поэтапно обновляем и проверяем интеграцию.

Ответ:

- Создать резервные копии всех модулей
- Обновить один модуль и протестировать
- Последовательно обновить оставшиеся модули
- Протестировать всю систему на интеграцию

📖 Урок 4 - Практика мини-интерпретатор Python

🔮 Шаг 01:

Миссия 1

Вы нашли баг в мини-интерпретаторе Python. Как лучше всего начать исправление?

Подсказка: Сначала важно понять, где именно проявляется ошибка.

Ответ:

- Изучить стек вызовов и локализовать ошибку
- Сразу переписать весь модуль

- Игнорировать баг и продолжать работу
- Добавить случайный print для отладки

🔗 Шаг 02:

Миссия 2

Вы хотите добавить новую функцию в мини-интерпретатор. Как безопаснее всего это сделать?

Подсказка: Добавление новой функции не должно ломать существующий код.

Ответ:

- **Создать отдельную функцию и написать тесты**
- Вставить код функции в середину существующего модуля
- Изменить существующие функции без тестов
- Добавить функцию, не проверяя работу интерпретатора

🔗 Шаг 03:

Миссия 3

При выполнении скрипта в интерпретаторе появляется NameError. Что правильнее всего сделать?

Подсказка: Ошибки NameError связаны с отсутствующими переменными или неправильными именами.

Ответ:

- **Проверить правильность написания переменной и область видимости**
- Игнорировать ошибку, она сама исчезнет
- Закомментировать все строки, кроме одной
- Удалить интерпретатор и поставить новый

🔗 Шаг 04:

Миссия 4

Вы хотите оптимизировать выполнение цикла в интерпретаторе. Что стоит сделать в первую очередь?

Подсказка: Оптимизация часто начинается с анализа текущего кода и выявления узких мест.

Ответ:

- **Проанализировать сложность текущего алгоритма и поискать узкие места**
- Удалять все циклы подряд
- Добавлять многопоточность без анализа
- Игнорировать производительность, пока код работает

🔗 Шаг 05:

Миссия 5

Вы хотите понять, как интерпретатор Python выполняет сложное выражение. Как лучше это сделать?

Подсказка: Иногда помогает разложение выражения на части и анализ каждого шага.

Ответ:

- **Разбить выражение на отдельные части и проанализировать байткод**
- Сразу переписать выражение в одну строку
- Использовать eval без проверки

- Пропустить разбор выражения

Шаг 06:

Миссия 6

Вы изучаете, как мини-интерпретатор хранит значения переменных. Что стоит проверить первым?

Подсказка: Обычно переменные хранятся в словарях локальных и глобальных областей видимости.

Ответ:

- **Посмотреть словари `locals()` и `globals()`**
- Искать переменные в случайных файлах
- Удалять переменные без анализа
- Игнорировать структуру хранения переменных

Шаг 07:

Миссия 7

Сопоставьте термины мини-интерпретатора с их определениями:

Подсказка: Подумайте, как Python хранит информацию и выполняет команды.

Ответ:

- **Bytecode** — Набор инструкций, которые выполняет интерпретатор Python.
- **Stack** — Структура данных для хранения промежуточных значений во время выполнения.
- **Frame** — Объект, содержащий локальные переменные и состояние выполнения функции.
- **Globals** — Словарь, содержащий глобальные переменные программы.

Шаг 08:

Миссия 8

Сопоставьте типы ошибок интерпретатора с их объяснением:

Подсказка: Ошибки помогают понять, что пошло не так в коде.

Ответ:

- **SyntaxError** — Ошибка, возникающая при неправильном синтаксисе кода.
- **NameError** — Ошибка, когда используется неопределённая переменная.
- **TypeError** — Ошибка, когда операция выполняется над несовместимыми типами.
- **IndexError** — Ошибка при обращении к элементу вне диапазона списка.

Шаг 09:

Миссия 9

Сопоставьте команды интерпретатора с их функцией:

Подсказка: Команды байткода выполняют разные действия — подумайте, что делает каждая.

Ответ:

- **LOAD_CONST** — Загружает константу на стек.
- **STORE_NAME** — Сохраняет значение из стека в переменной.
- **BINARY_ADD** — Складывает два значения с вершины стека.
- **RETURN_VALUE** — Возвращает значение из функции.

🔗 Шаг 10:

Миссия 10

Сопоставьте методы отладки с их описанием:

Подсказка: Подумайте, как можно проверить состояние интерпретатора на каждом шаге.

Ответ:

- **dis.dis** — Показывает разбор Python-кода на инструкции байткода.
- **pdb.set_trace** — Останавливает выполнение программы для пошаговой отладки.
- **print** — Выводит текущее значение переменной.
- **locals()** — Возвращает словарь локальных переменных текущей функции.

🔗 Шаг 11:

Миссия 11

Сопоставьте термин с его назначением в мини-интерпретаторе:

Подсказка: Подумайте, как каждая структура помогает интерпретатору работать.

Ответ:

- **Code Object** — Содержит байткод функции и информацию о переменных.
- **Interpreter Loop** — Цикл, выполняющий инструкции байткода.
- **Exception Handling** — Обработка ошибок во время выполнения.
- **Function Call** — Создание нового фрейма и запуск кода функции.

🔗 Шаг 12:

Миссия 12

Сопоставьте ключевые шаги выполнения скрипта с их описанием:

Подсказка: Подумайте о последовательности действий интерпретатора от кода до результата.

Ответ:

- **Парсинг** — Преобразование исходного кода в абстрактное синтаксическое дерево.
- **Компиляция** — Преобразование AST в байткод.
- **Выполнение** — Интерпретация байткода интерпретатором.
- **Возврат результата** — Возврат значения пользователю или программе.

🔗 Шаг 13:

Миссия 13

Расположите этапы создания мини-интерпретатора в правильном порядке:

Подсказка: Подумайте, как код превращается в результат шаг за шагом.

Ответ:

- Разбор исходного кода (парсинг)
- Компиляция AST в байткод
- Запуск интерпретатора и выполнение байткода
- Возврат результата пользователю

🔗 Шаг 14:

Миссия 14

Расположите шаги отладки скрипта в интерпретаторе:

Подсказка: Думайте, что проверяется раньше всего, а что позже.

Ответ:

- Проверка синтаксиса
- Использование print для локальных значений
- Пошаговая отладка с pdb
- Исправление найденных ошибок

Шаг 15:

Миссия 15

Расположите шаги анализа производительности интерпретатора:

Подсказка: Начинаем с идентификации узких мест, затем улучшаем их.

Ответ:

- Сбор данных о времени выполнения
- Идентификация узких мест
- Оптимизация алгоритмов
- Перепроверка результатов после оптимизации

Шаг 16:

Миссия 16

Расположите шаги обработки исключений в интерпретаторе:

Подсказка: Сначала нужно поймать ошибку, потом обработать, и только потом продолжить выполнение.

Ответ:

- Ловим исключение
- Обработка исключения
- Возврат контролю выполнения программы
- Логирование и уведомление

Урок 5 - Финал Python, который пишет, изменяет и оптимизирует себя

Шаг 01:

Миссия 1

Вы хотите, чтобы ваш скрипт мог динамически менять свой код. Что стоит проверить в первую очередь?

Подсказка: Начните с анализа того, где и как код может изменяться безопасно.

Ответ:

- **Какие переменные и функции будут изменяться динамически**
- Удалить все старые функции
- Использовать eval наугад
- Игнорировать потенциальные ошибки

Шаг 02:

Миссия 2

Вы хотите, чтобы скрипт создавал функции на лету. Как лучше всего это сделать?

Подсказка: Используйте возможности Python по работе с функциями как объектами.

Ответ:

- **Создавать функции через ``def`` внутри другой функции или ``exes`` с осторожностью**
- Прямо редактировать файл скрипта на диске
- Удалять старые функции перед созданием новых
- Не проверять синтаксис перед созданием

Шаг 03:

Миссия 3

Вы хотите, чтобы интерпретатор автоматически оптимизировал код. С чего начать?

Подсказка: Оптимизация начинается с анализа работы текущего кода.

Ответ:

- **Проанализировать узкие места с помощью профилировщика**
- Сразу переписать весь код
- Игнорировать результаты тестов
- Удалить функции, которые кажутся медленными

Шаг 04:

Миссия 4

Вы хотите, чтобы код мог сам изменять свои переменные в зависимости от условий. Как безопасно это реализовать?

Подсказка: Важно ограничить область действия таких изменений.

Ответ:

- **Использовать локальные структуры данных или словари для хранения изменяемых значений**
- Менять глобальные переменные без контроля
- Удалять переменные перед изменением
- Использовать ``eval`` без проверок

Шаг 05:

Миссия 5

Вы хотите, чтобы скрипт создавал и выполнял функции в рантайме. Как лучше всего проверить безопасность?

Подсказка: Проверьте, что выполняемый код не нарушает критические условия.

Ответ:

- **Тестировать каждую сгенерированную функцию на отдельном контролируемом примере**
- Выполнять код напрямую без тестов
- Удалять старые функции перед проверкой
- Использовать глобальный `exes` без проверки

Шаг 06:

Миссия 6

Вы хотите, чтобы интерпретатор мог оптимизировать собственный байткод. Как начать процесс?

Подсказка: Сначала изучите текущий байткод, прежде чем вносить изменения.

Ответ:

- **Использовать модуль ``dis`` для анализа текущего байткода**
- Переписать байткод вручную без анализа
- Игнорировать тесты и запустить изменения сразу
- Удалить старый код перед анализом

Шаг 07:

Миссия 7

Сопоставьте термины, связанные с динамическим кодом, с их определением:

Подсказка: Подумайте, как Python работает с кодом как с объектами.

Ответ:

- **exec** — Выполняет строку Python-кода в текущем пространстве имен.
- **eval** — Выполняет выражение Python и возвращает результат.
- **types.FunctionType** — Создает функцию динамически из кода и глобальных переменных.
- **dis** — Разбирает объект функции на байткод и инструкции интерпретатора.

Шаг 08:

Миссия 8

Сопоставьте методы оптимизации кода с их описанием:

Подсказка: Подумайте, какие изменения ускоряют выполнение, а какие упрощают структуру.

Ответ:

- **Профилирование** — Сбор информации о времени выполнения каждой функции.
- **Кэширование** — Сохранение результатов дорогостоящих вычислений для повторного использования.
- **Векторизация** — Замена циклов на операции с массивами для ускорения.
- **Удаление мертвого кода** — Удаление неиспользуемых функций и переменных для упрощения кода.

Шаг 09:

Миссия 9

Сопоставьте типы ошибок динамического кода с их причиной:

Подсказка: Ошибки могут появляться в момент исполнения, а не компиляции.

Ответ:

- **SyntaxError** — Возникает, если строка кода неверно оформлена.
- **NameError** — Используется переменная или функция, которая не определена.
- **TypeError** — Операция применяется к объекту неподходящего типа.
- **RuntimeError** — Ошибка возникает во время исполнения, часто при неправильном состоянии программы.

Шаг 10:

Миссия 10

Сопоставьте шаги динамического изменения функции с их описанием:

Подсказка: Подумайте, что выполняется сначала, а что потом.

Ответ:

- **Создание функции через `types.FunctionType`** — Генерация функции из байткода и словаря глобальных переменных.
- **Присвоение функции переменной** — Добавление новой функции в пространство имен.
- **Вызов функции** — Исполнение тела функции с аргументами.
- **Анализ результата** — Проверка возвращаемого значения и поведения функции.

Шаг 11:

Миссия 11

Сопоставьте объекты интерпретатора с их назначением:

Подсказка: Каждый объект управляет отдельным аспектом выполнения кода.

Ответ:

- **Frame** — Содержит локальные переменные и состояние выполнения функции.
- **Stack** — Хранит промежуточные значения во время исполнения инструкций байткода.
- **Code Object** — Содержит байткод функции и информацию о переменных.
- **Globals** — Словарь, содержащий глобальные переменные программы.

Шаг 12:

Миссия 12

Сопоставьте команды байткода с их действием:

Подсказка: Каждая инструкция изменяет стек определенным образом.

Ответ:

- **LOAD_CONST** — Помещает константу на вершину стека.
- **STORE_NAME** — Сохраняет значение из стека в переменной.
- **BINARY_ADD** — Складывает два значения с вершины стека.
- **RETURN_VALUE** — Возвращает значение из функции вызывающему коду.

Шаг 13:

Миссия 13

Расположите шаги динамической генерации и выполнения функции:

Подсказка: Подумайте, что выполняется сначала, а что потом.

Ответ:

- Создание байткода функции
- Создание объекта функции через `types.FunctionType`
- Присвоение функции переменной
- Вызов функции и получение результата

Шаг 14:

Миссия 14

Расположите шаги безопасной оптимизации кода интерпретатором:

Подсказка: Сначала анализируем, потом оптимизируем, потом проверяем.

Ответ:

- Профилирование кода
- Определение узких мест

- Внесение изменений и оптимизаций
- Перепроверка тестами

Шаг 15:

Миссия 15

Расположите шаги обработки ошибок в динамически генерируемом коде:

Подсказка: Сначала ловим ошибку, потом обрабатываем, потом логируем.

Ответ:

- Ловим исключение
- Обработка исключения
- Логирование ошибки
- Продолжение выполнения программы

Шаг 16:

Миссия 16

Расположите шаги отладки динамически созданных функций:

Подсказка: Сначала тестируем простые случаи, потом сложные.

Ответ:

- Проверка базовых вызовов функции
- Тестирование с различными аргументами
- Отслеживание состояния локальных переменных
- Исправление найденных проблем